

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**CogMem: Khung bộ nhớ hội thoại dài hạn dựa trên
Khoa học nhận thức**

Lê Minh Triết

triet.lm220045@sis.hust.edu.vn

Program: IT1

Supervisor: Cao Tuấn Dũng

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**CogMem: Khung bộ nhớ hội thoại dài hạn dựa trên
Khoa học nhận thức**

Lê Minh Triết

triet.lm220045@sis.hust.edu.vn

Program: IT1

Supervisor: Cao Tuấn Dũng _____

Signature

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENT

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc đến Thầy Cao Tuấn Dũng — người đã trực tiếp hướng dẫn và hỗ trợ tôi trong suốt quá trình thực hiện đồ án tốt nghiệp. Thầy không chỉ truyền đạt kiến thức chuyên môn mà còn là người thầy định hướng cho tôi về cách tiếp cận vấn đề một cách khoa học và có phương pháp, cũng như là người khơi gợi những ý tưởng đầu tiên của tôi về đồ án này.

Tôi cũng xin cảm ơn các thầy cô trong Khoa Kỹ thuật Máy tính, Trường Đại học Bách khoa Hà Nội, đã trang bị cho tôi nền tảng kiến thức vững chắc trong suốt bốn năm học tại đây.

TÓM TẮT

Các tác nhân AI (AI Agent) hiện tại, đặc biệt là các tác nhân hội thoại không chỉ cần một cửa sổ ngữ cảnh lớn trong các đoạn hội thoại dài, mà còn phải lưu giữ thông tin người dùng qua nhiều phiên, nối lại các thành phần phân tán, và trả lời những câu hỏi phụ thuộc vào thời gian, sở thích, quan hệ nhân quả hoặc suy luận nhiều bước. Các hướng tiếp cận hiện tại chỉ dựa vào truy vấn dựa trên ngữ nghĩa hoặc tóm tắt, nén thông tin thường chưa đủ, vì chúng dễ làm phẳng các loại thông tin khác nhau và làm mất dấu các chi tiết gốc quan trọng.

Đề án này trình bày **CogMem**, một hướng tiếp cận bộ nhớ dài hạn cho các tác nhân hội thoại theo định hướng cấu trúc hóa trí nhớ (memory) dựa trên khoa học nhận thức. Ở mức tổng quan, hệ thống hướng tới ba mục tiêu: biểu diễn thông tin theo nhiều nhóm ký ức có chức năng khác nhau, giữ được liên hệ với các hội thoại gốc khi cần trả lời chi tiết, và ưu tiên những tín hiệu phù hợp hơn với ý định của truy vấn thay vì xem mọi câu hỏi là như nhau.

Các đánh giá trong báo cáo cho thấy hướng tiếp cận này có chiều hướng cải thiện tích cực ở những tình huống cần kết hợp tín hiệu qua nhiều phiên hoặc cần phân biệt rõ hơn giữa các dạng thông tin cá nhân khác nhau. Điểm đáng chú ý của CogMem là nỗ lực kết hợp giữa biểu diễn giàu quan hệ hơn và chiến lược truy vấn có chọn lọc hơn, từ đó nâng cao chất lượng ngữ cảnh đưa vào quá trình trả lời của AI.

Tuy vậy, những điểm cần cải thiện vẫn còn rõ ở các truy vấn đòi hỏi suy luận thời gian thật chính xác, liệt kê đầy đủ nhiều mục, hoặc kiểm chứng sâu hơn vai trò của các tín hiệu trong các đoạn hội thoại thực tế. Vì vậy, CogMem nên được xem như một bước tiến có định hướng rõ ràng cho bộ nhớ dài hạn hơn là một lời giải đã hoàn tất.

Lê Minh Triết

triet.lm220045@sis.hust.edu.vn

MỤC LỤC

CHƯƠNG 1. ĐẶT VẤN ĐỀ	1
1.1 Bối cảnh của bài toán.....	1
1.2 Vấn đề cần giải quyết	1
1.2.1 Bộ nhớ dài hạn của các tác nhân hội thoại không chỉ là lưu lịch sử..	1
1.2.2 Giới hạn của các giải pháp phổ biến.....	2
1.3 Mục tiêu và hướng tiếp cận của đề án	2
1.4 Phạm vi của báo cáo	3
1.5 Cấu trúc của báo cáo.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC NGHIÊN CỨU LIÊN QUAN	5
2.1 Nền tảng của tác nhân hội thoại hiện đại	5
2.1.1 Mô hình ngôn ngữ lớn: kiến trúc, huấn luyện và năng lực cốt lõi	5
2.1.2 Giới hạn của cửa sổ ngữ cảnh và vấn đề trí nhớ.....	6
2.1.3 Tác nhân AI: từ chatbot đơn lượt đến hệ thống đa năng.....	6
2.1.4 Bộ nhớ ngoài cho các tác nhân hội thoại: các mô hình tổ chức phổ biến	7
2.1.5 Luồng xử lý có bộ nhớ trong tác nhân hội thoại.....	8
2.2 Cơ sở lý thuyết cho việc thiết kế bộ nhớ	8
2.2.1 Khoa học nhận thức và các hệ con của bộ nhớ dài hạn	8
2.2.2 Đồ thị tri thức và truy vấn nhiều bước	10
2.2.3 Xếp hạng lại bằng CrossEncoder trong truy vấn bộ nhớ.....	11
2.3 Các hướng nghiên cứu liên quan	12
2.3.1 Các hệ thống lưu trữ phẳng hoặc dùng kho lưu trữ vector.....	12
2.3.2 Các hệ thống quản lý ngữ cảnh và phân tầng bộ nhớ.....	12
2.3.3 Các hệ thống dựa trên đồ thị tri thức	12

2.3.4 Bộ nhớ cho tác nhân AI và kịch bản sử dụng công cụ	13
2.4 Các bộ dữ liệu đánh giá bộ nhớ dài hạn.....	14
2.4.1 LongMemEval: đánh giá trí nhớ cá nhân qua hội thoại dài	14
2.4.2 LoCoMo: suy luận chuỗi bằng chứng trên hội thoại dài	16
2.5 Khoảng trống nghiên cứu mà CogMem hướng tới.....	18
CHƯƠNG 3. ĐỀ XUẤT COGMEM: KHUNG BỘ NHỚ DÀI HẠN DỰA TRÊN KHOA HỌC NHẬN THỨC.....	19
3.1 Tổng quan hệ thống bộ nhớ CogMem	19
3.1.1 Phạm vi và giả định triển khai	20
3.2 Kiến trúc xử lý ba pipeline của CogMem	21
3.2.1 Luồng ghi nhớ thông tin mới	21
3.2.2 Luồng truy vấn thông tin cũ	22
3.2.3 Luồng tổng hợp câu trả lời	22
3.3 Đồ thị tri thức từ khoa học nhận thức.....	23
3.3.1 Động lực nhận thức và cấu trúc đa hệ thống.....	23
3.3.2 Các mạng phân định.....	23
3.3.3 Khác biệt cốt lõi giữa Habit và Action-Effect.....	29
3.3.4 Bản chất ngữ nghĩa của các liên kết đồ thị	30
3.4 Biểu diễn song song: ghi nhớ tóm tắt và dấu vết nguyên văn.....	31
3.4.1 Vấn đề nén dữ liệu có mất thông tin	31
3.4.2 Fuzzy Trace Theory và giải pháp hai lớp	32
3.5 Truy vấn đa kênh và kết hợp bằng chứng.....	33
3.5.1 Vì sao không dùng một kênh duy nhất	33
3.5.2 Bốn kênh truy vấn song song	34
3.5.3 Kết hợp bằng chứng bằng weighted RRF.....	34
3.5.4 Xếp hạng lại bằng CrossEncoder	35

3.6 Truy vấn trên đồ thị tri thức	35
3.6.1 Nhược điểm của toán tử MAX và khái niệm Episodic Buffer	35
3.6.2 Lan truyền tín hiệu tích lũy	36
3.6.3 Cơ sở thần kinh học và ba cơ chế bảo vệ chống nhiễu	38
3.7 Định tuyến truy vấn thích ứng (Adaptive Query Routing)	39
3.7.1 Attentional Selection và vấn đề trọng số cố định	39
3.7.2 Phân loại truy vấn và kết hợp theo nội dung câu hỏi	39
3.8 Sinh câu trả lời	41
3.8.1 Tạo ngữ cảnh cho mô hình sinh	41
3.8.2 Đầu ra phục vụ đánh giá và truy vết	42
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	43
4.1 Thiết kế thực nghiệm	43
4.1.1 Luồng đánh giá tổng quát	43
4.1.2 Chắt lọc dữ liệu	44
4.1.3 CogMem Bench: bộ dữ liệu tự xây dựng để kiểm định các loại nút .	45
4.1.4 Các thước đo đánh giá	48
4.1.5 Phương pháp chấm điểm: LLM làm giám khảo (LLM-as-a-Judge) .	49
4.1.6 Các cấu hình so sánh	51
4.2 Kết quả trên LongMemEval	51
4.2.1 So sánh tổng quan các cấu hình	52
4.2.2 Đối chứng SUM và MAX trên kênh đồ thị	53
4.2.3 Hiệu năng nạp dữ liệu	53
4.3 Kết quả trên LoCoMo	54
4.3.1 Kết quả tổng quan	54
4.3.2 Phân tích theo nhóm câu hỏi	55
4.3.3 Phân tích lỗi còn tồn tại	55

4.4 Đánh giá định tính trên CogMem Bench	56
4.4.1 Node ý định: khi kế hoạch chưa hoàn thành là thông tin quyết định.	56
4.4.2 Node hành động-kết quả: khi cần nhớ quan hệ điều kiện, hành động và kết quả.....	57
4.4.3 Ý nghĩa của các hình minh họa CogMem Bench	58
4.5 Tổng kết thực nghiệm.....	59
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
5.1 Kết luận	61
5.1.1 Tổng kết đóng góp.....	61
5.1.2 Kết quả chính	61
5.1.3 Hạn chế.....	62
5.1.4 Hướng phát triển	63
TÀI LIỆU THAM KHẢO.....	64
CHƯƠNG A. Knowledge Extraction Prompts (Retain Pipeline).....	64
A.0.1 A.1. Prompt Trích xuất Pass 1 (Fact Extraction)	64
A.0.2 A.2. Prompt Tinh chỉnh Pass 2 (Persona-Focused Revision)	65
CHƯƠNG B. Evaluation and Reflection Prompts	66
B.0.1 B.1. Prompt Tổng hợp Câu trả lời (Reflection & Generation).....	66
B.0.2 B.2. Prompt Chấm điểm Tự động (LLM-as-a-Judge).....	66

DANH MỤC HÌNH VẼ

Figure 3.1	Tổng quan luồng xử lý CogMem: hội thoại được chuyển thành đồ thị bộ nhớ, sau đó hệ thống truy vấn đa kênh và sinh câu trả lời dựa trên bằng chứng.	20
Figure 3.2	Sáu mạng bộ nhớ của CogMem, tương ứng với sáu loại thông tin có vai trò khác nhau trong truy vấn hội thoại dài hạn. . . .	24
Figure 3.3	Một chuỗi nhật ký nhiều tuần phù hợp hơn để kiểm tra Habit Network so với benchmark chỉ hỏi fact đơn lẻ.	26
Figure 3.4	Habit Memory và Habit Network	27
Figure 3.5	Intention Lifecycle	28
Figure 3.6	Action-effect phù hợp với trace AI agent thật, nơi hệ thống cần nhớ quan hệ giữa điều kiện lỗi, hành động công cụ và kết quả quan sát được.	29
Figure 3.7	Theory of Event Coding và Action-Effect Network	29
Figure 3.8	Phân biệt Habit và Action-Effect	30
Figure 3.9	Bảy loại liên kết trong CogMem	31
Figure 3.10	Schema hai lớp cho một đơn vị bộ nhớ	33
Figure 3.11	Fuzzy Trace Theory và Raw Snippet	33
Figure 3.12	Episodic Buffer của Baddeley	36
Figure 3.13	Minh họa trực quan sự khác biệt giữa MAX và SUM trên cùng một cấu trúc đồ thị	37
Figure 3.14	Ba cơ chế bảo vệ chu trình	39
Figure 3.15	Adaptive Query Routing	40
Figure 3.16	Attentional Selection và Adaptive Routing	41
Figure 4.1	Luồng đánh giá ngoại tuyến của CogMem trên các bộ dữ liệu hội thoại dài	44
Figure 4.2	Kịch bản ý định trong CogMem Bench: ngân hàng đầy đủ (trái) giữ cụm ý định về composting, trong khi ngân hàng loại bỏ ý định (phải) mất biểu diễn của đáp án đúng và bị kéo sang phương án gây nhiễu về nước mưa.	57
Figure 4.3	Kịch bản hành động-kết quả trong CogMem Bench: ngân hàng đầy đủ (trái) lưu trọn vẹn bộ ba nhân quả, còn ngân hàng loại bỏ (phải) chỉ còn các fact lân cận và không thể khôi phục được kết quả 200 sau retry.	58

DANH MỤC BẢNG BIỂU

Table 2.1	So sánh khái quát các hướng xây dựng bộ nhớ dài hạn liên quan	14
Table 3.1	Các loại fact cốt lõi và siêu dữ liệu chuyên biệt	25
Table 3.2	Vòng đời của intention và ý nghĩa các liên kết chuyển trạng thái	27
Table 3.3	So sánh HINDSIGHT (MAX) và CogMem (SUM)	37
Table 3.4	Trọng số RRF thích ứng theo loại truy vấn trong phiên bản triển khai hiện tại	40
Table 4.1	Tóm tắt dữ liệu đã chất lọc dùng trong đánh giá	45
Table 4.2	Các nhóm cấu hình so sánh chính và mục tiêu	51
Table 4.3	LongMemEval: so sánh các cấu hình truy vấn và loại node . .	52
Table 4.4	Kênh đồ thị trên LongMemEval: so sánh SUM và MAX . . .	53
Table 4.5	So sánh hiệu năng nạp dữ liệu trung bình cho một cuộc hội thoại	54
Table 4.6	LoCoMo: cấu hình cuối so với baseline trước đó	54
Table 4.7	LoCoMo: phân rã kết quả theo nhóm câu hỏi	55

CHƯƠNG 1. ĐẶT VẤN ĐỀ

1.1 Bối cảnh của bài toán

Trong vài năm gần đây, sự phát triển của các mô hình ngôn ngữ lớn (LLM) đã làm thay đổi đáng kể cách con người tương tác với hệ thống trí tuệ nhân tạo. Một trợ lý AI ngày nay không còn chỉ làm nhiệm vụ trả lời câu hỏi ngắn, mà còn được kỳ vọng hỗ trợ lập kế hoạch, đồng hành trong việc học tập, tư vấn mua sắm, theo dõi công việc và duy trì mối quan hệ với người dùng trong thời gian dài.

Kỳ vọng đó làm xuất hiện một yêu cầu cốt lõi: hệ thống phải nhớ được các thông tin quan trọng về người dùng qua nhiều phiên hội thoại, chứ không chỉ trong phạm vi một phiên ngắn hạn, riêng lẻ.

Trong thực tế, phần lớn các tác nhân hội thoại hiện tại vẫn phụ thuộc rất nhiều vào ngữ cảnh ở ngay phiên hiện tại. Khi lịch sử trò chuyện kéo dài qua nhiều phiên, hệ thống buộc phải cắt bớt nội dung cũ, tóm tắt lại, hoặc chỉ giữ những đoạn được đánh giá là quan trọng nhất. Những cách làm này tuy giúp mô hình tiếp tục vận hành, nhưng đồng thời cũng làm mất dần những chi tiết tưởng như nhỏ mà lại quan trọng với người dùng: một kế hoạch còn dang dở, một thói quen lặp lại, một lần thay đổi quyết định, hay một mối quan hệ nhân quả từng được nhắc thoáng qua trong một phiên từ rất lâu.

1.2 Vấn đề cần giải quyết

1.2.1 Bộ nhớ dài hạn của các tác nhân hội thoại không chỉ là lưu lịch sử

Thoạt nhìn, ta có thể nghĩ rằng bộ nhớ dài hạn chỉ đơn giản là lưu toàn bộ các đoạn chat cũ và tìm lại khi cần. Tuy nhiên, với các hội thoại kéo dài hàng chục phiên hoặc hàng trăm nghìn từ, cách làm đó nhanh chóng bộc lộ hạn chế. Một truy vấn tưởng như đơn giản, chẳng hạn “người dùng đã từng định làm gì nhưng chưa làm?”, có thể đòi hỏi hệ thống phải nối một sự thật ở phiên đầu, một lần xác nhận ở phiên giữa, và một chi tiết phủ định ở phiên cuối. Nếu chỉ dựa vào truy vấn theo mức độ giống nhau về từ ngữ và ngữ nghĩa, hệ thống rất dễ bỏ lỡ mối liên hệ này.

Khó khăn còn lớn hơn khi câu hỏi không chỉ yêu cầu nhắc lại một chi tiết đơn lẻ mà cần tái dựng một trạng thái nhận thức. Ví dụ, phát biểu “tôi đang định học Rust” khác về bản chất với “tôi đã học Rust”, dù hai câu này có thể cùng nhắc đến một chủ đề. Tương tự, “người dùng thường làm gì trước cuộc họp” khác với “người dùng đã làm gì trong một buổi sáng cụ thể”, và “làm cách nào để khắc phục lỗi này” khác với “lỗi này đã từng xảy ra”. Nếu mọi nội dung đều bị nén về cùng một

kiểu biểu diễn duy nhất, sự khác biệt nói trên sẽ bị làm mờ đi.

1.2.2 Giới hạn của các giải pháp phổ biến

Hai hướng tiếp cận đang được dùng phổ biến là lưu trữ văn bản trong kho lưu trữ (thường ở dưới dạng các vector), hoặc để mô hình trích xuất các sự thực (fact) từ hội thoại rồi lập chỉ mục trên các fact đó. Cả hai hướng đều có giá trị thực tế, nhưng chúng cũng có những điểm yếu dễ nhận thấy:

- **Mất mát chi tiết do nén:** Khi hội thoại bị rút gọn thành fact ngắn, nhiều thông tin nguyên bản như con số, tên riêng, sắc thái cảm xúc hoặc ngữ cảnh trích dẫn rất khó được phục hồi đầy đủ.
- **Biểu diễn chưa đủ chuyên biệt:** Các fact về tri thức khách quan, trải nghiệm cá nhân, thói quen, kế hoạch và quy luật nhân quả thường bị đặt chung vào một không gian biểu diễn, dẫn tới truy vấn chưa đúng trọng tâm.
- **Truy vấn đa bước yếu:** Những câu hỏi cần nối nhiều dấu vết nhỏ qua nhiều phiên thường không được xử lý tốt nếu hệ thống chỉ giữ một đường suy luận qua với các bằng chứng mạnh nhất.
- **Định tuyến truy vấn:** Một câu hỏi hỏi “khi nào” cần logic truy vấn khác với câu hỏi hỏi “vì sao” hoặc “thường làm gì”. Dùng cùng một cách truy vấn cho mọi loại làm giảm hiệu quả của từng kênh.

Những hạn chế trên cho thấy vấn đề không chỉ nằm ở việc mô hình có “nhớ nhiều hơn” hay không, mà nằm ở chỗ hệ thống có tổ chức trí nhớ đúng cách hay không. Một bộ nhớ dài hạn hữu ích cần vừa giữ được bằng chứng gốc khi cần, vừa cho phép biểu diễn sự khác nhau giữa tri thức, kinh nghiệm, kế hoạch, thói quen và quan hệ nhân quả.

1.3 Mục tiêu và hướng tiếp cận của đề án

Từ các quan sát trên, đề án này đề xuất **CogMem**, một khung bộ nhớ dài hạn cho tác nhân hội thoại được thiết kế theo hướng kết hợp *khoa học nhận thức* với *đồ thị tri thức*. Ý tưởng xuất phát là: bộ nhớ người không phải một kho lưu trữ đồng nhất, mà là tập hợp của nhiều hệ lưu trữ con cùng tồn tại, mỗi hệ lưu trữ con phù hợp với một loại câu hỏi và một loại suy luận khác nhau. Nếu muốn xây dựng một bộ nhớ dài hạn đáng tin cậy cho tác nhân hội thoại, tác nhân AI hay trợ lý cá nhân, hệ thống cũng cần một cấu trúc biểu diễn phân hóa tương tự.

Theo hướng đó, CogMem theo đuổi bốn đóng góp chính:

1. **Đồ thị bộ nhớ dựa trên khoa học nhận thức:** Thay vì chỉ lưu một loại fact chung, CogMem phân chia trí nhớ thành các nhóm biểu diễn như tri thức

khách quan, sự kiện cá nhân, quan điểm, thói quen, ý định tương lai và quan hệ hành động -kết quả.

2. **Lưu song song biểu diễn nén và hội thoại gốc:** Mỗi đơn vị bộ nhớ không chỉ chứa fact đã rút gọn mà còn giữ lại một đoạn trích nguyên văn để hỗ trợ trả lời có căn cứ.
3. **Lan truyền tín hiệu theo hướng cộng dồn:** Thay vì chỉ chọn một đường có tín hiệu mạnh nhất trong đồ thị, hệ thống cho phép nhiều đường trích dẫn cùng đóng góp để tăng khả năng truy vấn nhiều bước.
4. **Định tuyến truy vấn theo bản chất câu hỏi:** Cùng là truy vấn bộ nhớ, nhưng câu hỏi về thời gian, sở thích, nguyên nhân hay kế hoạch tương lai cần được ưu tiên những loại bằng chứng khác nhau.

Cần nhấn mạnh rằng đồ án không giả định các thành phần mới là một sự bảo đảm cho việc mọi bộ dữ liệu chuẩn đều tăng điểm đồng đều. Toàn bộ báo cáo được trình bày theo tinh thần thực nghiệm cẩn thận: kiến trúc được đề xuất vì nó hợp lý về mặt biểu diễn và truy vấn, còn mức độ hiệu quả thực tế của từng thành phần sẽ được kiểm chứng riêng rẽ trên các bộ dữ liệu chuẩn và trên bộ dữ liệu tự xây dựng của dự án.

1.4 Phạm vi của báo cáo

Báo cáo tập trung vào bài toán bộ nhớ dài hạn cho **tác nhân hội thoại định hướng cá nhân hóa**. Phạm vi này bao gồm:

- lưu trữ và truy hồi thông tin người dùng qua nhiều phiên;
- hỗ trợ câu hỏi về sự kiện đã xảy ra, sở thích, thói quen, kế hoạch tương lai và quan hệ nhân quả;
- đánh giá trên các bộ dữ liệu hội thoại dài tiêu biểu và trên một benchmark phục vụ dự án.

Những nội dung ngoài phạm vi chính của báo cáo gồm: tối ưu hóa hạ tầng phân tán ở quy mô sản phẩm hoàn chỉnh, huấn luyện lại mô hình nền, hoặc phát triển đầy đủ một hệ thống điều phối tác vụ phức tạp ngoài bài toán bộ nhớ. Dù vậy, trong nhiều đoạn phân tích, báo cáo vẫn tham chiếu đến các bối cảnh tác nhân AI thực tế để làm rõ vì sao các cấu trúc được đề xuất có ý nghĩa.

1.5 Cấu trúc của báo cáo

Báo cáo được tổ chức lại thành năm chương chính và các phụ lục như sau:

- **Chương 1 – Đặt vấn đề:** trình bày bối cảnh, bài toán, động lực nghiên cứu và phạm vi của đồ án.

- **Chương 2 – Cơ sở lý thuyết và các nghiên cứu liên quan:** tổng hợp kiến thức nền tảng về mô hình ngôn ngữ lớn, tác nhân AI, bộ nhớ ngoài cho tác nhân hội thoại, khoa học nhận thức, đồ thị tri thức, các hệ thống bộ nhớ liên quan và hai bộ dữ liệu đánh giá chính.
- **Chương 3 – Đề xuất CogMem: Khung bộ nhớ dài hạn dựa trên khoa học nhận thức:** mô tả đầy đủ kiến trúc CogMem, luồng xử lý, các lựa chọn biểu diễn, cơ sở khoa học của từng thành phần và các cơ chế truy vấn chính.
- **Chương 4 – Kết quả thực nghiệm và đánh giá:** trình bày thiết kế thực nghiệm, cách chất lọc dữ liệu, các thước đo, các cấu hình so sánh, kết quả trên LongMemEval và LoCoMo, cùng phần đánh giá định tính trên CogMem Bench.
- **Chương 5 – Kết luận và hướng phát triển:** tổng kết đóng góp, những giới hạn hiện tại và các hướng mở rộng thực tế của nghiên cứu trong tương lai.

Sau khi làm rõ bài toán và mục tiêu của đề án, Chương 2 sẽ đi vào nền tảng lý thuyết và các nghiên cứu liên quan để xác định chính xác khoảng trống mà CogMem sẽ xử lý.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC NGHIÊN CỨU LIÊN QUAN

Chương này đặt nền tảng cho toàn bộ báo cáo. Trước khi mô tả CogMem, cần làm rõ các kiến thức cơ bản về: mô hình ngôn ngữ lớn vận hành ra sao, tác nhân AI hiện đại được xây dựng như thế nào, vì sao chatbot cần bộ nhớ ngoài, khoa học nhận thức và đồ thị tri thức gợi ý cấu trúc biểu diễn nào, các hệ thống liên quan đã giải quyết bài toán ra sao, và các bộ dữ liệu nào thực sự đo được năng lực bộ nhớ dài hạn.

2.1 Nền tảng của tác nhân hội thoại hiện đại

2.1.1 Mô hình ngôn ngữ lớn: kiến trúc, huấn luyện và năng lực cốt lõi

Mô hình ngôn ngữ lớn (Large Language Model – LLM) là thành phần trung tâm đứng sau phần lớn hệ thống hội thoại hiện nay. Về mặt kiến trúc, các LLM hiện đại hầu hết được xây dựng dựa trên kiến trúc Transformer do Vaswani và cộng sự đề xuất năm 2017. Điểm cốt lõi của Transformer là cơ chế tự chú ý (self-attention), cho phép mô hình tính toán mức độ liên quan giữa mọi cặp từ trong một chuỗi đầu vào cùng một lúc, thay vì phải xử lý tuần tự như các kiến trúc hồi quy trước đây. Chính cơ chế này đã mở đường cho việc huấn luyện trên lượng văn bản khổng lồ và cho ra đời các mô hình có khả năng hiểu ngữ cảnh rất dài.

Quá trình huấn luyện một LLM thường trải qua nhiều giai đoạn.

- **Tiền huấn luyện (pre-training):** mô hình được cho tiếp xúc với hàng nghìn tỉ từ văn bản từ internet, sách, bài báo khoa học và mã nguồn. Mục tiêu của giai đoạn này đơn giản là dự đoán từ tiếp theo trong một chuỗi, nhưng nó buộc mô hình phải học được một lượng lớn tri thức về ngữ pháp, sự kiện, quan hệ logic và những kiểu suy luận cơ bản.
- **Tinh chỉnh theo hướng dẫn (instruction fine-tuning):** mô hình được huấn luyện thêm trên các cặp câu hỏi–câu trả lời có chất lượng cao để học cách làm theo yêu cầu của người dùng.
- **Huấn luyện từ phản hồi của con người (RLHF – Reinforcement Learning from Human Feedback):** giúp mô hình sinh ra câu trả lời an toàn hơn, hữu ích hơn và phù hợp với mong đợi của người dùng hơn.

Nhờ quá trình huấn luyện nhiều tầng như vậy, LLM sở hữu những năng lực vượt xa việc chỉ sinh văn bản trôi chảy. Chúng có thể phân tích yêu cầu phức tạp, chia nhỏ bài toán thành các bước, gọi công cụ bên ngoài, tổng hợp thông tin từ nhiều nguồn, và thậm chí lập kế hoạch nhiều bước. Những năng lực này khiến LLM trở thành “bộ não” lý tưởng cho các hệ thống tác nhân AI (AI agent), nơi mô hình

không chỉ trả lời câu hỏi mà còn thực hiện hành động trong môi trường thực tế.

2.1.2 Giới hạn của cửa sổ ngữ cảnh và vấn đề trí nhớ

Dù năng lực của LLM đã tiên bộ vượt bậc, một giới hạn căn bản vẫn tồn tại: mỗi lần suy luận, mô hình chỉ có thể “nhìn thấy” một lượng văn bản hữu hạn được nạp vào cửa sổ ngữ cảnh (Context Window). Cửa sổ này, dù đã được mở rộng đáng kể từ vài nghìn từ lên hàng trăm nghìn từ ở các mô hình mới nhất, vẫn bị chặn bởi chi phí tính toán bậc hai của cơ chế tự chú ý: xử lý một ngữ cảnh dài gấp đôi đòi hỏi tài nguyên tính toán gấp bốn lần.

Hệ quả trong thực tế là khi lịch sử hội thoại kéo dài qua hàng chục phiên, hệ thống buộc phải lựa chọn: hoặc cắt bớt nội dung cũ, hoặc nén chúng thành tóm tắt, hoặc chỉ giữ lại những phần được đánh giá là quan trọng nhất. Mỗi lựa chọn đều đánh đổi một phần thông tin. Một câu hỏi tưởng như đơn giản, ví dụ “người dùng hiện còn đang theo đuổi kế hoạch nào?”, có thể phụ thuộc vào ít nhất ba loại tín hiệu nằm rải rác qua nhiều phiên: lần đầu nêu kế hoạch, một lần cập nhật tiến độ, và một câu phủ định cho biết việc đó vẫn chưa được hoàn thành. Nếu một trong các tín hiệu đó không còn nằm trong ngữ cảnh hiện tại, mô hình rất dễ suy luận sai hoặc trả lời dựa trên phỏng đoán.

Vấn đề này càng nghiêm trọng hơn trong các kịch bản cá nhân hóa, nơi người dùng kỳ vọng hệ thống nhớ được không chỉ những gì họ đã nói, mà còn cả những gì họ đã làm, những gì họ thích, những gì họ định làm và những gì họ đã từ bỏ. Đây không còn là bài toán “nhớ nhiều hơn”, mà là bài toán “nhớ đúng loại thông tin vào đúng thời điểm”.

2.1.3 Tác nhân AI: từ chatbot đơn lượt đến hệ thống đa năng

Khi LLM được tích hợp vào một vòng lặp tương tác với môi trường, chúng trở thành *tác nhân AI*. Khác với chatbot truyền thống chỉ nhận tin nhắn và trả lời, một tác nhân AI hiện đại thường vận hành theo chu trình: quan sát môi trường, suy nghĩ về bước tiếp theo, chọn và thực thi một hành động (có thể là gọi API, chạy lệnh, truy vấn cơ sở dữ liệu, mở tập tin), nhận kết quả, rồi tiếp tục chu trình cho đến khi hoàn thành nhiệm vụ.

Tác nhân hội thoại là một trường hợp đặc biệt của tác nhân AI, nơi môi trường chính là cuộc hội thoại với người dùng. Ở đây, bộ nhớ không chỉ giúp duy trì ngữ cảnh, mà còn cần lưu giữ các tín hiệu cá nhân hóa: sở thích, thói quen, lịch sử tương tác, và các mục tiêu dài hạn của người dùng. Việc thiết kế bộ nhớ cho tác nhân hội thoại vì thế đòi hỏi sự kết hợp giữa khả năng truy hồi nhanh, khả năng lưu trữ lâu dài và khả năng phân loại thông tin theo chức năng.

Một số framework tiêu biểu cho mô hình tác nhân bao gồm:

- **ReAct (Reasoning + Acting):** đan xen giữa suy luận và hành động, trong đó mô hình suy nghĩ kỹ trước khi quyết định gọi công cụ nào, rồi quan sát kết quả để điều chỉnh hướng suy luận tiếp theo.
- **Plan-and-Execute:** mô hình lập một kế hoạch tổng thể trước, sau đó thực thi từng bước, cho phép nhìn xa hơn và tránh bị lạc hướng bởi các kết quả trung gian.
- **Multi-agent collaboration:** nhiều tác nhân với vai trò khác nhau cùng phối hợp để giải quyết một nhiệm vụ phức tạp, mỗi tác nhân có bộ nhớ và mục tiêu riêng.

Trong tất cả các mô hình trên, một thành phần xuyên suốt và không thể thiếu là **bộ nhớ (memory)**. Tác nhân cần nhớ những gì đã xảy ra trong phiên hiện tại (trí nhớ ngắn hạn), những gì đã xảy ra trong các phiên trước (trí nhớ dài hạn), những quy luật hay mẫu hành vi đã được kiểm chứng, và những kế hoạch hay mục tiêu còn đang dang dở. Không có bộ nhớ, tác nhân sẽ liên tục lặp lại sai lầm, không thể học từ kinh nghiệm và không thể duy trì mối quan hệ lâu dài với người dùng.

2.1.4 Bộ nhớ ngoài cho các tác nhân hội thoại: các mô hình tổ chức phổ biến

Để vượt qua giới hạn của cửa sổ ngữ cảnh, một hướng tiếp cận đã trở thành tiêu chuẩn là *Retrieval-Augmented Generation* (RAG). Ý tưởng cốt lõi của RAG rất trực quan: thay vì buộc mô hình phải nhớ mọi thứ, hệ thống giữ một kho tài liệu bên ngoài và truy vấn những thành phần liên quan đến câu hỏi hiện tại trước khi sinh câu trả lời. Kho tài liệu này có thể là văn bản thô, các đoạn hội thoại cũ, các bản ghi sự kiện, hoặc các đơn vị tri thức đã được trích xuất và cấu trúc hóa.

Trong thực tế, có ba mô hình tổ chức bộ nhớ ngoài phổ biến:

1. **Kho lưu trữ vector (Vector Store):** mỗi đoạn văn bản được biểu diễn nhúng thành một vector và lưu trong cơ sở dữ liệu vector. Khi có câu hỏi, hệ thống tìm các vector gần nhất (theo độ tương đồng) và đưa văn bản tương ứng vào prompt. Mô hình này đơn giản, nhanh, nhưng không giữ được quan hệ giữa các mảnh thông tin.
2. **Bộ nhớ phân tầng:** thông tin được tổ chức thành nhiều mức, từ bộ nhớ làm việc (working memory) chứa ngữ cảnh gần nhất, đến bộ nhớ ngắn hạn chứa các phiên gần đây, và bộ nhớ dài hạn chứa tri thức ổn định. Mỗi tầng có chiến lược truy vấn và tần suất cập nhật riêng.
3. **Bộ nhớ có cấu trúc:** thay vì lưu văn bản thô, hệ thống trích xuất các đơn vị

tri thức có cấu trúc (thực thể, sự kiện, quan hệ) và tổ chức chúng thành đồ thị tri thức hoặc cơ sở dữ liệu quan hệ. Mô hình này cho phép truy vấn chính xác hơn và suy luận đa bước tốt hơn.

Mỗi mô hình đều có những đánh đổi riêng. Kho lưu trữ vector dễ triển khai nhưng gặp khó khăn trước các câu hỏi cần nối nhiều mảnh thông tin. Bộ nhớ phân tầng giúp quản lý ngữ cảnh hiệu quả nhưng chưa trả lời được câu hỏi “lưu dưới dạng gì”. Bộ nhớ có cấu trúc mạnh về suy luận nhưng chất lượng phụ thuộc rất nhiều vào khâu trích xuất. CogMem, như sẽ được trình bày ở Chương 3, theo đuổi mô hình thứ ba nhưng bổ sung thêm chiều sâu về nhận thức: không chỉ lưu trữ có cấu trúc, mà còn lưu trữ theo đúng kiểu trí nhớ mà con người vẫn dùng.

2.1.5 Luồng xử lý có bộ nhớ trong tác nhân hội thoại

Để hiểu rõ vị trí của bộ nhớ trong một hệ thống hoàn chỉnh, cần hình dung toàn bộ vòng đời xử lý của một tác nhân hội thoại. Mỗi khi người dùng gửi một tin nhắn, hệ thống trải qua các bước sau:

1. **Tiếp nhận và phân tích:** hệ thống nhận tin nhắn, phân tích ý định người dùng, trích xuất các thực thể và ràng buộc thời gian nếu có.
2. **Truy vấn bộ nhớ:** dựa trên nội dung và ý định đã phân tích, hệ thống truy vấn tầng bộ nhớ để lấy các thông tin liên quan từ các phiên trước. Bước này có thể bao gồm nhiều kênh truy vấn song song.
3. **Xây dựng ngữ cảnh suy luận:** hệ thống ghép tin nhắn hiện tại, lịch sử gần đây, và các mảnh bộ nhớ đã truy vấn thành một prompt hoàn chỉnh cho LLM.
4. **Suy luận và sinh phản hồi:** LLM đọc prompt, suy luận, và sinh ra câu trả lời cuối cùng cho người dùng.
5. **Cập nhật bộ nhớ:** sau khi trả lời, hệ thống trích xuất những thông tin mới từ lượt tương tác vừa rồi và cập nhật vào tầng bộ nhớ dài hạn.

Trong vòng lặp này, bộ nhớ đóng vai trò kép: vừa là nguồn cung cấp ngữ cảnh cho suy luận, vừa là nơi lưu giữ kết quả của tương tác. Một bộ nhớ được thiết kế tốt cần đáp ứng đồng thời cả hai chiều: đọc (truy vấn) phải nhanh và chính xác, còn ghi (lưu trữ) phải bảo toàn được thông tin và tổ chức phục vụ cho lần đọc sau.

2.2 Cơ sở lý thuyết cho việc thiết kế bộ nhớ

2.2.1 Khoa học nhận thức và các hệ con của bộ nhớ dài hạn

Một luận điểm quan trọng của khoa học nhận thức là bộ nhớ con người không phải một khối đồng nhất. Từ những năm 1970, Tulving đã phân biệt giữa trí nhớ tình tiết (episodic memory) – nơi lưu những trải nghiệm cá nhân gắn với thời gian

và bối cảnh, và trí nhớ ngữ nghĩa (semantic memory) – nơi lưu tri thức khái quát về thế giới. Sau này, các nghiên cứu tiếp tục mở rộng bức tranh với nhiều hệ con khác: trí nhớ thủ tục (procedural memory) cho kỹ năng và thói quen, trí nhớ triển vọng (prospective memory) cho những việc cần làm trong tương lai, và hệ thống gắn kết hành động–kết quả (action-effect binding) cho việc học quan hệ nhân quả giữa hành vi và hệ quả của nó.

Mỗi hệ con này không chỉ khác nhau về nội dung lưu trữ, mà còn khác nhau về cách truy xuất. Khi một người cố nhớ “tối qua ăn gì”, họ dùng trí nhớ tình tiết và thường tìm theo mốc thời gian. Khi trả lời “thủ đô của Pháp là gì”, họ dùng trí nhớ ngữ nghĩa và tìm theo liên tưởng khái niệm. Khi quyết định “sáng mai thức dậy sẽ làm gì đầu tiên”, họ dùng trí nhớ triển vọng kết hợp với trí nhớ thói quen. Sự khác biệt trong cách truy xuất này gợi ý một nguyên tắc thiết kế then chốt: *nếu các loại tín hiệu nhận thức có bản chất khác nhau, chúng không nên bị ép chung vào một kiểu biểu diễn duy nhất.*

Đối với bài toán bộ nhớ hội thoại, có thể rút ra bốn nhóm ý tưởng từ khoa học nhận thức:

- **Trí nhớ ngữ nghĩa:** lưu tri thức tương đối ổn định, ít phụ thuộc vào một lần trải nghiệm cụ thể. Trong hội thoại, đây là những fact như nghề nghiệp, kỹ năng, mối quan tâm dài hạn của người dùng.
- **Trí nhớ tình tiết:** lưu những gì đã xảy ra, gắn với thời gian và hoàn cảnh. Đây là những sự kiện cụ thể mà người dùng đã kể, đã trải qua trong các phiên hội thoại.
- **Trí nhớ thói quen:** phản ánh các mẫu hành vi lặp lại, thường bền vững hơn một sự kiện đơn lẻ. Ví dụ: “người dùng thường kiểm tra email vào buổi sáng” là một thói quen, không phải một sự kiện một lần.
- **Trí nhớ triển vọng:** liên quan đến việc nhớ những gì sẽ làm trong tương lai, tức các kế hoạch và mục tiêu chưa hoàn tất. Đây là loại trí nhớ quan trọng cho các trợ lý cá nhân, vì người dùng thường xuyên đề cập đến những điều họ định làm.

Điểm cốt yếu ở đây không nằm ở việc sao chép một mô hình nhận thức của con người sang hệ thống phần mềm. Điều đáng học hỏi chính là nguyên tắc thiết kế: các loại tín hiệu nhận thức khác nhau cần có không gian biểu diễn khác nhau, với thông tin bổ trợ, kiểu liên kết và chiến lược truy vấn phù hợp với bản chất riêng của từng loại. Đây chính là nền tảng trực tiếp cho quyết định chia bộ nhớ của CogMem thành sáu loại chuyên biệt thay vì chỉ lưu một dạng biểu diễn duy nhất.

2.2.2 Đồ thị tri thức và truy vấn nhiều bước

Đồ thị tri thức (Knowledge Graph) cung cấp một cách biểu diễn rất phù hợp cho bộ nhớ dài hạn vì nó cho phép mô tả đồng thời *nội dung* và *quan hệ*. Một node có thể lưu một sự kiện, một tri thức hoặc một kế hoạch; một cạnh có thể biểu diễn rằng hai node cùng nhắc tới một thực thể, diễn ra trước–sau, có quan hệ nhân quả, hay nằm trong cùng một vòng đời trạng thái.

Về mặt hình thức, một đồ thị tri thức có thể được định nghĩa là $G = (V, E, L_V, L_E)$, trong đó V là tập đỉnh (node), $E \subseteq V \times V$ là tập cạnh (edge), L_V là hàm gán nhãn cho đỉnh (ví dụ: loại fact), và L_E là hàm gán nhãn cho cạnh (ví dụ: quan hệ thời gian, quan hệ nhân quả). Khi áp dụng vào bộ nhớ hội thoại, mỗi đỉnh tương ứng với một đơn vị thông tin đã được trích xuất từ hội thoại, và mỗi cạnh tương ứng với một mối liên hệ giữa các đơn vị thông tin đó.

So với cách lưu từng fact độc lập, đồ thị có ba ưu điểm cốt lõi:

- **Giữ quan hệ một cách tường minh:** thay vì phải suy ra mọi liên kết chỉ từ độ tương đồng ngữ nghĩa, hệ thống có thể lưu trực tiếp các quan hệ như thời gian, nhân quả, hay tham chiếu tới các thực thể giống nhau. Điều này đặc biệt quan trọng với các câu hỏi dạng “tại sao” hay “sau khi”, vốn cần truy vấn chính xác loại quan hệ chứ không chỉ cần nội dung liên quan.
- **Hỗ trợ truy vấn nhiều bước:** câu trả lời có thể được khôi phục qua một chuỗi liên kết trên đồ thị, thay vì chỉ tìm kiếm một lần. Ví dụ, để trả lời câu hỏi “người dùng đã làm gì sau khi mất việc”, hệ thống có thể đi từ node sự kiện mất việc, qua cạnh thời gian, đến node sự kiện tiếp theo.
- **Cho phép kiểm soát quá trình lan truyền bằng chứng:** thay vì chỉ xếp hạng từng mảnh tri thức độc lập, hệ thống có thể dùng cấu trúc đồ thị để tích lũy tín hiệu từ nhiều đường đi. Một bằng chứng đúng đôi khi không nằm ở một fact thật mạnh, mà xuất hiện như kết quả hội tụ của nhiều dấu vết vừa phải nằm ở các phiên khác nhau.

Trong bài toán hội thoại dài hạn, ưu điểm thứ ba mang ý nghĩa then chốt. Thông tin cần để trả lời thường bị phân tán qua nhiều phiên: một phần ở phiên đầu, một phần ở phiên giữa, và một chi tiết quyết định lại nằm ở phiên gần cuối. Nếu chỉ dùng truy vấn vector dựa trên độ tương đồng đơn thuần, mỗi mảnh được đánh giá độc lập và có thể không đủ mạnh để lọt vào danh sách kết quả. Trên đồ thị, các mảnh này được nối với nhau qua các cạnh, và tín hiệu lan truyền từ nhiều nguồn có thể hội tụ để đẩy bằng chứng đúng lên vị trí cao hơn.

2.2.3 Xếp hạng lại bằng CrossEncoder trong truy vấn bộ nhớ

Trong các hệ thống RAG và bộ nhớ ngoài, truy vấn thường được chia thành hai tầng. Tầng đầu tiên có nhiệm vụ tìm nhanh một tập ứng viên rộng từ kho lưu trữ: đó có thể là các đoạn văn bản gần nhất trong không gian vector, các kết quả khớp từ khóa, hoặc các node được kích hoạt trên đồ thị. Tầng này cần bao phủ tốt, vì nếu bằng chứng đúng không xuất hiện trong tập ứng viên thì các bước phía sau không còn cơ hội sửa sai. Tuy nhiên, bao phủ rộng cũng kéo theo nhiều: nhiều ứng viên có thể nhắc tới cùng thực thể hoặc cùng chủ đề nhưng không thật sự trả lời câu hỏi.

Xếp hạng lại (reranking) là bước xử lý tầng hai nhằm giải quyết vấn đề đó. Thay vì tìm thêm ứng viên mới, hệ thống đọc lại một danh sách ứng viên đã có và sắp xếp chúng theo mức độ phù hợp với câu hỏi. Khác với các cơ chế dung hợp như Reciprocal Rank Fusion, vốn chủ yếu dựa trên thứ hạng do nhiều kênh truy vấn trả về, reranking đánh giá trực tiếp từng cặp *câu hỏi–bằng chứng*. Vì vậy, nó phù hợp với giai đoạn cuối của truy vấn: số lượng ứng viên đã đủ nhỏ để có thể dùng một mô hình chậm hơn nhưng tinh hơn.

CrossEncoder là một kiến trúc thường được dùng cho bước xếp hạng lại này. Điểm khác biệt chính của CrossEncoder nằm ở cách nó đọc đầu vào. Với mô hình hai bộ mã hóa, câu hỏi và bằng chứng được mã hóa riêng thành hai vector, sau đó hệ thống so sánh hai vector bằng một độ đo khoảng cách. Cách này nhanh và cho phép tính trước vector của tài liệu, nhưng nó chỉ so sánh hai biểu diễn đã tách rời. Ngược lại, CrossEncoder đưa cả câu hỏi và bằng chứng vào cùng một mô hình, để cơ chế chú ý có thể xét trực tiếp từng từ của câu hỏi với từng từ của bằng chứng. Nhờ đó, mô hình có thể phân biệt những khác biệt nhỏ như phủ định, trạng thái đã hoàn thành hay chưa hoàn thành, quan hệ trước–sau, hoặc một chi tiết số liệu quyết định.

Đổi lại, CrossEncoder tốn chi phí tính toán cao hơn vì mỗi cặp câu hỏi–bằng chứng phải được chạy qua mô hình một lần. Đây là lý do nó hiếm khi được dùng làm bước truy vấn đầu tiên trên toàn bộ kho nhớ. Vai trò hợp lý hơn của nó là đứng sau tầng truy vấn nhanh: tầng đầu tạo danh sách ứng viên, còn CrossEncoder chọn lại những bằng chứng đáng tin nhất trước khi đưa vào prompt cho LLM. Trong bộ nhớ hội thoại dài hạn, sự phân vai này đặc biệt quan trọng vì các bằng chứng sai thường không hoàn toàn lạc đề; chúng thường giống đúng ở tên người, thời điểm hoặc chủ đề, nhưng sai ở trạng thái, quan hệ nhân quả hoặc chi tiết cần trả lời.

2.3 Các hướng nghiên cứu liên quan

2.3.1 Các hệ thống lưu trữ phẳng hoặc dùng kho lưu trữ vector

Một nhánh phổ biến của bộ nhớ cho tác nhân hội thoại là lưu lại các bản ghi ngắn rồi truy vấn bằng độ tương đồng ngữ nghĩa. *Mem0* là một đại diện tiêu biểu cho hướng này: hệ thống lưu các fact dưới dạng văn bản ngắn, nhúng chúng thành vector, và truy vấn bằng tìm kiếm tương đồng. Ưu điểm của nhánh này là dễ triển khai, tốc độ cao và tương thích tốt với các quy trình RAG hiện có.

Tuy nhiên, điểm yếu cốt lõi của hướng tiếp cận này nằm ở hai khía cạnh. *Thứ nhất*, các fact sau khi được tách ra thường đứng độc lập với nhau, khiến hệ thống khó nắm bắt được những quan hệ như một kế hoạch bị trì hoãn, một hành động dẫn tới kết quả cụ thể, hay một cập nhật làm thay đổi trạng thái của một fact đã có từ trước. *Thứ hai*, khi câu hỏi đòi hỏi truy hồi qua nhiều bước – ví dụ “người dùng đã từng định làm gì nhưng chưa làm?” – việc chỉ tìm những đoạn văn bản giống nhất về mặt ngôn từ không phải lúc nào cũng đưa được đúng bằng chứng lên đầu danh sách, bởi câu trả lời thường nằm ở sự kết hợp của nhiều fact chứ không nằm gọn trong một fact đơn lẻ.

2.3.2 Các hệ thống quản lý ngữ cảnh và phân tầng bộ nhớ

Một số nghiên cứu khác không tập trung vào biểu diễn tri thức, mà nhấn mạnh cách quản lý ngữ cảnh và chuyển thông tin giữa vùng đang hoạt động và vùng lưu trữ lâu dài. Các hệ thống như MemGPT hay Letta đã chứng minh rằng việc mô phỏng hệ điều hành bộ nhớ – với phân trang, ngắt, và chuyển đổi ngữ cảnh – có thể giúp LLM xử lý các phiên hội thoại dài hơn nhiều so với cửa sổ ngữ cảnh vật lý. Hướng này rất hữu ích trong bối cảnh trợ lý hội thoại vì nó giảm gánh nặng phải giữ toàn bộ lịch sử trong prompt và giúp hệ thống mở rộng tốt hơn theo thời gian.

Tuy vậy, nếu chỉ dừng ở mức quản lý ngữ cảnh mà không làm rõ cấu trúc của dữ liệu được lưu, hệ thống vẫn gặp khó khi phải phân biệt giữa tri thức ổn định, kinh nghiệm cá nhân, thói quen hay kế hoạch tương lai. Nói cách khác, phân tầng bộ nhớ giải quyết tốt câu hỏi “lưu ở đâu” và “khi nào nên chuyển dữ liệu giữa các tầng”, nhưng chưa giải quyết triệt để câu hỏi “lưu dưới dạng gì” – câu hỏi mà CogMem đặt làm trọng tâm.

2.3.3 Các hệ thống dựa trên đồ thị tri thức

Nhóm hệ thống gần với CogMem nhất là các bộ nhớ hội thoại dựa trên đồ thị tri thức. HINDSIGHT là đại diện nổi bật trong nhóm này. Hệ thống này đã chứng minh rằng việc lưu trữ hội thoại thành đồ thị dữ thể với các loại thông tin như world, experience, opinion, observation và truy vấn bằng nhiều kênh song song (semantic,

BM25, graph, temporal) có thể tạo ra chất lượng rất mạnh trên các benchmark hội thoại dài. Ở mức phương pháp luận, đây là một cột mốc quan trọng vì nó cho thấy bộ nhớ đồ thị không chỉ hấp dẫn về mặt ý tưởng mà còn có hiệu quả thực nghiệm.

Tuy nhiên, khi phân tích kỹ kiến trúc của HINDSIGHT, có thể thấy một số giới hạn. Thứ nhất, hệ thống vận hành với bốn nhóm trí nhớ chính nhưng chưa có các cấu trúc chuyên biệt hơn cho hành động, tương lai hay thói quen. Điều này có nghĩa là những tín hiệu như “tôi định học Rust nhưng chưa bắt đầu” hay “gọi API với header Retry-After thì được 200” không có một không gian biểu diễn phù hợp trong đồ thị. Thứ hai, cơ chế kết hợp kết quả truy vấn từ các kênh của HINDSIGHT dùng trọng số cố định cho các kênh, trong khi nhiều câu hỏi hội thoại rõ ràng cần ưu tiên một loại bằng chứng hơn loại khác tùy theo bản chất câu hỏi. Thứ ba, cơ chế lan truyền trên đồ thị của HINDSIGHT dùng phép lấy cực đại (MAX), chỉ giữ đường mạnh nhất, có thể bỏ lỡ các tín hiệu yếu nhưng bổ sung cho nhau từ nhiều phía.

Một hướng đồ thị khác, tiêu biểu như HippoRAG và HippoRAG2, cho thấy sức mạnh của việc coi truy vấn như quá trình lan truyền trên mạng tri thức thay vì chỉ đơn thuần tìm láng giềng gần nhất trong không gian vector. Các hệ thống này sử dụng thuật toán lan truyền xác suất (Personalized PageRank) để mô phỏng quá trình suy nghĩ liên tưởng của con người trên đồ thị, và đã đạt được kết quả ấn tượng trên các bài toán đòi hỏi suy luận qua nhiều bước. Tuy vậy, các hệ thống này thường thiên về liên kết tri thức khách quan và suy luận quan hệ trong miền mở, trong khi bài toán hội thoại cá nhân hóa còn cần biểu diễn rõ những thành phần chủ quan như thói quen, ý định và trạng thái chưa hoàn thành.

2.3.4 Bộ nhớ cho tác nhân AI và kịch bản sử dụng công cụ

Trong bối cảnh tác nhân AI sử dụng công cụ, bộ nhớ không chỉ phải lưu “điều gì đã được nói” mà còn phải lưu “điều gì đã được làm” và “hành động đó dẫn tới kết quả gì”. Đây là một sự dịch chuyển quan trọng: tri thức hữu ích không còn chỉ là tri thức mô tả (declarative knowledge), mà còn là tri thức can thiệp (interventional knowledge). Các benchmark như ToolEval, AgentBench hay SWE-Bench đã cho thấy nhu cầu này một cách gián tiếp: tác nhân thường thất bại không phải vì thiếu năng lực suy luận, mà vì không nhớ được rằng một chuỗi hành động cụ thể đã từng thất bại hoặc thành công trong quá khứ.

Ý tưởng này rất gần với loại node *action-effect* của CogMem. Nếu một tác nhân từng gặp lỗi xác thực, thử một lệnh tái đăng nhập với tham số cụ thể và sau đó triển khai thành công, hệ thống cần nhớ được toàn bộ quan hệ “điều kiện – hành động – kết quả”, chứ không chỉ nhớ riêng lẻ rằng lỗi đã xảy ra hoặc rằng lệnh nào đó có

tồn tại. Việc lưu trữ tách rời ba thành phần này cho phép hệ thống trả lời chính xác các câu hỏi như “làm cách nào để khắc phục lỗi này” bằng cách truy xuất đúng quy luật nhân quả đã được kiểm chứng, thay vì chỉ liệt kê các fact liên quan đến lỗi một cách rời rạc.

Bảng 2.1: So sánh khái quát các hướng xây dựng bộ nhớ dài hạn liên quan

Tiêu chí	Kho vector / Mem0	Phân tầng ngữ cảnh	HippoRAG2	HINDSIGHT	CogMem (đề xuất)
Đồ thị	Không	Hạn chế	Có	Có	Có
Thời gian	Hạn chế	Có (vận hành)	Có	Có	Có
Kế hoạch / Thói quen	Không tách riêng	Không phải trọng tâm	Chưa chuyên biệt	Chưa có mạng riêng	Có
Nhân quả	Không tách riêng	Không phải trọng tâm	Chưa chuyên biệt	Chưa có mạng riêng	Có
Đặc trưng nổi bật	Truy hỏi nhanh, dễ triển khai; khó suy luận nhiều bước.	Giảm áp lực prompt, hữu ích cho phiên dài.	Mạnh nổi thực thể và truy hỏi nhiều bước trên đồ thị.	Kết hợp đồ thị dị thể, truy hỏi đa kênh, dung hợp kết quả.	Mở rộng đồ thị theo hướng nhận thức; giữ bằng chứng gốc; định tuyến theo loại câu hỏi.

2.4 Các bộ dữ liệu đánh giá bộ nhớ dài hạn

2.4.1 LongMemEval: đánh giá trí nhớ cá nhân qua hội thoại dài

LongMemEval là một trong những benchmark đầu tiên được thiết kế chuyên biệt để đo năng lực nhớ thông tin cá nhân của hệ thống qua các cuộc hội thoại rất dài. Bộ dữ liệu này được xây dựng dựa trên ý tưởng rằng một trợ lý cá nhân thực thụ cần nhớ được không chỉ những fact đơn lẻ, mà còn cả những thay đổi, cập nhật, và mối liên hệ giữa các thông tin qua thời gian.

Trong phiên bản được sử dụng làm nguồn dữ liệu cho dự án, LongMemEval bao gồm các cuộc hội thoại nhiều phiên với độ dài trung bình khoảng 115 nghìn từ cho mỗi câu hỏi. Mỗi cuộc hội thoại mô phỏng tương tác giữa một người dùng và một trợ lý AI qua nhiều ngày, trong đó người dùng dần tiết lộ thông tin về bản thân, công việc, sở thích, kế hoạch và các mối quan hệ. Cái khó của LongMemEval không nằm ở một câu chuyện quá phức tạp, mà nằm ở việc thông tin trả lời thường bị rải ra ở nhiều phiên và có thể bị nhiễu bởi các chi tiết cập nhật hoặc tương tự.

Các nhóm câu hỏi quan trọng của LongMemEval bao gồm:

- **Truy hỏi nhiều phiên:** câu hỏi yêu cầu tổng hợp thông tin từ ít nhất hai phiên khác nhau. Ví dụ: “Người dùng đã đến những thành phố nào trong năm qua?”, đòi hỏi hệ thống phải gom các lần nhắc đến địa điểm trong nhiều phiên.

- **Cập nhật tri thức:** câu hỏi kiểm tra xem hệ thống có nhận biết được khi một thông tin cũ bị ghi đè bởi thông tin mới hay không. Ví dụ: người dùng nói đã đổi việc ở phiên 12, nhưng ở phiên 25 lại nhắc đến công việc cũ trong một ngữ cảnh khác – hệ thống cần phân biệt được đâu là thông tin hiện tại.
- **Suy luận thời gian:** câu hỏi yêu cầu xác định thứ tự hoặc khoảng cách thời gian giữa các sự kiện. Ví dụ: “Người dùng bắt đầu công việc mới trước hay sau khi chuyển nhà?”
- **Câu hỏi về sở thích:** hỏi về những gì người dùng thích hoặc không thích, thường được đề cập thoáng qua trong hội thoại.

Ví dụ mẫu từ LongMemEval

Bối cảnh: Một người dùng trò chuyện với trợ lý AI trong 50 phiên, trải dài từ tháng 3 đến tháng 9. Dưới đây là các đoạn hội thoại rải rác có liên quan đến cùng một câu hỏi:

[Phiên 3 — 12/03]

User: I just got assigned as the lead for Project Alpha. It's a big data migration project.

Assistant: Congratulations! That sounds like a major responsibility.

[Phiên 17 — 05/05]

User: Good news — they added me to Project Beta as well. So now I'm juggling two projects.

Assistant: Two projects at once? That must be demanding.

[Phiên 28 — 18/07]

User: Just wrapped up Project Alpha yesterday. Feels good to have that one off my plate.

Assistant: Well done! Does this mean you're back to just one project now?

[Phiên 42 — 03/09]

User: I'm looking for a co-lead for Project Gamma. It's ramping up and I can't handle it alone.

Câu hỏi: How many projects have I led or am currently leading?

Đáp án đúng: 2 — gồm Project Beta (vẫn đang dẫn dắt) và Project Gamma (mới nhận, đang tìm người hỗ trợ). Project Alpha đã kết thúc ở phiên 28 nên không còn được tính.

Ý nghĩa: Hệ thống không thể chỉ nhớ lời nói gần nhất. Nó phải tổng hợp thông tin từ bốn phiên rải rác trong suốt 6 tháng, hiểu được trạng thái “đã kết thúc” khác với “đang dần dặt”, và phân biệt giữa dự án đã đóng với dự án còn hoạt động.

2.4.2 LoCoMo: suy luận chuỗi bằng chứng trên hội thoại dài

LoCoMo (Long Context Memory) là một benchmark được thiết kế với trọng tâm mạnh hơn vào suy luận nối chuỗi bằng chứng qua nhiều bước. Không giống LongMemEval vốn tập trung vào nhớ thông tin cá nhân, LoCoMo đặt ra các câu hỏi mà để trả lời đúng, hệ thống cần kết nối nhiều mảnh thông tin rải rác – đôi khi nằm cách nhau hàng chục nghìn từ – thành một chuỗi suy luận có ý nghĩa.

Dữ liệu gốc của LoCoMo bao gồm 50 cuộc hội thoại dài, mỗi cuộc có trung bình khoảng 305 lượt trao đổi giữa hai người nói. Các cuộc hội thoại này mô phỏng những câu chuyện đời thường kéo dài qua nhiều năm: một người kể về công việc, gia đình, sở thích, các mối quan hệ và những thay đổi lớn trong cuộc sống. Điểm đặc biệt của LoCoMo là các sự kiện không được kể theo trình tự thời gian tuyến tính; người kể thường nhảy qua lại giữa các mốc thời gian, khiến cho việc xâu chuỗi sự kiện trở nên khó khăn hơn nhiều.

Câu hỏi trong LoCoMo được phân thành năm nhóm lớn, mỗi nhóm kiểm tra một năng lực truy hồi khác nhau:

- **Single-hop:** câu hỏi chỉ cần một mảnh thông tin duy nhất để trả lời. Đây là nhóm dễ nhất, chiếm đa số câu hỏi (khoảng 109/161 câu trong bản rút gọn), nhưng vẫn khó vì thông tin có thể bị chôn vùi giữa hàng trăm nghìn từ.
- **Multi-hop:** câu hỏi cần nối ít nhất hai mảnh thông tin từ các phần khác nhau của hội thoại. Ví dụ: “Jon làm nghề gì trước khi mở vũ trường?”, đòi hỏi hệ thống phải biết Jon từng làm nghề gì và mở vũ trường là sự kiện xảy ra sau đó.
- **Temporal:** câu hỏi về thời gian, thứ tự hoặc khoảng cách giữa các sự kiện. Đây là nhóm khó nhất đối với CogMem.
- **Preference:** câu hỏi về sở thích, ý kiến hoặc đánh giá cá nhân của nhân vật trong hội thoại.
- **Causal:** câu hỏi về nguyên nhân, lý do đằng sau một sự kiện hoặc quyết định. Ví dụ: “Tại sao Jon quyết định mở vũ trường riêng?”

Ví dụ mẫu từ LoCoMo

Bối cảnh: Jon trò chuyện với một người bạn qua 40 phiên, kể về cuộc đời mình trong suốt nhiều năm. Các sự kiện được kể không theo trình tự thời gian — Jon thường nhảy cóc giữa quá khứ và hiện tại. Dưới đây là các đoạn hội thoại rải rác có liên quan đến cùng một câu hỏi:

[Phiên 5]

Jon: I worked at Miller & Sons for about ten years. It was stable, comfortable — you know, the kind of job you don't leave unless something pushes you.

[Phiên 12 — Jon đang kể về thời thơ ấu]

Jon: I've been dancing since I was seven. My mom put me in a ballet class because I wouldn't sit still. By high school, I was competing in ballroom and Latin. If I'm being honest, dance is the only thing I've ever truly loved doing.

[Phiên 18 — Jon quay lại chuyện công việc]

Jon: So they let me go. After ten years. The company restructured and my whole division got cut. I sat in the parking lot for an hour, not knowing what to do next.

[Phiên 22]

Jon: I think I'm going to do it. Open my own dance studio. It's terrifying — I've never run a business before — but after the layoff, I realized I don't want to spend another decade doing something I don't care about.

[Phiên 30]

Jon: The studio's been open for three months now. We have about forty students. It's not making me rich, but I wake up every morning actually excited to go to work. That's worth more than the paycheck ever was.

Câu hỏi: Why did Jon decide to start his dance studio?

Đáp án đúng: After being laid off from his ten-year job at Miller & Sons, he decided to pursue his lifelong passion for dance and open his own studio rather than find another corporate job.

Ý nghĩa: Đây là câu hỏi nhân quả điển hình của LoCoMo. Để trả lời đúng, hệ thống phải nối ba mảnh thông tin nằm ở ba phiên khác nhau, cách nhau hàng chục nghìn từ: (1) đam mê khiêu vũ từ nhỏ (phiên 12), (2) bị sa thải sau 10 năm (phiên 18), và (3) quyết định mở studio (phiên 22). Thiếu bất kỳ mảnh nào, câu trả lời sẽ không đầy đủ. Thêm vào đó, thông tin về việc studio

đã hoạt động 3 tháng (phiên 30) là một yếu tố gây nhiễu — nó có vẻ liên quan nhưng không trả lời câu hỏi “tại sao”.

2.5 Khoảng trống nghiên cứu mà CogMem hướng tới

Từ các phân tích trên, có thể tóm lược khoảng trống nghiên cứu hiện tại thành bốn ý chính:

1. **Các hệ thống mạnh nhất hiện nay vẫn chưa biểu diễn đầy đủ các dạng trí nhớ khác nhau.** Nhiều kiến trúc đã rất tốt ở tri thức chung và sự kiện, nhưng chưa có không gian biểu diễn thật rõ cho thói quen lặp lại, kế hoạch chưa hoàn thành và quy luật hành động–kết quả. Đây là những dạng trí nhớ mà khoa học nhận thức đã chỉ ra là cần thiết cho hành vi thông minh, nhưng lại chưa được phản ánh đầy đủ trong các hệ thống bộ nhớ hội thoại.
2. **Nén thông tin vẫn gây mất mát đáng kể.** Khi hội thoại dài được rút gọn thành các fact ngắn, nhiều chi tiết quan trọng như con số chính xác, tên riêng, sắc thái cảm xúc và ngữ cảnh trích dẫn thường bị mất đi. Hệ thống có thể trả lời trúng chủ đề nhưng sai chi tiết – một lỗi đặc biệt nghiêm trọng trong các ứng dụng trợ lý cá nhân.
3. **Truy vấn nhiều bước cần được xem như quá trình tích lũy bằng chứng, không chỉ là chọn một kết quả tốt nhất.** Các cơ chế truy hồi hiện tại thường dừng ở việc tìm một fact mạnh nhất hoặc một đường dẫn ngắn nhất. Trong thực tế, câu trả lời đúng thường đến từ sự hội tụ của nhiều dấu vết yếu, và việc chỉ giữ đường mạnh nhất có thể bỏ lỡ chính xác bằng chứng cần thiết.
4. **Cùng một công thức không phù hợp cho mọi loại truy vấn.** Một câu hỏi về thời gian cần ưu tiên bằng chứng temporal, câu hỏi về nguyên nhân cần ưu tiên liên kết causal trên đồ thị, câu hỏi về kế hoạch cần ưu tiên intention node. Việc dùng một bộ trọng số cố định cho mọi truy vấn là một thiếu sót về mặt phương pháp luận.

Từ khoảng trống đó, Chương 3 sẽ trình bày CogMem như một đề xuất kiến trúc vừa giàu cấu trúc hơn về mặt biểu diễn, vừa thận trọng hơn về mặt thực nghiệm: hệ thống được xây dựng để cung cấp không gian cho các loại tín hiệu nhận thức khác nhau, sau đó đánh giá xem mỗi cấu phần giúp ích đến đâu trên từng loại dữ liệu cụ thể.

CHƯƠNG 3. ĐỀ XUẤT COGMEM: KHUNG BỘ NHỚ DÀI HẠN DỰA TRÊN KHOA HỌC NHẬN THỨC

Dựa trên các giới hạn đã phân tích ở Chương 2, chương này trình bày CogMem như một khung bộ nhớ dài hạn cho tác nhân hội thoại. Mạch trình bày của chương đi từ cơ sở nhận thức, đến vấn đề hệ thống, rồi đến cách CogMem hiện thực hóa từng quyết định thiết kế trong ba luồng xử lý: ghi nhớ thông tin mới, truy vấn thông tin cũ, và tổng hợp câu trả lời từ bằng chứng thu được qua truy vấn.

3.1 Tổng quan hệ thống bộ nhớ CogMem

CogMem được thiết kế để giải quyết một vấn đề cụ thể của tác nhân hội thoại dài hạn: hệ thống không chỉ cần nhớ lại một đoạn trò chuyện cũ, mà còn phải biết loại thông tin nào đang được hỏi, bằng chứng nào đáng tin, và các mảnh thông tin rải rác qua nhiều phiên liên hệ với nhau như thế nào. Trong chương này, mỗi *fact* là một đơn vị thông tin được rút ra từ lời người dùng, chẳng hạn một sự kiện đã kể, một sở thích hoặc một kế hoạch tương lai. Nếu các *fact* này chỉ được lưu như những ghi chú rời nhau, hệ thống sẽ dễ xem mọi thông tin là ngang hàng. Khi đó, “người dùng đã học Rust” và “người dùng định học Rust” có thể cùng nằm trong bộ nhớ, nhưng hệ thống thiếu dấu hiệu rõ ràng để biết câu nào nói về quá khứ, câu nào nói về một kế hoạch chưa xảy ra.

Ba yêu cầu trên không được giải quyết bằng một cơ chế duy nhất. Để phân biệt loại thông tin và quan hệ giữa các *fact*, CogMem biểu diễn hội thoại thành một đồ thị bộ nhớ có kiểu. Thay vì chỉ đặt các *fact* cạnh nhau, đồ thị gắn nhãn rõ ràng cho từng vai trò: một *experience* là sự kiện đã xảy ra, một *intention* là kế hoạch hướng tới tương lai, còn liên kết *temporal* cho biết hai sự kiện diễn ra trước hay sau nhau.

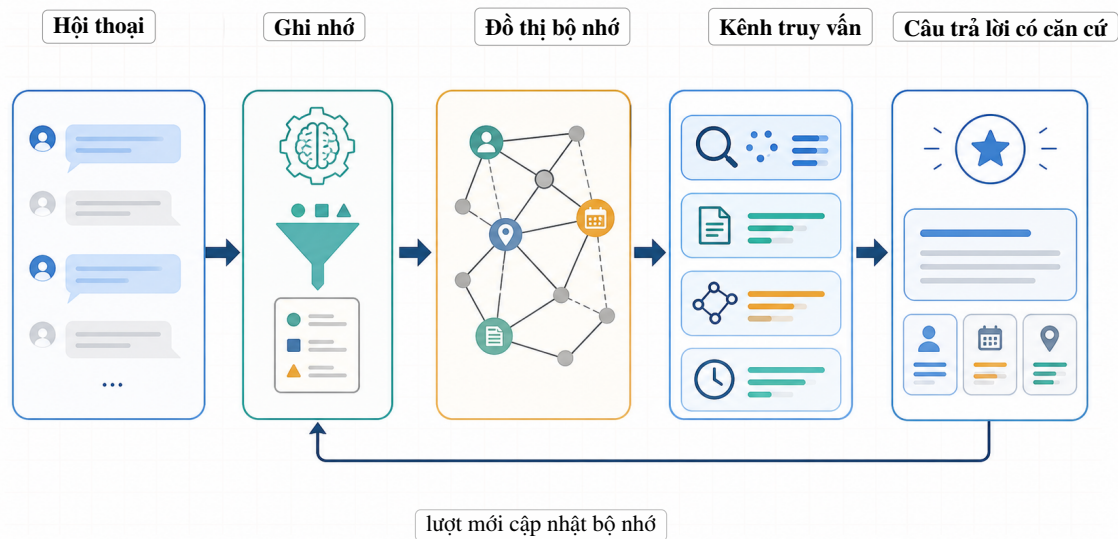
Riêng yêu cầu về bằng chứng được xử lý bằng một quyết định khác: mỗi đơn vị bộ nhớ không chỉ có bản tóm tắt để tìm kiếm nhanh, mà còn giữ đoạn trích nguyên văn để hỗ trợ câu trả lời cần chi tiết chính xác. Như vậy, phần tổng quan của chương đi qua ba lớp thiết kế liên quan nhưng không đồng nhất: đồ thị có kiểu để tổ chức thông tin, biểu diễn kép để bảo toàn bằng chứng, và truy vấn đa kênh để chọn bằng chứng phù hợp với câu hỏi.

Ở mức kiến trúc, CogMem vận hành qua ba luồng xử lý chính, mỗi luồng là một chuỗi bước nối tiếp nhau, trong đó đầu ra của bước trước trở thành đầu vào của bước sau:

1. **Pipeline ghi nhớ (Retain):** tiếp nhận hội thoại mới, trích xuất các đơn vị thông tin, phân loại chúng theo chức năng nhận thức, đổi mỗi đơn vị thành vector

nhúng, lưu vào đồ thị bộ nhớ và tạo các liên kết giữa các đơn vị thông tin. Vector nhúng là dạng biểu diễn bằng số của ý nghĩa câu, nhờ đó hệ thống có thể so sánh hai câu có gần nhau về nội dung hay không.

2. **Pipeline truy vấn (Recall)**: tìm lại thông tin cũ khi có câu hỏi. Hệ thống chạy song song nhiều kênh truy vấn gồm truy vấn ngữ nghĩa, truy vấn từ khóa, truy vấn đồ thị và truy vấn theo thời gian. Sau đó, hệ thống kết hợp kết quả bằng cách trộn và xếp hạng lại các bằng chứng từ nhiều kênh theo trọng số phù hợp với loại câu hỏi.
3. **Pipeline tổng hợp (Synthesis/Reflection/Generation)**: chọn các bằng chứng đã truy vấn, đưa vào bước sinh câu trả lời, và ưu tiên câu trả lời bám vào bằng chứng thay vì suy đoán từ mô hình ngôn ngữ.



Hình 3.1: Tổng quan luồng xử lý CogMem: hội thoại được chuyển thành đồ thị bộ nhớ, sau đó hệ thống truy vấn đa kênh và sinh câu trả lời dựa trên bằng chứng.

3.1.1 Phạm vi và giả định triển khai

Chương này mô tả phương pháp được triển khai và đánh giá trong phạm vi đề án, không giả định rằng mọi thành phần đều tạo cải thiện độc lập trên mọi bộ dữ liệu. CogMem cung cấp một cấu trúc biểu diễn giàu hơn bộ nhớ phẳng, nhưng hiệu quả thực tế của từng loại thông tin phụ thuộc vào dữ liệu, chất lượng trích xuất và loại câu hỏi trong các bộ benchmark. Benchmark là bộ dữ liệu đánh giá gồm câu hỏi và đáp án chuẩn, dùng để đo khả năng nhớ và trả lời của hệ thống.

Các giả định triển khai chính gồm:

- Trích xuất fact vẫn phụ thuộc vào mô hình ngôn ngữ, nên một đơn vị thông tin

có thể bị thiếu, bị gộp với đơn vị khác, hoặc bị phân loại sang loại thông tin chưa thật phù hợp. Phần phương pháp của CogMem cải thiện cách biểu diễn và truy vấn sau khi fact đã được trích xuất; nó không loại bỏ hoàn toàn sai số ở bước đọc hiểu hội thoại ban đầu.

- Truy vấn trên đồ thị không thay thế truy vấn ngữ nghĩa hoặc truy vấn từ khóa. Cấu hình chính của CogMem là truy vấn đa kênh, trong đó mỗi kênh bù cho điểm yếu của các kênh còn lại: kênh ngữ nghĩa tìm thông tin gần nghĩa với câu hỏi, kênh từ khóa tìm các đoạn có từ hoặc cụm từ trùng khớp, kênh đồ thị đi theo các liên kết đã lưu, và kênh thời gian ưu tiên các sự kiện đúng mốc hoặc đúng thứ tự.
- Trường *raw_snippet* được giữ như bằng chứng nguyên văn của hội thoại gốc. Đây là đoạn văn bản gốc nơi fact được rút ra, không phải bản tóm tắt. Tùy chính sách của từng lần chạy, đoạn trích này có thể được đưa vào prompt sinh câu trả lời để làm ngữ cảnh đầu vào mà mô hình ngôn ngữ đọc trước khi trả lời.
- Các loại thông tin như *habit*, *intention* và *action_effect* là các không gian biểu diễn có điều kiện. *Habit* lưu thói quen lặp lại, *intention* lưu kế hoạch tương lai, còn *action_effect* lưu quan hệ điều kiện–hành động–kết quả. Chúng được thêm vào để xử lý những loại câu hỏi mà world hoặc experience thường biểu diễn chưa đủ, không phải để khẳng định mọi câu hỏi đều cần chúng.

Trong phạm vi đồ án này, phương pháp truy vấn đồ thị mặc định là duyệt đồ thị theo chiều rộng: hệ thống bắt đầu từ các đơn vị thông tin gần câu hỏi nhất, rồi mở rộng dần sang các đơn vị lân cận theo từng lớp liên kết. Cách duyệt này được kết hợp với lan truyền tín hiệu cộng dồn và ba cơ chế kiểm soát chu trình. Một số biến thể truy vấn khác có thể được thêm trong quá trình phát triển để phục vụ so sánh, nhưng lõi phương pháp luận của chương này vẫn là: luồng ghi nhớ tạo đồ thị bộ nhớ có kiểu, luồng truy vấn lấy bằng chứng từ nhiều kênh, và luồng tổng hợp sinh câu trả lời dựa trên bằng chứng đó.

3.2 Kiến trúc xử lý ba pipeline của CogMem

Sau phần tổng quan, mục này mô tả ba pipeline ở mức vận hành: mỗi pipeline nhận đầu vào gì, tạo đầu ra gì, và chuyển thông tin cho pipeline kế tiếp như thế nào. Chi tiết về các cơ chế trong mỗi pipeline sẽ được trình bày ở những mục sau.

3.2.1 Luồng ghi nhớ thông tin mới

Trong luồng ghi nhớ, CogMem xử lý hội thoại theo sáu bước:

1. **Chia đoạn hội thoại:** hội thoại dài được chia thành các đoạn có kích thước

phù hợp để mô hình ngôn ngữ có thể đọc và trích xuất ổn định.

2. **Trích xuất lượt toàn cục:** mô hình đọc cả lời người dùng và trợ lý để nhận diện tri thức, sự kiện, quan điểm, thói quen, ý định và quan hệ hành động–kết quả.
3. **Trích xuất lượt theo góc nhìn người dùng:** một lượt bổ sung tập trung vào các thông tin cá nhân của người dùng, đặc biệt là trải nghiệm, sở thích, thói quen và kế hoạch.
4. **Loại bỏ trùng lặp và chuẩn hóa:** hệ thống hợp nhất các kết quả lặp, chuẩn hóa trạng thái ý định, chuẩn hóa thời gian và các trường thông tin chuyên biệt.
5. **Tạo vector nhúng và lưu trữ:** mỗi đơn vị thông tin được chuyển thành vector số để phục vụ truy vấn ngữ nghĩa. Nếu hai đơn vị có ý nghĩa gần nhau, vector của chúng cũng thường gần nhau trong không gian số. Ở cùng bước lưu trữ, hệ thống giữ thêm đoạn trích nguyên văn để làm bằng chứng nguồn cho giai đoạn tổng hợp.
6. **Tạo liên kết:** hệ thống thiết lập các quan hệ giữa các đơn vị thông tin, gồm quan hệ thực thể, thời gian, ngữ nghĩa, nhân quả, kích thích–phản ứng, hành động–kết quả và chuyển trạng thái.

3.2.2 Luồng truy vấn thông tin cũ

Luồng truy vấn bắt đầu từ câu hỏi của người dùng. Hệ thống trước hết phân tích câu hỏi để xác định nó thiên về thời gian, nguyên nhân, sở thích, kế hoạch tương lai, ngữ nghĩa chung hay suy luận nhiều bước. Với câu hỏi nhiều bước, câu trả lời không nằm trọn trong một fact duy nhất mà cần nối ít nhất hai mảnh bằng chứng.

Sau bước phân tích, CogMem chạy bốn kênh truy vấn song song: kênh ngữ nghĩa, kênh từ khóa (BM25), kênh đồ thị và kênh thời gian. Mỗi kênh tạo một danh sách ứng viên riêng. Các danh sách này được kết hợp bằng điểm xếp hạng có trọng số để tạo ra một danh sách bằng chứng cuối cùng cho bước tổng hợp. Phần 3.5 trình bày chi tiết từng kênh và cách kết hợp.

3.2.3 Luồng tổng hợp câu trả lời

Luồng tổng hợp nhận các bằng chứng mạnh nhất sau truy vấn, chuẩn hóa chúng thành các mục bằng chứng, rồi dựng ngữ cảnh đầu vào cho mô hình sinh câu trả lời. Ở bước này, đoạn trích nguyên văn được dùng cho một mục đích khác với vector nhúng: nó giúp mô hình trả lời những chi tiết dễ bị mất khi chỉ nhìn vào fact đã tóm tắt, ví dụ con số, ngày tháng, lời nói gốc hoặc sắc thái cảm xúc.

Nếu có mô hình sinh câu trả lời, prompt yêu cầu mô hình chỉ trả lời dựa trên

bằng chứng đã cung cấp, ưu tiên *raw_snippet* khi có và nói rõ khi bằng chứng chưa đủ. Nếu không có mô hình sinh, hệ thống vẫn có thể tạo một câu trả lời mặc định bằng cách liệt kê các bằng chứng đã chọn. Phần cuối chương quay lại pipeline này để mô tả logic chuẩn hóa bằng chứng, dựng prompt và kiểm soát suy đoán.

3.3 Đồ thị tri thức từ khoa học nhận thức

3.3.1 Động lực nhận thức và cấu trúc đa hệ thống

Cơ sở nhận thức đầu tiên của CogMem là quan sát rằng bộ nhớ con người không vận hành như một kho lưu trữ đồng nhất. Trí nhớ ngữ nghĩa lưu tri thức tương đối ổn định; trí nhớ tình tiết lưu sự kiện đã xảy ra; trí nhớ thói quen lưu các mẫu hành vi lặp lại; trí nhớ triển vọng lưu việc cần làm trong tương lai; còn các cơ chế gắn kết hành động—kết quả giúp con người hiểu hành động nào dẫn tới hệ quả nào.

Vấn đề của một bộ nhớ hội thoại đồng nhất là các loại tín hiệu này bị đặt chung vào một kiểu biểu diễn. Câu “người dùng định học Rust” có thể bị lưu gần giống câu “người dùng đã học Rust”, dù một câu nói về kế hoạch tương lai chưa xảy ra, còn câu kia nói về một trải nghiệm đã hoàn thành. Tương tự, “người dùng thường kiểm tra email trước họp” khác với “người dùng đã kiểm tra email sáng thứ Hai”, vì một bên là mẫu hành vi lặp lại, bên còn lại là sự kiện đơn lẻ.

CogMem hiện thực hóa nguyên tắc này bằng một đồ thị tri thức dị thể, nơi nhiều loại đơn vị thông tin và nhiều loại liên kết cùng tồn tại thay vì bị ép vào một dạng chung. Mỗi đơn vị thông tin là một nút trong đồ thị, mang loại, các trường mô tả bổ sung và liên kết phù hợp với vai trò nhận thức của nó. Các trường bổ sung này ghi những chi tiết như thời điểm xảy ra, độ tin cậy, hoặc trạng thái của một kế hoạch. Nhờ vậy, hệ thống có thể phân biệt những thông tin có câu chữ và ngữ nghĩa gần nhau nhưng chức năng khác nhau trong suy luận.

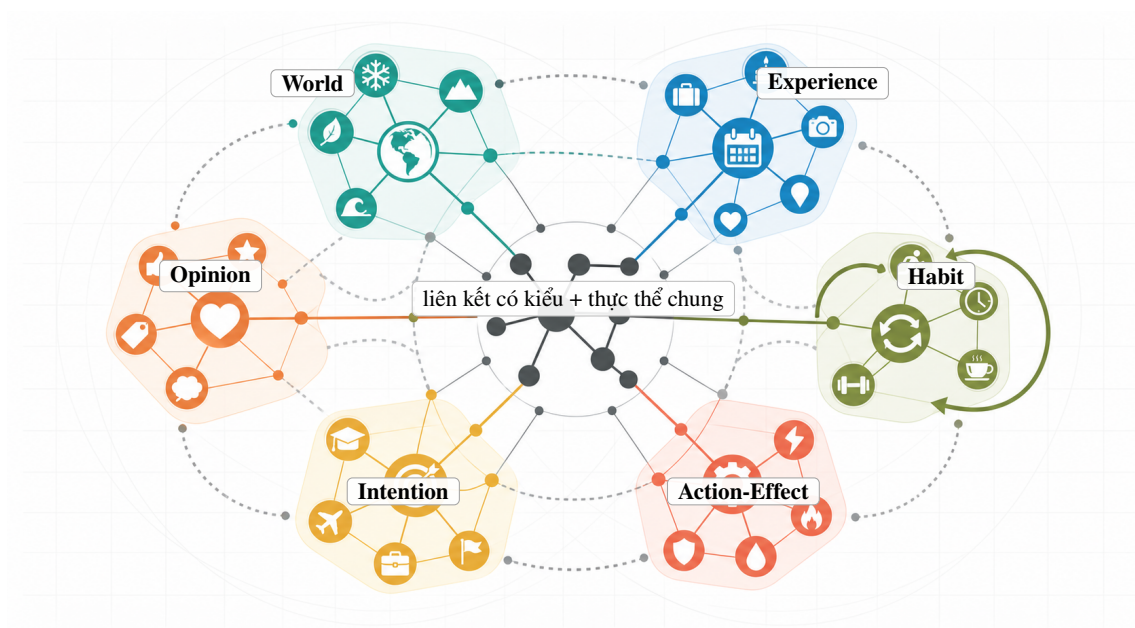
3.3.2 Các mạng phân định

CogMem sử dụng sáu loại thông tin cốt lõi với các tên gọi *world*, *experience*, *opinion*, *habit*, *intention* và *action-effect* là nhãn biểu diễn trong hệ thống; mỗi nhãn cho biết fact đó nên được hiểu và truy vấn theo cách nào. Mỗi loại được chọn vì nó tương ứng với một nhu cầu truy vấn khác nhau trong hội thoại dài hạn.

1. **World:** lưu tri thức khách quan hoặc tương đối ổn định. Đây là các thông tin như nghề nghiệp, kỹ năng, tổ chức, mối quan tâm dài hạn hoặc tri thức chung. World khác experience ở chỗ nó không nhất thiết gắn với một lần xảy ra cụ thể. Ví dụ, “người dùng là kỹ sư phần mềm” là world; “người dùng phỏng vấn ở một công ty hôm qua” là experience.
2. **Experience:** lưu sự kiện cá nhân đã xảy ra, thường có thời gian, bối cảnh hoặc

thứ tự trước–sau. Đây là loại thông tin phù hợp với câu hỏi “đã xảy ra chuyện gì”, “khi nào”, hoặc “sau sự kiện nào”. Điểm quan trọng là experience luôn nói về một việc đã diễn ra, không phải kế hoạch hay thói quen.

3. **Opinion:** lưu đánh giá, niềm tin, sở thích hoặc lập trường chủ quan. Loại này có thể đi kèm độ tin cậy vì quan điểm có thể yếu, mạnh hoặc thay đổi theo thời gian. Ví dụ, “người dùng thích Python hơn JavaScript cho thử nghiệm nhanh” là opinion, không phải world, vì đó là đánh giá cá nhân.
4. **Habit:** lưu mẫu hành vi lặp lại. Một habit không chỉ là một sự kiện đã xảy ra, mà là khuynh hướng “thường làm gì trong bối cảnh nào”. Vì vậy, habit thường được nối với nhiều experience cụ thể để những sự kiện đã quan sát được trở thành bằng chứng cho mẫu hành vi lặp lại.
5. **Intention:** lưu kế hoạch hoặc mục tiêu hướng tới tương lai. Một intention đại diện cho việc người dùng dự định làm nhưng tại thời điểm ghi nhớ có thể chưa xảy ra. Trạng thái *planning* mô tả kế hoạch đang tồn tại và chưa hoàn tất; *fulfilled* mô tả kế hoạch đã được thực hiện và thường có một experience mới làm bằng chứng; *abandoned* mô tả trường hợp người dùng đã hủy bỏ hoặc thay đổi mục tiêu nên kế hoạch không còn được theo đuổi.
6. **Action-Effect:** lưu quan hệ nhân quả ở cấp hành động, gồm điều kiện ban đầu, hành động được chọn và kết quả quan sát được. Loại này phục vụ các câu hỏi như “làm cách nào để khắc phục”, “vì sao cách đó hiệu quả”, hoặc “hành động nào dẫn tới kết quả này”.



Hình 3.2: Sáu mạng bộ nhớ của CogMem, tương ứng với sáu loại thông tin có vai trò khác nhau trong truy vấn hội thoại dài hạn.

Bảng 3.1: Các loại fact cốt lõi và siêu dữ liệu chuyên biệt

Loại fact	Vai trò biểu diễn	Siêu dữ liệu / tín hiệu chuyên biệt
world	Kiến thức khách quan, mô tả ổn định	text, entities, raw_snippet
experience	Sự kiện cá nhân đã xảy ra	occurred_start, occurred_end, mentioned_at
opinion	Đánh giá hoặc sở thích chủ quan	confidence
habit	Mẫu hành vi lặp lại	liên kết s_r_link đến các experience liên quan
intention	Kế hoạch và mục tiêu tương lai	intention_status; có thể mang thêm mô tả lý do hoặc tín hiệu thời gian từ bước trích xuất
action_effect	Quy luật nhân quả ở cấp hành động	precondition, action, outcome, confidence, devalue_sensitive

Các tên trường trong bảng như *occurred_start*, *confidence* hay *precondition* là tên ngắn gọn trong triển khai, nhưng ý nghĩa của chúng đơn giản hơn: thời điểm bắt đầu sự kiện, độ tin cậy của thông tin, và điều kiện ban đầu của một hành động. Nhờ các trường này, hai fact có câu chữ gần giống nhau vẫn có thể được xử lý khác nhau nếu một fact là sự kiện đã xảy ra còn fact kia là kế hoạch tương lai.

Việc tách riêng ba loại habit, intention và action-effect không nhằm khẳng định chúng luôn cải thiện mọi tập dữ liệu. Mục tiêu của chúng là mở thêm không gian biểu diễn cho những tín hiệu mà world hoặc experience dễ mô tả thiếu: hành vi lặp lại, kế hoạch chưa hoàn thành và tri thức can thiệp kiểu “nếu ở điều kiện P, làm hành động A thì tạo ra kết quả O”. Dạng tri thức này khác với tri thức mô tả đơn thuần vì nó giúp hệ thống suy luận về hành động có thể làm thay đổi kết quả. Trong các thí nghiệm hiện tại, habit là nhóm cần diễn giải thận trọng nhất vì benchmark chuẩn chưa có nhiều câu hỏi thật sự đòi hỏi mô hình hóa thói quen lặp lại qua nhiều tuần hoặc nhiều phiên.

a, Trí nhớ ngữ nghĩa và mạng World

Trí nhớ ngữ nghĩa lưu tri thức không phụ thuộc chặt vào một tình tiết riêng lẻ. Trong CogMem, world fact đảm nhận vai trò này. Khi người dùng nói “tôi là kỹ sư phần mềm” hoặc “DI là công ty làm về hạ tầng LLM”, hệ thống nên lưu đó như tri thức tương đối ổn định, không phải như một experience chỉ có giá trị trong một phiên. LLM là mô hình ngôn ngữ lớn, loại mô hình AI có khả năng đọc, sinh và suy luận trên văn bản.

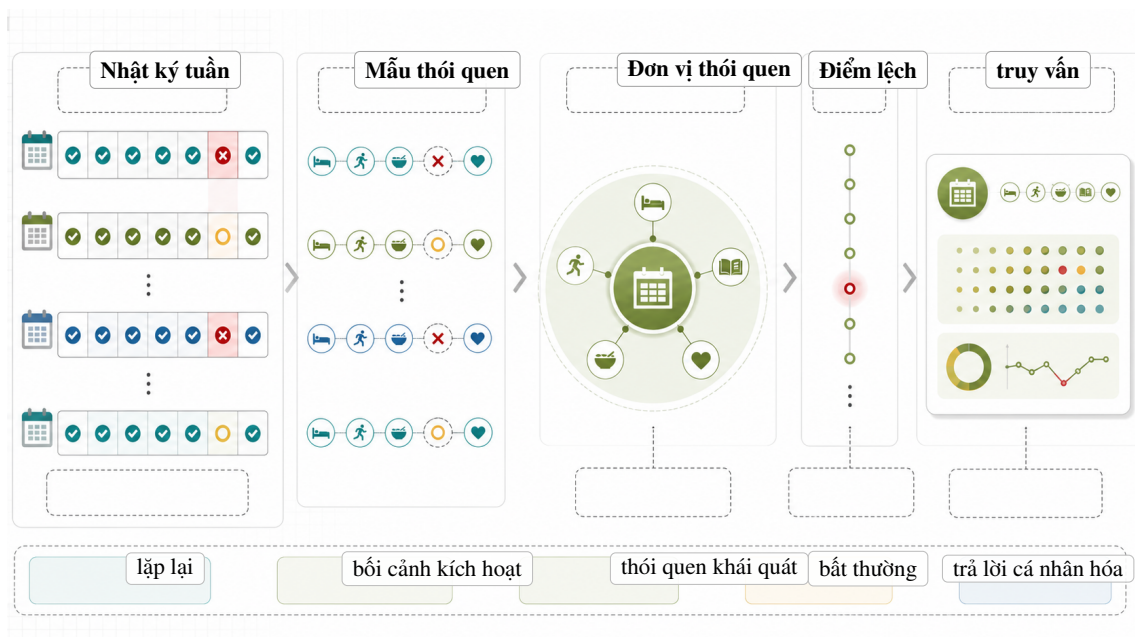
Sự phân biệt này giúp truy vấn đúng trọng tâm hơn, cụ thể là các câu hỏi về danh tính, năng lực, nơi làm việc hoặc tri thức chuyên ngành nên ưu tiên world

fact. Ngược lại, câu hỏi về việc gì đã xảy ra hôm qua hoặc sau một biến cố cụ thể nên ưu tiên experience.

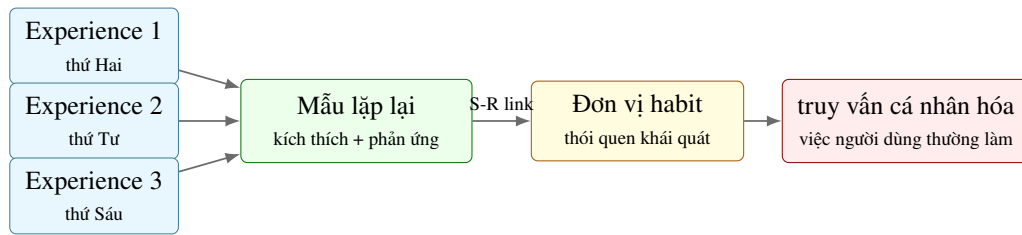
b, Habit Memory và Habit Network

Habit memory liên quan đến các mẫu hành vi được lặp lại trong bối cảnh quen thuộc. Trong CogMem, habit biểu diễn các câu kiểu “người dùng thường làm gì”, “hay chọn gì”, hoặc “thường phản ứng ra sao khi gặp một tình huống”. Một experience nói về một lần xảy ra; một habit nói về một mẫu ổn định hơn được tổng quát từ nhiều lần. Vì vậy, nếu người dùng một lần nói “sáng nay tôi kiểm tra email trước họp”, đó chưa đủ là habit; nếu nhiều phiên cho thấy hành vi này lặp lại, hệ thống mới có cơ sở mạnh hơn để xem đó là thói quen.

Về triển khai, CogMem dùng liên kết kích thích–phản ứng để nối habit với các experience có liên quan. Kích thích là bối cảnh làm hành vi xuất hiện, ví dụ “trước standup”; phản ứng là hành vi quen thuộc, ví dụ “kiểm tra email”. Cách làm này không phải một mô hình học tăng cường đầy đủ, nhưng đủ để phân biệt mẫu lặp lại với sự kiện đơn lẻ và hỗ trợ các câu hỏi cá nhân hóa như “người dùng thường chuẩn bị thế nào trước cuộc họp”.



Hình 3.3: Một chuỗi nhật ký nhiều tuần phù hợp hơn để kiểm tra Habit Network so với benchmark chỉ hỏi fact đơn lẻ.



Hình 3.4: Habit Memory và Habit Network

c, Prospective Memory và Intention Network

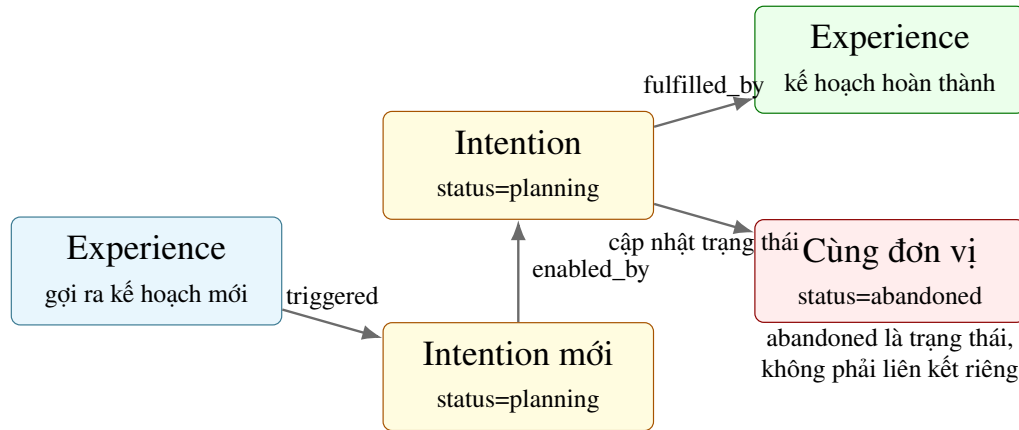
Prospective memory là năng lực giữ trong trí nhớ một việc cần làm ở tương lai. Nhờ dạng trí nhớ này, một người không chỉ nhớ điều đã xảy ra, mà còn nhớ rằng mình đang có một nhiệm vụ hoặc mục tiêu chưa hoàn thành. Trong hội thoại dài hạn, loại tín hiệu này rất quan trọng vì người dùng thường nói về kế hoạch, mục tiêu và việc chưa làm. Nếu hệ thống chỉ có world và experience, câu “tôi định học Rust trong quý tới” dễ bị hiểu mơ hồ như một thông tin ổn định hoặc một sự kiện đã xảy ra.

CogMem biểu diễn kế hoạch bằng trường *intention_status*, dùng để theo dõi kế hoạch đang ở giai đoạn nào trong vòng đời của nó. Ba trạng thái có ý nghĩa cụ thể:

- *planning*: kế hoạch đang tồn tại, người dùng chưa nói rằng nó đã hoàn thành hoặc bị hủy bỏ.
- *fulfilled*: kế hoạch đã được thực hiện; hệ thống có thể tạo hoặc liên kết tới một experience mô tả việc hoàn thành.
- *abandoned*: kế hoạch đã bị hủy, bị thay thế, hoặc người dùng không còn theo đuổi mục tiêu đó nữa. Đây là trạng thái của chính intention, không phải một loại cạnh riêng.

Bảng 3.2: Vòng đời của intention và ý nghĩa các liên kết chuyển trạng thái

Tình huống	Biểu diễn trên intention	Ý nghĩa chuyển trạng thái
Lập kế hoạch mới	status = planning	kế hoạch chưa cần liên kết chuyển trạng thái
Kế hoạch được thực hiện	status có thể cập nhật; experience mới được tạo	fulfilled_by: intention → experience
Sự kiện gợi ra kế hoạch mới	intention planning mới xuất hiện	triggered: experience → intention
Kế hoạch phụ thuộc kế hoạch khác	hai intention cùng tồn tại	enabled_by: intention → intention
Kế hoạch bị hủy bỏ	status = abandoned	không cần cạnh riêng; trạng thái nằm trên intention

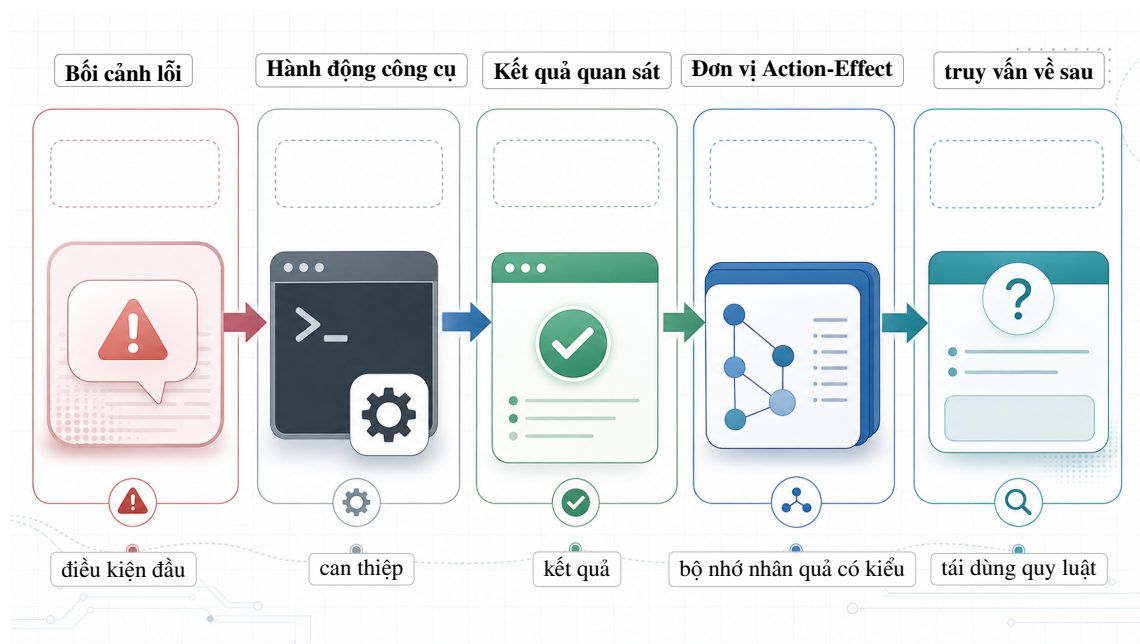


Hình 3.5: Intention Lifecycle

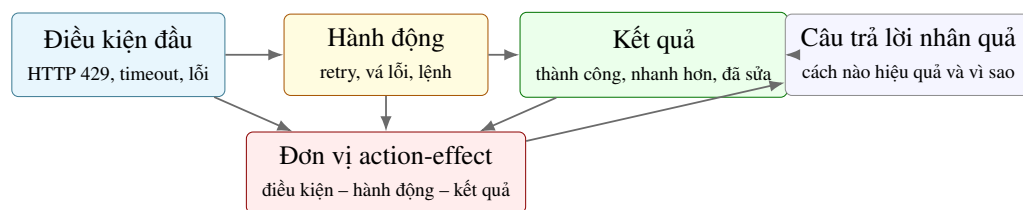
d, Theory of Event Coding và Action-Effect Network

Theory of Event Coding nhấn mạnh rằng hành động và hệ quả cảm nhận được của hành động thường được biểu diễn cùng nhau. Nói theo ngôn ngữ đời thường, khi ta học một hành động, ta thường nhớ luôn kết quả mà hành động đó tạo ra. Với tác nhân hội thoại hoặc tác nhân sử dụng công cụ, đây là điểm rất quan trọng: hệ thống không chỉ cần nhớ “lỗi đã xảy ra”, mà còn cần nhớ “trong điều kiện đó, hành động nào đã dẫn tới kết quả nào”.

CogMem hiện thực hóa ý tưởng này bằng action-effect, một loại thông tin gồm ba phần: *precondition* là điều kiện ban đầu, *action* là hành động được chọn, và *outcome* là kết quả quan sát được. Ví dụ, precondition có thể là “API trả về timeout”, action là “chạy lại lệnh với retry”, và outcome là “tác vụ hoàn tất”. Cấu trúc này hữu ích hơn một world fact chung chung khi câu hỏi yêu cầu giải thích nguyên nhân hoặc tái sử dụng một quy luật can thiệp đã từng thành công.



Hình 3.6: Action-effect phù hợp với trace AI agent thật, nơi hệ thống cần nhớ quan hệ giữa điều kiện lỗi, hành động công cụ và kết quả quan sát được.

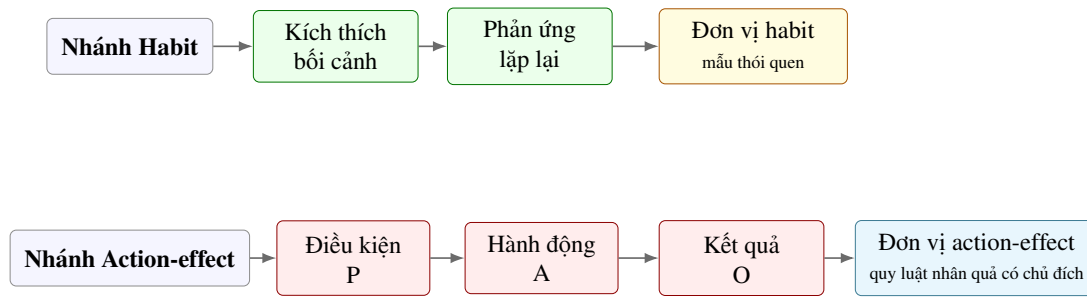


Hình 3.7: Theory of Event Coding và Action-Effect Network

3.3.3 Khác biệt cốt lõi giữa Habit và Action-Effect

Habit và action-effect đều liên quan đến hành vi, nhưng chúng không biểu diễn cùng một hiện tượng. Habit là mẫu kích thích–phản ứng hình thành qua lặp lại: gặp bối cảnh quen thuộc thì hành vi quen thuộc xuất hiện. Action-effect là quy luật nhân quả có chủ đích, trong đó một hành động được chọn vì người dùng hoặc tác nhân kỳ vọng nó tạo ra kết quả mong muốn.

Ví dụ, “người dùng thường kiểm tra email trước standup” là habit vì nó mô tả một thói quen, một chuỗi hành vi quen thuộc. Ngược lại, “khi API trả về timeout, chạy lại lệnh với tham số retry giúp hoàn tất tác vụ” là action-effect vì nó mô tả một quan hệ điều kiện–hành động–kết quả có thể được dùng để suy luận nguyên nhân.



Hình 3.8: Phân biệt Habit và Action-Effect

Sự phân biệt này cũng giúp thiết kế thí nghiệm rõ hơn. Với câu hỏi về sở thích hoặc thói quen lặp lại, habit là lựa chọn hợp lý hơn vì nó chứa tín hiệu lặp lại. Với câu hỏi về “vì sao” hoặc “làm gì để đạt kết quả”, action-effect mới là biểu diễn trực tiếp hơn vì nó chứa quan hệ nhân quả. Nếu gộp hai loại này thành một fact chung, hệ thống khó biết nên truy vấn theo tần suất hành vi hay theo quan hệ nhân quả.

3.3.4 Bản chất ngữ nghĩa của các liên kết đồ thị

Một đồ thị bộ nhớ chỉ hữu ích khi các liên kết có ý nghĩa rõ ràng. Mỗi liên kết trả lời cho câu hỏi: “hai đơn vị thông tin này liên quan với nhau theo cách nào?” CogMem dùng bảy loại liên kết, mỗi loại biểu diễn một quan hệ khác nhau giữa các đơn vị thông tin:

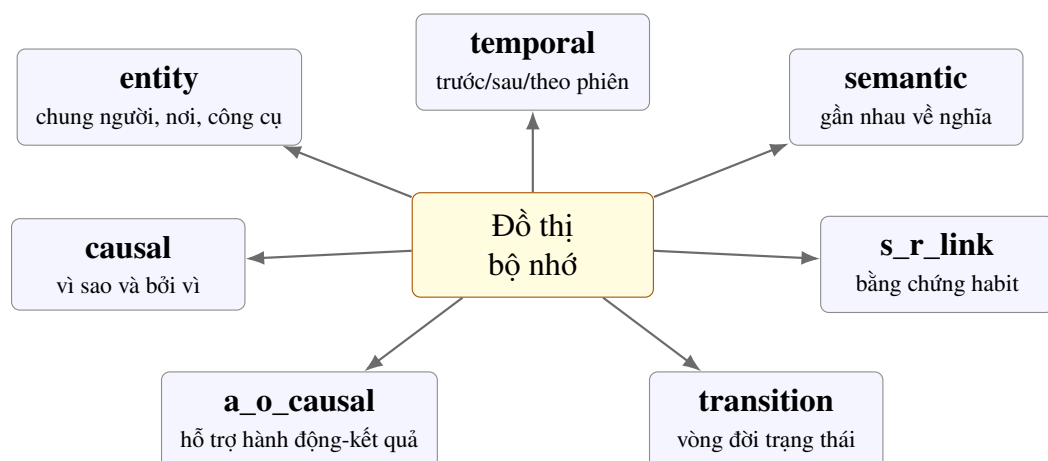
1. **entity**: nối các đơn vị thông tin cùng nhắc tới một thực thể, ví dụ cùng người, dự án, công ty hoặc công cụ. Nếu hai fact đều nhắc tới “Project Alpha”, liên kết entity giúp hệ thống gom chúng lại.
2. **temporal**: mô tả thứ tự thời gian hoặc quan hệ trước–sau giữa các sự kiện. Loại liên kết này giúp trả lời các câu hỏi như “sau khi đổi việc thì người dùng làm gì”.
3. **semantic**: nối các đơn vị thông tin có nội dung gần nhau về nghĩa, thường dựa trên độ tương đồng embedding. Ví dụ, “học Rust” và “viết inference server bằng Rust” có thể gần nhau về ngữ nghĩa dù không trùng toàn bộ từ khóa.
4. **causal**: biểu diễn quan hệ nguyên nhân, ảnh hưởng hoặc giải thích giữa các fact. Đây là liên kết cho câu hỏi kiểu “vì sao”.
5. **s_r_link**: nối habit với các experience cùng mẫu kích thích–phản ứng. Ký hiệu S-R là viết tắt của stimulus-response, tương ứng với kích thích và phản ứng.
6. **a_o_causal**: nối action-effect với các fact quan sát được làm bằng chứng cho quan hệ hành động–kết quả. Ký hiệu A-O là viết tắt của action-outcome, tương ứng với hành động và kết quả.
7. **transition**: biểu diễn thay đổi trạng thái của cùng một ý niệm theo thời gian,

ví dụ một kế hoạch được hoàn thành hoặc một quan điểm được sửa đổi. Nếu không có transition, hệ thống có thể biết hai fact cùng nói về Rust nhưng không biết fact sau đã hoàn tất kế hoạch ở fact trước.

Transition là một nhóm liên kết quan trọng vì nó giữ lại quá trình thay đổi của thông tin theo thời gian. Một thông tin có thể bắt đầu như kế hoạch, sau đó trở thành trải nghiệm đã hoàn thành, hoặc bị hủy bỏ khi mục tiêu thay đổi. Trong triển khai hiện tại, các kiểu chuyển trạng thái chính gồm:

- `fulfilled_by`: intention $\rightarrow$ experience, khi một kế hoạch được thực hiện.
- `triggered`: experience $\rightarrow$ intention, khi một sự kiện làm nảy sinh kế hoạch mới.
- `enabled_by`: intention $\rightarrow$ intention, khi một kế hoạch là điều kiện để kế hoạch khác trở nên khả thi.
- `revised_to`: opinion $\rightarrow$ opinion, khi người dùng điều chỉnh quan điểm.
- `contradicted_by`: world $\rightarrow$ world, khi một thông tin ổn định bị cập nhật hoặc phủ định bởi thông tin mới.

abandoned là trạng thái của intention, không phải một liên kết transition riêng. Thiết kế này giúp hệ thống vẫn nhớ rằng kế hoạch từng tồn tại, đồng thời biết rằng nó không còn được theo đuổi.



Hình 3.9: Bảy loại liên kết trong CogMem

3.4 Biểu diễn song song: ghi nhớ tóm tắt và dấu vết nguyên văn

3.4.1 Vấn đề nén dữ liệu có mất thông tin

Các hệ thống bộ nhớ dựa trên trích xuất thường phải nén hội thoại dài thành một số fact ngắn để lập chỉ mục và truy vấn hiệu quả. Bước lập chỉ mục chuẩn bị dữ liệu thành dạng có thể tìm kiếm nhanh, tương tự tạo mục lục tự động cho một kho

lưu trữ hội thoại. Cách làm này cần thiết, nhưng nó gây mất mát thông tin. Những chi tiết như con số chính xác, tên riêng, lời nói nguyên văn, cảm xúc hoặc bối cảnh nhỏ có thể biến mất khỏi fact tóm tắt.

Original conversation:

"I just quit my job at VCCorp today.
I'm really sad because my old team was
like family. DI offered me 40% more pay.
I start there on April 1st."

Compressed fact:

"User switched from VCCorp to DI in
April 2024."

Fact tóm tắt trên đủ để tìm đúng chủ đề “chuyển việc”, nhưng không còn giữ chi tiết “tăng 40% lương”, cảm xúc buồn vì rời đội cũ, hay ngày bắt đầu chính xác. Nếu người dùng hỏi “mức lương mới cao hơn bao nhiêu”, hệ thống có thể truy vấn đúng fact nhưng vẫn không đủ bằng chứng để trả lời.

3.4.2 Fuzzy Trace Theory và giải pháp hai lớp

Fuzzy Trace Theory của Brainerd và Reyna cho rằng con người có thể lưu đồng thời hai dạng dấu vết trí nhớ. Dấu vết *gist* giữ phần tóm tắt ý nghĩa, còn dấu vết *verbatim* giữ chi tiết nguyên văn. Tên “fuzzy trace” không có nghĩa là trí nhớ mờ hồ theo nghĩa xấu; nó nhấn mạnh rằng ý chính và chi tiết có thể cùng tồn tại, nhưng phục vụ các nhiệm vụ khác nhau. CogMem chuyển nguyên tắc này thành schema hai lớp, một cấu trúc trường dữ liệu dùng để lưu một đơn vị bộ nhớ:

- **fact text / embedding**: lớp tóm tắt ngữ nghĩa, ngắn gọn và phù hợp cho truy vấn nhanh. *Fact text* là câu tóm tắt đọc được bằng ngôn ngữ tự nhiên; *embedding* là biểu diễn số của câu đó.
- **raw_snippet**: lớp nguyên văn, giữ lại bằng chứng gốc để hỗ trợ sinh câu trả lời chi tiết. Trường này đặc biệt hữu ích khi câu hỏi cần số liệu hoặc cách diễn đạt chính xác.

```
{  
  "fact_type": "experience",  
  "text": "User switched from VCCorp to DI  
    in April 2024",  
  "embedding": [...],  
  "raw_snippet": "I just quit my job at  
    VCCorp today... DI offered  
    me 40% more pay..."
```

```
"metadata": {  
  "raw_snippet_present": true  
}
```

Hai lớp này phục vụ hai mục tiêu khác nhau. Lớp tóm tắt giúp truy vấn vì nó ngắn, rõ chủ đề và dễ biểu diễn thành vector. Lớp nguyên văn tạo grounding cho bước sinh câu trả lời: mô hình có bằng chứng gốc để bám vào thay vì chỉ dựa trên suy đoán từ fact đã nén. Nhờ vậy, CogMem không buộc hệ thống phải chọn giữa truy vấn hiệu quả và bảo toàn bằng chứng.



Hình 3.10: Schema hai lớp cho một đơn vị bộ nhớ



Hình 3.11: Fuzzy Trace Theory và Raw Snippet

3.5 Truy vấn đa kênh và kết hợp bằng chứng

3.5.1 Vì sao không dùng một kênh duy nhất

Một câu hỏi hội thoại mang tính dài hạn có thể cần nhiều kiểu bằng chứng khác nhau. Câu hỏi “người dùng thích công cụ nào” thường cần bằng chứng ngữ nghĩa hoặc quan điểm; câu hỏi “việc đó xảy ra khi nào” cần dấu vết thời gian; câu hỏi “vì sao cách sửa đó hiệu quả” cần quan hệ nhân quả; còn câu hỏi suy luận nhiều bước cần đi qua các liên kết giữa nhiều fact.

Vì vậy, CogMem không chọn một cơ chế tìm kiếm duy nhất cho mọi câu hỏi mà lựa chọn luồng truy vấn đa kênh nhận cùng một câu hỏi đầu vào và chạy nhiều cách tìm kiếm song song trên cùng kho bộ nhớ. Mỗi kênh trả về một danh sách ứng viên kèm điểm riêng. Điểm của các kênh không được so sánh trực tiếp với nhau, vì

điểm cosine của embedding, điểm BM25, điểm kích hoạt đồ thị và điểm thời gian có thang đo khác nhau. Do đó, CogMem kết hợp chúng ở mức thứ hạng thay vì cộng trực tiếp điểm gốc.

3.5.2 Bốn kênh truy vấn song song

Bốn kênh truy vấn có vai trò bổ sung cho nhau:

1. **Kênh ngữ nghĩa:** chuyển câu hỏi thành vector và so sánh với vector của fact đã lưu. Kênh này mạnh khi câu hỏi dùng từ khác với hội thoại gốc nhưng vẫn gần nghĩa, ví dụ “công việc mới” gần với “chuyển sang DI”. Điểm yếu của nó là có thể bỏ qua chi tiết chính xác như con số, ngày tháng hoặc tên riêng hiếm.
2. **Kênh từ khóa BM25:** tìm các fact hoặc đoạn liên quan dựa trên từ và cụm từ xuất hiện trong câu hỏi. BM25 hữu ích khi câu hỏi chứa tên riêng, công cụ, mã lỗi hoặc cụm từ đặc thù. Điểm yếu của nó là khó tìm được bằng chứng nếu người dùng diễn đạt cùng ý bằng từ khác.
3. **Kênh đồ thị:** bắt đầu từ các đơn vị thông tin gần câu hỏi rồi đi qua các liên kết như entity, temporal, causal, s_r_link, a_o_causal và transition. Kênh này hữu ích cho câu hỏi nhiều bước hoặc câu hỏi cần nối trải nghiệm, kế hoạch và quan hệ nhân quả. Điểm yếu của nó là phụ thuộc vào chất lượng liên kết đã tạo ở bước ghi nhớ.
4. **Kênh thời gian:** ưu tiên các fact có mốc thời gian phù hợp với câu hỏi, chẳng hạn “hôm qua”, “tuần trước”, “sau khi đổi việc” hoặc một tháng cụ thể. Kênh này mạnh với câu hỏi có ràng buộc thời gian rõ, nhưng ít hữu ích nếu câu hỏi không chứa tín hiệu thời gian.

Kết quả của bốn kênh được giữ tách biệt cho đến bước kết hợp. Cách tổ chức này giúp hệ thống không phụ thuộc hoàn toàn vào một tín hiệu đơn lẻ. Một bằng chứng có thể không đứng đầu ở kênh ngữ nghĩa nhưng vẫn được nâng lên nếu nó cũng xuất hiện trong kênh từ khóa, đồ thị hoặc thời gian.

3.5.3 Kết hợp bằng chứng bằng weighted RRF

CogMem dùng weighted Reciprocal Rank Fusion để trộn các danh sách ứng viên. Reciprocal Rank Fusion, viết tắt là RRF, được tính bằng cộng điểm theo thứ hạng: bằng chứng ở hạng cao nhận điểm lớn hơn, và bằng chứng xuất hiện ở nhiều kênh sẽ được cộng điểm từ nhiều nguồn. Phiên bản có trọng số cho phép mỗi kênh đóng góp khác nhau tùy loại câu hỏi.

$$\text{RRF}_{\text{weighted}}(d, q) = \sum_i w_i(q) \cdot \frac{1}{60 + \text{rank}_i(d)} \quad (3.1)$$

Trong công thức này, d là một bằng chứng ứng viên, q là câu hỏi, i là kênh truy vấn, $w_i(q)$ là trọng số của kênh i đối với loại câu hỏi q , và $\text{rank}_i(d)$ là thứ hạng của d trong kênh đó. Hằng số 60 làm giảm chênh lệch quá lớn giữa các hạng đầu, nhờ đó một bằng chứng tốt ở nhiều kênh vẫn có thể vượt một bằng chứng chỉ mạnh ở một kênh.

3.5.4 Xếp hạng lại bằng CrossEncoder

Sau khi RRF tạo danh sách ứng viên thống nhất, CogMem có thể dùng CrossEncoder để xếp hạng lại các bằng chứng mạnh nhất. Khác với kênh ngữ nghĩa dùng hai vector tách rời rồi so sánh khoảng cách, CrossEncoder đọc đồng thời cặp *câu hỏi–bằng chứng* và ước lượng trực tiếp mức độ phù hợp của bằng chứng với câu hỏi. Vì vậy, bước này phù hợp với các trường hợp cần phân biệt tinh hơn giữa hai fact có từ khóa giống nhau nhưng trả lời các ý khác nhau.

Trong pipeline đầy đủ, CrossEncoder nằm sau RRF: RRF quyết định những bằng chứng nào đủ tốt để được xem xét, còn CrossEncoder sắp xếp lại cửa sổ ứng viên trước khi đưa sang bước sinh câu trả lời. Riêng trong thí nghiệm so sánh SUM và MAX trên kênh đồ thị, bước xếp hạng lại này được tắt để kết quả phản ánh riêng khác biệt giữa hai cách lan truyền tín hiệu trên đồ thị, thay vì bị trộn thêm bởi một bộ chấm điểm sau truy vấn.

3.6 Truy vấn trên đồ thị tri thức

3.6.1 Nhược điểm của toán tử MAX và khái niệm Episodic Buffer

Trong các câu hỏi nhiều bước, bằng chứng đúng thường không nằm ở một fact duy nhất. Nó có thể xuất hiện như sự hội tụ của nhiều dấu vết nhỏ: một mốc thời gian, một thực thể chung, một sự kiện trước đó và một đoạn trích nguyên văn. Nếu truy vấn đồ thị chỉ lấy đường có tín hiệu mạnh nhất, các dấu vết yếu hơn sẽ bị bỏ qua dù chúng cùng hướng tới một câu trả lời.

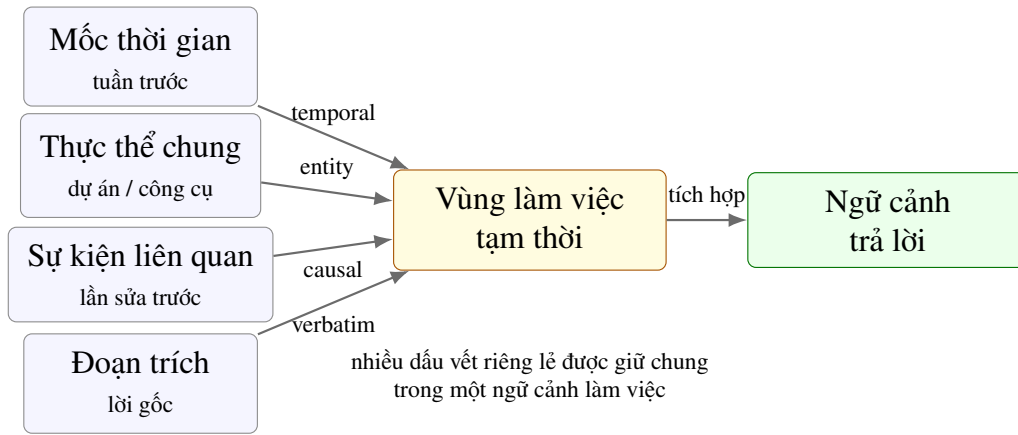
Trong truy vấn đồ thị, cơ chế lấy cực đại, hay MAX, chỉ giữ đường có điểm cao nhất khi có nhiều đường dẫn cùng đi tới một đơn vị thông tin. Cơ chế này có thể viết như sau:

$$A(v, t + 1) = \max_{u \in N(v)} [A(u, t) \cdot w(u, v) \cdot \delta \cdot \mu(\ell)] \quad (3.2)$$

Trong công thức này, u là đơn vị nguồn, v là đơn vị đích, t là bước lan truyền, A là mức kích hoạt, $N(v)$ là tập láng giềng có thể truyền tín hiệu vào v , $w(u, v)$ là trọng số liên kết, δ là hệ số suy giảm theo bước lan truyền, và $\mu(\ell)$ là hệ số theo loại liên kết. Mức kích hoạt biểu thị mức độ “đang được chú ý” của một đơn vị thông

tin trong quá trình tìm kiếm.

Từ góc nhìn nhận thức, mô hình Episodic Buffer của Baddeley gợi ý một cách nhìn khác. Episodic Buffer đóng vai trò như vùng làm việc tạm thời, nơi nhiều mảnh thông tin được ghép lại thành một bức tranh chung. Khi con người tái dựng một tình tiết, họ thường tích hợp nhiều nguồn thông tin tạm thời thành một ngữ cảnh làm việc thống nhất, thay vì chỉ chọn một dấu vết mạnh nhất. CogMem lấy cảm hứng từ nguyên tắc này để thiết kế truy vấn đồ thị theo hướng cộng dồn bằng chứng.



Hình 3.12: Episodic Buffer của Baddeley

3.6.2 Lan truyền tín hiệu tích lũy

CogMem thay phép lấy cực đại bằng phép cộng dồn có giới hạn trên, gọi là SUM. Với SUM, nhiều đường bằng chứng cùng được cộng vào điểm của một đơn vị thông tin, còn giới hạn trên bảo đảm điểm không tăng vô hạn. Công thức truy vấn đồ thị mặc định là:

$$A(v, t + 1) = \text{clip} \left[A(v, t) + \delta \cdot \sum_{u \in N(v)} A(u, t) \cdot w(u, v) \cdot \mu(\ell) \cdot \text{refractory}(u), A_{\max} \right] \quad (3.3)$$

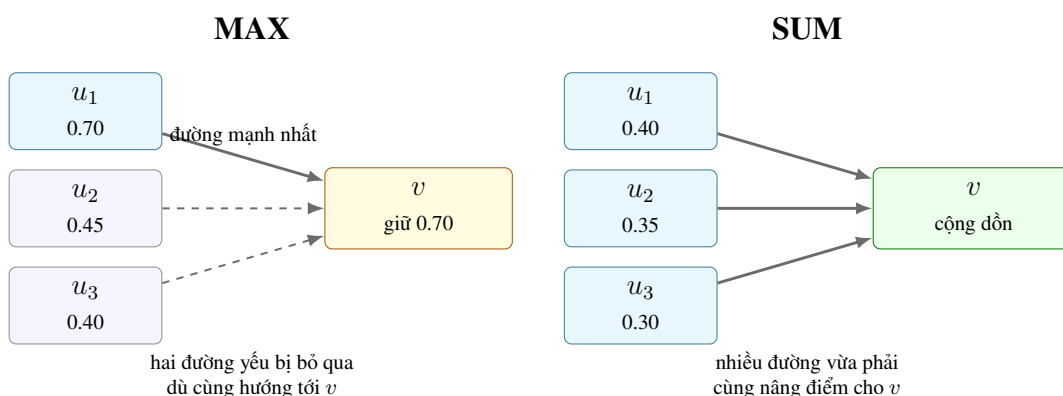
Các thành phần trong công thức SUM được hiểu như sau:

- $A(v, t + 1)$ là mức kích hoạt mới của đơn vị thông tin v sau một bước lan truyền; $A(v, t)$ là mức kích hoạt đã có của chính đơn vị đó ở bước trước.
- u là một đơn vị thông tin nguồn đang truyền tín hiệu sang v ; $N(v)$ là tập các đơn vị nguồn có liên kết đi vào v .
- $A(u, t)$ là mức kích hoạt hiện tại của nguồn u , còn $w(u, v)$ là độ mạnh của liên

kết từ u sang v .

- $\mu(\ell)$ điều chỉnh tín hiệu theo loại liên kết ℓ , ví dụ liên kết thời gian, ngữ nghĩa, nhân quả hoặc chuyển trạng thái có thể đóng góp khác nhau.
- δ là hệ số suy giảm theo bước lan truyền, giúp tín hiệu yếu dần khi đi xa khỏi các điểm khởi đầu.
- $\text{refractory}(u)$ là mặt nạ chống tái kích hoạt ngay: giá trị 0 chặn nguồn vừa phát tín hiệu, giá trị 1 cho phép nguồn đó tiếp tục đóng góp.
- $\text{clip}[\cdot, A_{\max}]$ là phép chặn trên, bảo đảm tổng kích hoạt không vượt quá ngưỡng $A_{\max}$.

Điểm khác biệt chính là nhiều nguồn u có thể cùng đóng góp vào kích hoạt của v . Nhờ đó, một đơn vị thông tin không cần phải được tìm thấy qua một đường duy nhất thật mạnh, nó có thể được nâng hạng nếu nhiều bằng chứng yếu cùng hội tụ về nó. Đây là trực giác quan trọng của CogMem: một câu trả lời có thể đáng tin không phải vì một bằng chứng quá mạnh, mà vì nhiều bằng chứng vừa phải cùng chỉ về một hướng.



Hình 3.13: Minh họa trực quan sự khác biệt giữa MAX và SUM trên cùng một cấu trúc đồ thị

Bảng 3.3: So sánh HINDSIGHT (MAX) và CogMem (SUM)

Khía cạnh	HINDSIGHT (MAX)	CogMem (SUM)
Bằng chứng đa đường	chỉ giữ đường mạnh nhất	cộng dồn nhiều đường liên quan
Yêu cầu điểm vào	cần trùng đơn vị thông tin rất mạnh ngay từ đầu	cho phép nhiều tín hiệu vừa phải bổ sung nhau
Độ bền với truy vấn nhiều bước	dễ mất chuỗi nếu một liên kết yếu	nhiều liên kết yếu có thể củng cố cùng kết quả
Rủi ro kỹ thuật	ít nguy cơ bùng nổ điểm	cần cơ chế kiểm soát chu trình và giới hạn điểm

Để kiểm tra đóng góp này một cách công bằng, thí nghiệm ở Chương 4 dùng một đối chứng hẹp: cùng kho bộ nhớ đã được ghi, cùng truy vấn chỉ bằng đồ thị, cùng tập loại fact, cùng bộ xếp hạng lại (reranking), chỉ thay cách tổng hợp điểm kích hoạt trên đồ thị từ SUM sang MAX. Vì vậy, khác biệt đo được phản ánh chủ yếu cách kênh đồ thị xếp hạng bằng chứng khi nhiều đường yếu cùng hội tụ về một câu trả lời.

3.6.3 Cơ sở thần kinh học và ba cơ chế bảo vệ chống nhiễu

Lan truyền cộng dồn trên đồ thị có thể gặp chu trình: đường đi theo các liên kết một lúc rồi quay lại đơn vị thông tin đã gặp trước đó. Nếu không kiểm soát, tín hiệu có thể dao động quanh cùng một cụm thông tin hoặc tăng điểm quá mức. CogMem dùng ba cơ chế bảo vệ:

1. **Local Refractory Period:** lấy cảm hứng từ refractory period trong thần kinh học, khoảng nghỉ ngắn sau khi neuron vừa phát tín hiệu. Trong CogMem, cơ chế này chặn một đơn vị vừa truyền tín hiệu khỏi việc được tái kích hoạt ngay ở bước kế tiếp. Nó làm giảm vòng lặp dao động tín hiệu giữa hai đơn vị thông tin.
2. **Global Firing Quota:** mỗi đơn vị chỉ được kích hoạt một số lần hữu hạn trong một lượt lan truyền. Khi một đơn vị đã đạt hạn mức kích hoạt, nó không tiếp tục nhận thêm điểm trong lượt đó. Cơ chế này chặn các chu trình dài hơn, nơi tín hiệu đi qua nhiều đơn vị rồi quay lại điểm cũ.
3. **Saturation Threshold:** điểm kích hoạt bị chặn trên bởi $A_{\max}$. Sau ngưỡng bão hòa này, điểm không tăng thêm nữa. Cơ chế này ngăn một nhóm bằng chứng lấn át toàn bộ danh sách kết quả và giúp giữ đa dạng ứng viên cho bước kết hợp và sinh câu trả lời.

Refractory period có thể viết dưới dạng một hệ số bật/tắt, hay mặt nạ: giá trị 0 chặn truyền tín hiệu, còn giá trị 1 cho phép truyền:

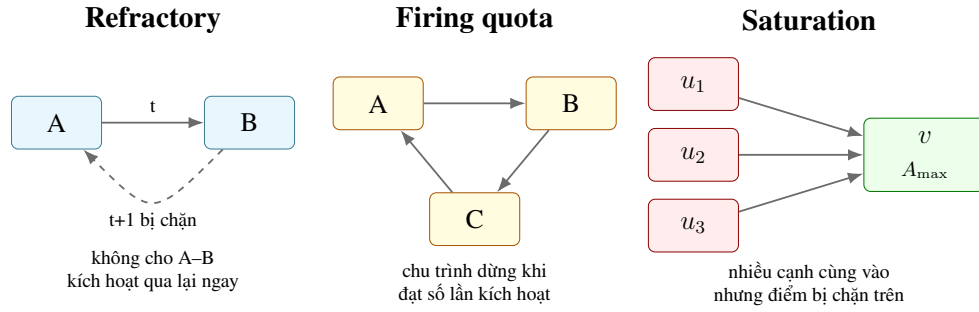
$$\text{refractory}(u) = \begin{cases} 0, & \text{nếu } u \text{ vừa được kích hoạt} \\ 1, & \text{ngược lại} \end{cases} \quad (3.4)$$

Saturation threshold tương ứng với phép chặn điểm:

$$A(v, t + 1) = \min(A(v, t + 1)_{\text{raw}}, A_{\max}) \quad (3.5)$$

Ba cơ chế này bổ sung cho nhau. Refractory period chặn dao động cục bộ, firing

quota chặn chu trình dài, còn saturation threshold chặn bùng nổ điểm ngay cả khi chu trình không rõ ràng.



Hình 3.14: Ba cơ chế bảo vệ chu trình

3.7 Định tuyến truy vấn thích ứng (Adaptive Query Routing)

3.7.1 Attentional Selection và vấn đề trọng số cố định

Trong nhận thức người, chú ý không phân bổ đều cho mọi nguồn thông tin. Câu hỏi “khi nào” khiến ta ưu tiên dấu vết thời gian; câu hỏi “vì sao” khiến ta tìm nguyên nhân; câu hỏi “tôi còn định làm gì” khiến ta tìm các kế hoạch chưa hoàn tất. CogMem mô hình hóa nguyên tắc này bằng adaptive query routing, hay định tuyến truy vấn thích ứng: trước khi tìm kiếm, hệ thống chọn chiến lược truy vấn phù hợp với nội dung câu hỏi.

Nếu mọi câu hỏi đều dùng cùng một bộ trọng số cho bốn kênh đã mô tả ở Mục 3.5, hệ thống dễ truy vấn lệch trọng tâm. Một câu hỏi về nguyên nhân-kết quả cần ưu tiên bằng chứng đồ thị và các quan hệ action-effect hơn một câu hỏi mô tả đơn giản. Một câu hỏi về kế hoạch tương lai hoặc việc chưa làm cần ưu tiên intention đang ở trạng thái planning hơn các experience đã xảy ra. Vì vậy, bước định tuyến không tạo thêm kênh truy vấn mới; nó điều chỉnh cách các kênh hiện có đóng góp vào danh sách bằng chứng cuối cùng.

3.7.2 Phân loại truy vấn và kết hợp theo nội dung câu hỏi

CogMem phân loại truy vấn thành sáu nhóm: *semantic* cho câu hỏi mô tả chung theo ý nghĩa, *temporal* cho câu hỏi về thời gian, *causal* cho câu hỏi về nguyên nhân, *prospective* cho câu hỏi về kế hoạch tương lai, *preference* cho câu hỏi về sở thích hoặc khuynh hướng, và *multi-hop* cho câu hỏi cần nối nhiều bằng chứng. Trong phiên bản triển khai hiện tại, việc phân loại này chủ yếu dựa trên các mẫu từ vựng và dấu hiệu thời gian: ví dụ các từ như “why” hoặc “cause” gợi ý causal, các từ như “plan” hoặc “future” gợi ý prospective, còn các biểu thức ngày tháng được nhận diện bằng bộ phân tích thời gian.

Kết quả phân loại này không trực tiếp tạo câu trả lời; nó quyết định trọng số

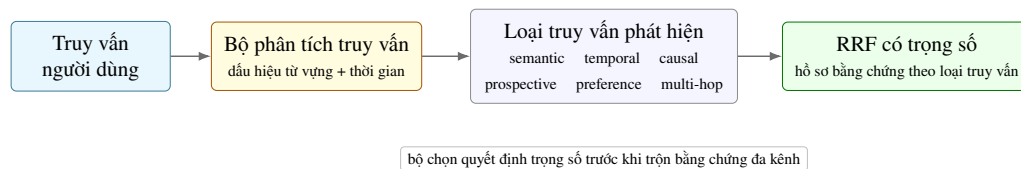
$w_i(q)$ trong công thức RRF có trọng số. Bảng 3.4 liệt kê cấu hình trọng số dùng trong phiên bản triển khai hiện tại.

Bảng 3.4: Trọng số RRF thích ứng theo loại truy vấn trong phiên bản triển khai hiện tại

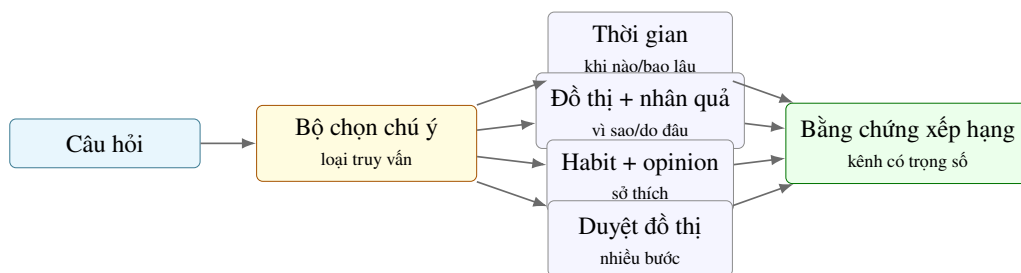
Query type	semantic	BM25	graph	temporal / ghi chú
semantic	1.0	1.0	1.0	1.0
temporal	0.8	0.6	0.8	2.2; ưu tiên ràng buộc thời gian
causal	0.8	0.7	2.4	1.0; ưu tiên action-effect và bằng chứng nhân quả
prospective	0.9	0.8	2.0	1.4; ưu tiên intention đang lập kế hoạch và bằng chứng chuyển trạng thái
preference	1.0	1.2	1.4	0.5; ưu tiên habit và opinion
multi-hop	0.9	0.7	2.6	0.8; ưu tiên đi theo liên kết đồ thị

Ngoài trọng số kênh, CogMem còn áp dụng các bước hậu xử lý:

- **Ưu tiên theo loại thông tin:** câu hỏi causal đưa action-effect và các chuỗi bằng chứng có liên kết nhân quả lên vị trí cao hơn trong danh sách kết quả; câu hỏi preference ưu tiên opinion và habit; câu hỏi prospective ưu tiên intention đang ở trạng thái planning.
- **Lọc kế hoạch tương lai:** với câu hỏi về việc sắp làm hoặc còn định làm, hệ thống ưu tiên intention chưa hoàn thành. Nếu thí nghiệm loại bỏ intention network, bộ định tuyến tự chuyển sang các fact còn lại để tránh thiên vị vào một loại thông tin không tồn tại. Cơ chế này giúp các thí nghiệm loại bỏ thành phần vẫn công bằng: hệ thống không bị ép tìm trong một mạng đã bị tắt.
- **Ưu tiên bằng chứng có nền đồ thị hoặc thời gian:** những bằng chứng được nhiều kênh cùng hỗ trợ thường đáng tin hơn bằng chứng chỉ xuất hiện trong một kênh.



Hình 3.15: Adaptive Query Routing



Hình 3.16: Attentional Selection và Adaptive Routing

Như vậy, định tuyến truy vấn thích ứng không chỉ là một thủ thuật đổi trọng số mà đây là cách CogMem đưa nguyên tắc lựa chọn chú ý vào hệ thống truy vấn: trước khi tìm bằng chứng, hệ thống xác định câu hỏi đang cần loại bằng chứng nào. Điều này giúp luồng truy vấn bám sát nội dung truy vấn hơn, đặc biệt với các câu hỏi về thời gian, nhân quả, sở thích, kế hoạch tương lai và suy luận nhiều bước.

3.8 Sinh câu trả lời

3.8.1 Tạo ngữ cảnh cho mô hình sinh

Sau khi các kênh truy vấn đã được kết hợp và xếp hạng, CogMem không đưa toàn bộ bộ nhớ vào mô hình sinh câu trả lời. Hệ thống chỉ chọn một số bằng chứng mạnh nhất, chuẩn hóa chúng thành các mục có cùng cấu trúc, rồi dựng prompt cho bước tổng hợp. Mỗi mục bằng chứng gồm định danh bộ nhớ, loại fact, nội dung tóm tắt, điểm xếp hạng, nguồn điểm và đoạn trích nguyên văn nếu có.

Việc chuẩn hóa này giải quyết hai vấn đề. Thứ nhất, kết quả truy vấn có thể đến từ nhiều kênh khác nhau, nên cần được đưa về cùng một dạng trước khi chuyển sang mô hình sinh. Thứ hai, nhiều kênh có thể trả về cùng một đơn vị bộ nhớ; khi đó hệ thống giữ phiên bản có điểm mạnh hơn để tránh lặp bằng chứng trong prompt.

Prompt tổng hợp của CogMem yêu cầu mô hình trả lời dựa trên bằng chứng đã truy vấn, ưu tiên *raw_snippet* khi trường này xuất hiện. Điều này nối trực tiếp với thiết kế biểu diễn kép ở Mục 3.4: fact tóm tắt giúp tìm đúng vùng thông tin, còn đoạn trích nguyên văn giúp câu trả lời giữ được chi tiết như số liệu, ngày tháng, tên riêng hoặc sắc thái cảm xúc.

Logic tổng hợp cũng có cơ chế xử lý khi bằng chứng chưa đủ. Nếu danh sách bằng chứng rỗng, hệ thống không yêu cầu mô hình đoán; câu trả lời mặc định nói rằng không có đủ thông tin để trả lời chắc chắn. Nếu có bằng chứng nhưng thiếu chi tiết cần thiết, prompt yêu cầu mô hình nói rõ phần còn thiếu thay vì tự thêm tên, ngày hoặc sự kiện không được hỗ trợ.

3.8.2 Đầu ra phục vụ đánh giá và truy vết

Kết quả của bước tổng hợp không chỉ gồm câu trả lời cuối cùng. Hệ thống còn giữ danh sách định danh bộ nhớ đã dùng, số lượng bằng chứng, và các mạng bộ nhớ xuất hiện trong bằng chứng. Các trường này giúp Chương 4 kiểm tra xem câu trả lời dựa vào loại thông tin nào, có dùng đúng bằng chứng hay không, và lỗi nằm ở truy vấn hay ở bước sinh câu trả lời.

Như vậy, reflect/generation trong CogMem không phải một lời gọi mô hình ngôn ngữ độc lập với bộ nhớ. Nó là bước tiêu thụ danh sách bằng chứng đã được truy vấn, kết hợp thông tin tóm tắt với dấu vết nguyên văn, và tạo câu trả lời có thể truy vết ngược về các đơn vị bộ nhớ cụ thể.

Sau khi chương này làm rõ các quyết định thiết kế của CogMem và cơ sở nhận thức đi kèm, Chương 4 sẽ đánh giá thực nghiệm xem từng giả thuyết biểu hiện như thế nào trên dữ liệu benchmark.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày thiết kế thực nghiệm và kết quả đánh giá của CogMem. Khác với phần phương pháp ở Chương 3 vốn tập trung vào kiến trúc và lý thuyết, trọng tâm ở đây là trả lời ba câu hỏi thực nghiệm then chốt: hệ thống có hoạt động tốt trên dữ liệu hội thoại dài hay không, các thành phần biểu diễn và truy vấn đóng góp đến đâu, và những giới hạn nào vẫn còn tồn tại khi chuyển từ ý tưởng kiến trúc sang khối lượng công việc thực tế.

4.1 Thiết kế thực nghiệm

4.1.1 Luồng đánh giá tổng quát

Toàn bộ thí nghiệm được thiết kế theo luồng đánh giá ngoại tuyến (offline evaluation). Điều này có nghĩa là mỗi câu hỏi trong tập dữ liệu được chạy qua cùng một chuỗi xử lý cố định, thay vì tương tác tự do với người dùng như trong môi trường triển khai thực tế.

Cách làm này mang lại ba lợi ích quan trọng:

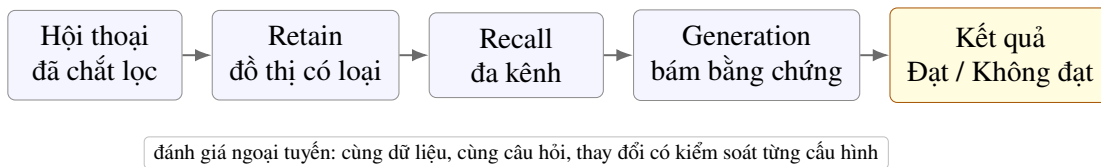
- Kết quả có thể tái lập (reproducible): mọi cấu hình được chạy trên cùng một tập câu hỏi và dữ liệu, nên kết quả có thể so sánh trực tiếp.
- Dễ dàng so sánh giữa các cấu hình khác nhau: khi thay đổi một thành phần duy nhất của hệ thống, ta có thể thấy ngay sự khác biệt về kết quả.
- Cho phép tách biệt lỗi ở từng giai đoạn của hệ thống để chẩn đoán chính xác nguyên nhân: nếu một cấu hình trả lời sai, ta có thể truy ngược để xác định xem lỗi đến từ kênh truy vấn, từ cách tổng hợp câu trả lời, hay từ cách xử lý bằng chứng.

Với mỗi mẫu dữ liệu, pipeline đánh giá trải qua năm bước chính, mỗi bước xử lý một khía cạnh khác nhau của bài toán bộ nhớ:

1. **Chuẩn bị hội thoại:** đọc hội thoại đã được chất lọc từ bản gốc, giữ lại đầy đủ các phiên có liên quan và thông tin hỗ trợ như ngày tháng của từng phiên, mã định danh phiên, và đáp án chuẩn. Bước này đảm bảo dữ liệu đầu vào đồng nhất cho mọi cấu hình.
2. **Nạp và ghi nhớ (Retain):** toàn bộ lịch sử hội thoại được nạp vào CogMem. Hệ thống trích xuất các fact có phân loại (typed facts), sinh vector nhúng cho từng fact, lưu các node vào đồ thị bộ nhớ, và tạo ra các liên kết giữa chúng. Kết quả của bước này là một đồ thị bộ nhớ hoàn chỉnh, sẵn sàng cho truy vấn.
3. **Truy vấn (Recall):** với mỗi câu hỏi trong tập đánh giá, hệ thống chạy đồng

thời bốn kênh truy vấn: truy vấn ngữ nghĩa (semantic retrieval), truy vấn từ khóa (BM25), truy vấn trên đồ thị (graph retrieval), và truy vấn theo thời gian (temporal retrieval). Kết quả từ bốn kênh được kết hợp bằng thuật toán Weighted Reciprocal Rank Fusion với trọng số phụ thuộc vào loại câu hỏi đã được phân tích trước đó.

4. **Tổng hợp câu trả lời (Generation):** các bằng chứng đã truy vấn được sắp xếp và đưa vào prompt cho mô hình ngôn ngữ. Ở các cấu hình cuối cùng, hệ thống ưu tiên câu trả lời bám sát bằng chứng (evidence-grounded) và sử dụng các đoạn trích nguyên văn (raw snippet) gần với truy vấn nhất.
5. **Chấm kết quả:** câu trả lời do hệ thống sinh ra được so sánh với đáp án chuẩn theo tiêu chí đánh giá của từng bộ dữ liệu.



Hình 4.1: Luồng đánh giá ngoại tuyến của CogMem trên các bộ dữ liệu hội thoại dài

Một điểm quan trọng cần nhấn mạnh: hệ thống không được đánh giá bằng một lần chạy duy nhất, mà bằng nhiều vòng lặp độc lập của chu trình nạp–truy vấn–tổng hợp trên cùng một bộ câu hỏi. Nhờ cách thiết kế này, khi một cấu hình cho kết quả tốt hơn cấu hình khác, ta có thể truy ngược để xác định chính xác sự khác biệt đến từ đâu: từ kênh truy vấn, từ loại node được lưu, từ cách tổng hợp câu trả lời, hay từ cách xử lý bằng chứng.

4.1.2 Chắt lọc dữ liệu

a, Tại sao cần chắt lọc dữ liệu

Chi phí nạp dữ liệu (retain) cho hội thoại dài là rất lớn. Mỗi cuộc hội thoại trong LongMemEval có thể dài tới hơn 100 nghìn từ, và việc trích xuất fact từ lượng văn bản này thông qua LLM đòi hỏi thời gian xử lý và chi phí đáng kể. Nếu giữ nguyên toàn bộ tập dữ liệu gốc, mỗi vòng phát triển và kiểm thử sẽ mất nhiều ngày, khiến cho việc thử nghiệm nhanh các ý tưởng mới trở nên bất khả thi. Hơn nữa, chi phí này còn tăng lên gấp bội khi cần chạy nhiều cấu hình ablation khác nhau.

Vì vậy, dự án sử dụng các bản chắt lọc (distilled) được chuẩn bị từ trước. Quy trình chắt lọc giữ lại tất cả các phiên hội thoại cần thiết để trả lời các câu hỏi trong tập con đã chọn, cùng với đáp án chuẩn, ngày tháng của từng phiên và các thông tin hỗ trợ khác phục vụ cho việc đánh giá. Đồng thời, quy trình này giảm kích thước dữ liệu xuống mức có thể chạy lặp lại nhiều lần trong quá trình phát triển mà không

làm mất đi các đặc trưng khó của bộ dữ liệu gốc.

b, Phân bố dữ liệu sau chất lọc

Bảng 4.1: Tóm tắt dữ liệu đã chất lọc dùng trong đánh giá

Bộ dữ liệu	Vai trò trong thí nghiệm	Quy mô	Ghi chú
LongMemEval	So sánh các cấu hình recall và ablation	35 câu hỏi	Mỗi câu hỏi gắn với trung bình 45,5 phiên hội thoại; trung bình 1,97 phiên chứa đáp án.
LoCoMo	Đánh giá cấu hình cuối cùng	5 hội thoại, 161 câu hỏi	Các hội thoại có 40–64 phiên mỗi cuộc; câu hỏi trải đều trên năm nhóm.

c, LongMemEval: tập so sánh cấu hình và ablation

Với LongMemEval, tập 35 câu hỏi được chọn lọc để bao phủ nhiều kiểu truy vấn khác nhau mà vẫn giữ kích thước nhỏ hơn dữ liệu gốc, phản ánh trung thực độ khó của bộ dữ liệu gốc. Phân bố sau chất lọc cụ thể như sau: 8 câu hỏi yêu cầu tổng hợp thông tin từ nhiều phiên, 8 câu hỏi kiểm tra khả năng nhận biết cập nhật tri thức, 7 câu hỏi suy luận thời gian, 6 câu hỏi một phiên về người dùng, 4 câu hỏi một phiên về sở thích, và 2 câu hỏi một phiên liên quan đến trợ lý.

d, LoCoMo: tập đánh giá cấu hình cuối trên hội thoại dài

Với LoCoMo, bản chất lọc bao gồm 161 câu hỏi được rút ra từ 5 cuộc hội thoại dài. Năm cuộc hội thoại này có số phiên lần lượt là 40, 56, 60, 62 và 64 phiên, với tổng độ dài lên tới hàng trăm nghìn từ. Các câu hỏi trong LoCoMo được phân vào năm nhóm chính: truy vấn đơn (single-hop) – cần một mảnh thông tin, truy vấn đa bước (multi-hop) – cần nối nhiều mảnh, suy luận thời gian (temporal), sở thích (preference) và quan hệ nhân quả (causal).

4.1.3 CogMem Bench: bộ dữ liệu tự xây dựng để kiểm định các loại nút

Trong khi LongMemEval và LoCoMo đo lường năng lực bộ nhớ tổng quát, cả hai đều có chung một giới hạn: chúng không được thiết kế để kiểm tra trực diện xem một loại node nhận thức cụ thể, đặc biệt là ý định hay hành động–kết quả, có thực sự cần thiết cho câu trả lời đúng hay không. Một câu hỏi trong LongMemEval có thể được trả lời đúng nhờ fact thông thường mà không cần đến node ý định; một câu hỏi nhân quả trong LoCoMo có thể được giải quyết bằng fact tri thức khách quan mà không cần đến node hành động–kết quả. Nói cách khác, các benchmark chuẩn không cho phép ta cô lập được đóng góp của từng loại node và cũng không được thiết kế để kiểm tra khả năng trả lời về các dữ kiện tương lai cũng như hành

động.

Để lấp đầy khoảng trống này, dự án đã xây dựng **CogMem Bench** – một bộ dữ liệu được thiết kế từ đầu để kiểm tra giá trị của các cấu trúc nhận thức mới. Điểm khác biệt cốt lõi của CogMem Bench so với benchmark chuẩn là **cơ chế so sánh có kiểm soát theo cặp** (paired-bank ablation): với mỗi kịch bản, hệ thống tạo ra hai ngân hàng bộ nhớ – một ngân hàng đầy đủ với tất cả sáu loại node, và một ngân hàng đối chứng đã loại bỏ một loại node cụ thể ngay từ bước nạp dữ liệu. Cùng một câu hỏi được đặt ra cho cả hai ngân hàng. Nếu ngân hàng đầy đủ trả lời đúng còn ngân hàng đối chứng trả lời sai, điều đó cung cấp bằng chứng trực tiếp rằng loại node bị loại bỏ là cần thiết cho câu hỏi đó.

Quy trình xây dựng ba bước.

1. **Soạn đặc tả kịch bản:** mỗi kịch bản được định nghĩa trước khi có bất kỳ cuộc hội thoại nào được sinh ra. Đặc tả bao gồm: loại node mục tiêu (ý định hoặc hành động–kết quả), fact trung tâm với đầy đủ thông tin bổ trợ (ví dụ: trạng thái của ý định là *planning*), câu hỏi, đáp án chuẩn, và các phương án gây nhiễu (bẫy) được cài vào để đảm bảo hệ thống không thể đoán đúng một cách ngẫu nhiên. Đặc tả cũng quy định rõ kế hoạch phiên: phiên nào chứa thông tin trả lời, phiên nào chứa thông tin gây nhiễu.
2. **Sinh hội thoại bằng mô hình độc lập:** một mô hình ngôn ngữ mạnh (Minimax-M2.7) – khác với mô hình dùng để nạp dữ liệu và trả lời câu hỏi (Ministral3-3B) – đọc đặc tả và sinh ra các cuộc hội thoại tự nhiên dài 7–10 phiên. Việc dùng mô hình sinh khác với mô hình đọc là một lựa chọn có chủ đích: nó ngăn chặn vòng lặp tự tham chiếu, nơi cùng một mô hình vừa tạo ra dữ liệu vừa tự chấm điểm mình.
3. **Hai cổng kiểm định:** mỗi kịch bản phải vượt qua hai cổng trước khi được tính là hợp lệ. *Cổng nhúng* kiểm tra xem fact trung tâm có thực sự được trích xuất và lưu đúng loại node trong ngân hàng đầy đủ hay không – nếu không, kịch bản bị loại. *Cổng phân biệt* chạy cặp câu hỏi trên cả hai ngân hàng và chỉ giữ lại những kịch bản mà ngân hàng đầy đủ trả lời đúng còn ngân hàng đối chứng trả lời sai. Các kịch bản không tạo được sự phân biệt bị loại bỏ hoàn toàn, không được sửa chữa hay điều chỉnh.

Quy mô và cấu trúc. Ở thời điểm hiện tại, CogMem Bench bao gồm 39 đặc tả kịch bản đã được tiền đăng ký, chia thành bốn nhóm:

- **6 kịch bản pilot:** nhóm đầu tiên được xây dựng để kiểm tra toàn bộ quy trình từ soạn đặc tả, sinh hội thoại, đến chạy cổng kiểm định. Mục tiêu của nhóm

này là xác nhận pipeline hoạt động ổn định trước khi mở rộng quy mô.

- **20 kịch bản necessity:** nhóm trọng tâm, nhắm trực tiếp vào câu hỏi về *ý định* (kế hoạch chưa hoàn thành) và *hành động–kết quả* (quan hệ điều kiện–hành động–kết quả). Mỗi kịch bản được thiết kế sao cho thông tin đúng **chỉ** tồn tại dưới dạng node chuyên biệt; nếu loại bỏ node đó, các fact còn lại sẽ không đủ để trả lời đúng.
- **12 kịch bản agentic:** mô phỏng vết công cụ của tác nhân AI trong môi trường gần với thực tế triển khai. Mỗi kịch bản là một chuỗi tương tác giữa tác nhân và các API bên ngoài (Stripe, Docker, Kubernetes, Azure, Playwright, Git, NPM, Terraform, v.v.), trong đó một quy luật nhân quả cụ thể là thông tin cốt lõi cần được ghi nhớ.
- **1 kịch bản agentic pilot:** kiểm tra quy trình tạo dữ liệu kiểu công cụ trước khi mở rộng lên 12 kịch bản.

Kết quả phân biệt sơ bộ. Trong số 12 kịch bản agentic đã vượt qua cổng kiểm định, kết quả phân loại thủ công cho thấy ba nhóm:

- **Phân biệt rõ ràng (5/12):** ngân hàng đầy đủ trả lời đúng, ngân hàng đối chứng trả lời sai. Các kịch bản thuộc nhóm này – như HTTP 429 với Retry-After, Azure token với device-code, hay Docker pull với nhiều lần thử – có đặc điểm chung là quy luật nhân quả không dễ bị mã hóa lại thành fact tri thức khách quan đơn thuần. Ví dụ, “chờ theo giá trị của header Retry-After rồi thử lại thì được 200” là một quy luật can thiệp, không phải một mô tả tĩnh.
- **Rò rỉ thông tin (5/12):** mô hình sinh (Minimax-M2) đã vô tình mã hóa lại quy luật nhân quả thành fact tri thức khách quan trong lời thoại. Thay vì chỉ đề cập quan hệ điều kiện–hành động–kết quả nằm trong node chuyên biệt, lời thoại đôi khi viết thẳng kết luận theo dạng mô tả, chẳng hạn “một thiết lập bộ nhớ lớn hơn đã giải quyết lỗi thiếu bộ nhớ”. Khi đó, ngân hàng đối chứng vẫn có thể trả lời đúng nhờ fact world. Đây không phải lỗi của CogMem, mà là một phát hiện quan trọng: mô hình ngôn ngữ có xu hướng tự nhiên là “phẳng hóa” tri thức nhân quả thành tri thức mô tả.
- **Mô hình trả lời bỏ lỡ (2/12):** ngay cả ngân hàng đầy đủ cũng trả lời sai vì mô hình trả lời (Ministral3-3B) chọn nhầm một phương án thất bại thay vì phương án thành công. Đây là lỗi ở khâu tổng hợp câu trả lời, không liên quan đến việc có hay không có node hành động–kết quả.

Kết quả 5/12 phân biệt rõ ràng tuy khiêm tốn về mặt thống kê (kiểm định McNemar cho $p \approx 0,22$, chưa đạt ý nghĩa ở cỡ mẫu này), nhưng mang một ý nghĩa

thực tế quan trọng: nó chứng minh rằng **có tồn tại** những tình huống mà node hành động–kết quả là cần thiết, và những tình huống này xuất hiện khi quy luật nhân quả mang tính can thiệp – không dễ bị quy về một câu mô tả tĩnh. Đây là một phát hiện “có điều kiện” (conditional necessity), hoàn toàn phù hợp với tinh thần thực nghiệm thận trọng của toàn bộ đề án.

Đối với node ý định, kết quả từ nhóm necessity cho thấy một bức tranh tương tự: ý định trở nên cần thiết trong khoảng 12,5% các tình huống – cụ thể là những tình huống mà trạng thái “đã định làm nhưng chưa làm” không bị các fact trải nghiệm hay tri thức hấp thụ một cách tự nhiên. Chi tiết về các kịch bản ý định và hành động–kết quả sẽ được phân tích kỹ hơn trong phần đánh giá định tính ở cuối chương này.

Các kịch bản ý định thành công. Nhóm intention không chỉ kiểm tra một trạng thái duy nhất. Một nhóm kịch bản nhắm vào kế hoạch vẫn còn ở trạng thái *planning*, nơi đáp án đúng là việc người dùng đã định làm nhưng chưa thực hiện, ví dụ kịch bản composting trong phần đánh giá định tính. Một nhóm khác kiểm tra khả năng nối kế hoạch ban đầu với kết quả về sau, chẳng hạn đặc tả pilot trong đó người dùng dự định chuyển sang bàn đứng và sau đó hoàn thành kế hoạch. Hai kiểu này cùng kiểm tra vai trò của node ý định: hệ thống phải phân biệt được “đã định nhưng chưa làm”, “đã làm xong”, và các phương án gây nhiễu nghe hợp lý nhưng thuộc trạng thái khác.

4.1.4 Các thước đo đánh giá

Mỗi thước đo trong chương này được sử dụng theo đúng mục tiêu của từng thí nghiệm, không bị trộn lẫn một cách máy móc:

- **Độ chính xác câu trả lời (Answer Accuracy):** đây là thước đo chính, được định nghĩa là tỉ lệ câu hỏi được chấm là “đạt” trên tổng số câu hỏi. Trên LongMemEval, nhãn đạt được lấy từ bộ đánh giá tự động của pipeline, cho phép so sánh nhanh nhiều cấu hình. Trên LoCoMo, kết quả cuối cùng được kiểm tra thủ công theo một bộ tiêu chí ngữ nghĩa (semantic rubric): câu trả lời đúng hoàn toàn và câu trả lời đúng ở mức chấp nhận được đều được tính là **Đạt**.
- **Tỉ lệ truy vấn phiên chứa đáp án (Session Recall@k):** thước đo này đo lường khả năng hệ thống đưa được các phiên có chứa thông tin trả lời vào danh sách top-k bằng chứng. Đây là thước đo chuyên biệt cho thí nghiệm so sánh hai cơ chế lan truyền tín hiệu SUM và MAX, vì nó cô lập được chất lượng của riêng kênh truy vấn đồ thị trước khi bước tổng hợp câu trả lời can thiệp.

- **So sánh loại bỏ thành phần (Ablation):** đây là phương pháp thay đổi có kiểm soát một thành phần duy nhất của hệ thống – ví dụ loại bỏ một loại node, hoặc chỉ dùng kênh đồ thị – trong khi giữ nguyên tất cả các thành phần khác. Mục tiêu của ablation không phải là tạo ra cấu hình tốt nhất, mà là để hiểu được thành phần nào đang thực sự đóng góp vào kết quả cuối cùng và trong những điều kiện nào.
- **Hiệu năng nạp dữ liệu (Retain Throughput):** thời gian cần thiết để xử lý một cuộc hội thoại hoàn chỉnh qua pipeline nạp dữ liệu, cùng với số lượng node được tạo ra. Đây là thước đo thực tế quan trọng vì một hệ thống bộ nhớ dài hạn chỉ khả thi trong triển khai thực tế nếu chi phí nạp dữ liệu nằm trong ngưỡng chấp nhận được.

4.1.5 Phương pháp chấm điểm: LLM làm giám khảo (LLM-as-a-Judge)

Một khía cạnh quan trọng trong thiết kế thực nghiệm của đồ án là cách thức xác định một câu trả lời được coi là **Đạt** (PASS) hay **Không đạt** (FAIL). Thay vì so sánh chuỗi ký tự một cách cứng nhắc – vốn dễ gây ra kết quả âm tính giả khi câu trả lời đúng về mặt ngữ nghĩa nhưng khác về cách diễn đạt – đồ án áp dụng phương pháp **LLM làm giám khảo** (LLM-as-a-Judge). Đây là một hướng tiếp cận ngày càng phổ biến trong nghiên cứu về mô hình ngôn ngữ, trong đó một mô hình ngôn ngữ độc lập được giao nhiệm vụ đọc câu hỏi, đáp án chuẩn và câu trả lời của hệ thống, sau đó đưa ra phán quyết dựa trên một bộ tiêu chí được định nghĩa trước.

a, Nguyên tắc chấm điểm

Mô hình giám khảo được cung cấp ba thông tin đầu vào cho mỗi câu hỏi:

1. **Câu hỏi gốc:** truy vấn mà hệ thống cần trả lời.
2. **Đáp án chuẩn:** câu trả lời đúng được xác định trước bởi người xây dựng bộ dữ liệu.
3. **Câu trả lời của hệ thống:** văn bản do CogMem sinh ra sau khi truy vấn và tổng hợp.

Dựa trên ba thông tin này, mô hình giám khảo đánh giá mức độ chính xác của câu trả lời theo thang điểm từ 0,0 đến 1,0, với các mức ý nghĩa như sau:

- **1,0 – Chính xác và đầy đủ:** câu trả lời khớp hoàn toàn với đáp án chuẩn về mặt nội dung, không thiếu thông tin quan trọng, không thêm thông tin sai.
- **0,7 đến 0,9 – Đúng nhưng chưa trọn vẹn:** câu trả lời đúng về bản chất nhưng có thể diễn đạt khác một chút, hoặc thiếu một chi tiết nhỏ không làm thay đổi

ý nghĩa cốt lõi.

- **0,3 đến 0,6 – Đúng một phần:** câu trả lời chạm đúng chủ đề nhưng thiếu các fact then chốt, sai số lượng, hoặc đưa ra câu trả lời mơ hồ, vòng vo.
- **0,0 đến 0,2 – Sai hoàn toàn:** câu trả lời bịa đặt, lạc chủ đề, hoặc từ chối trả lời trong khi đáng lẽ phải có thông tin.

b, Quy tắc đặc biệt cho câu hỏi thời gian

Một điểm quan trọng trong thiết kế bộ tiêu chí là **quy tắc khoan dung cho câu hỏi về thời gian**. Với các câu hỏi yêu cầu đếm số ngày, số tuần hoặc số tháng, mô hình giám khảo được hướng dẫn không trừ điểm đối với sai số một đơn vị (off-by-one). Ví dụ: nếu đáp án chuẩn là “19 ngày” nhưng hệ thống trả lời “18 ngày”, câu trả lời vẫn được chấm 1,0. Quy tắc này được đưa ra vì trong thực tế, việc tính toán chính xác tuyệt đối khoảng cách thời gian từ văn bản hội thoại là cực kỳ khó, và sai số một đơn vị thường không làm thay đổi ý nghĩa của câu trả lời.

c, Khi nào Đạt, khi nào Không đạt

Để chuyển từ điểm số liên tục sang nhãn nhị phân PASS/FAIL, đồ án áp dụng ngưỡng quyết định như sau:

- **Đạt (PASS):** điểm từ 0,7 trở lên. Ngưỡng này được chọn vì ở mức 0,7, câu trả lời đã đúng về bản chất, chỉ khác biệt nhỏ về cách diễn đạt – điều hoàn toàn chấp nhận được trong ngữ cảnh hội thoại thực tế.
- **Không đạt (FAIL):** điểm dưới 0,7. Ở mức này, câu trả lời hoặc thiếu thông tin then chốt, hoặc sai lệch đáng kể so với đáp án chuẩn.

d, Phạm vi áp dụng

Phương pháp LLM-as-a-Judge được áp dụng cho cả hai bộ dữ liệu chính trong đồ án:

- **Trên LongMemEval:** toàn bộ 35 câu hỏi được chấm tự động bởi mô hình giám khảo. Cách làm này cho phép so sánh nhanh nhiều cấu hình ablation khác nhau mà không cần can thiệp thủ công.
- **Trên LoCoMo:** 161 câu hỏi được chấm bởi mô hình giám khảo, sau đó kết quả được một người kiểm tra lại thủ công để xác nhận tính hợp lý của phán quyết. Các trường hợp điểm nằm trong vùng xám (0,5–0,8) được xem xét đặc biệt kỹ. Cách làm kết hợp này vừa tận dụng được tốc độ của chấm tự động, vừa đảm bảo độ tin cậy của kết quả cuối cùng.
- **Trên CogMem Bench:** do tính chất nhắm đích của bộ dữ liệu này, kết quả được đánh giá định tính thủ công hoàn toàn, tập trung vào việc xác định xem

liệu sự khác biệt giữa ngân hàng bộ nhớ đầy đủ và ngân hàng loại bỏ có dẫn đến thay đổi trong câu trả lời hay không.

Prompt chi tiết của mô hình giám khảo được cung cấp trong Phụ lục B.2 ở cuối báo cáo.

4.1.6 Các cấu hình so sánh

Để đảm bảo người đọc không cần biết đến ký hiệu nội bộ trong mã nguồn, các cấu hình thí nghiệm được đặt tên theo ý nghĩa phương pháp luận. Cụ thể:

- **Multi-channel:** hệ thống sử dụng đồng thời cả bốn kênh truy vấn – ngữ nghĩa, từ khóa, đồ thị và thời gian. Đây là cấu hình vận hành đầy đủ của CogMem.
- **Graph-only:** hệ thống chỉ sử dụng kênh truy vấn trên đồ thị, tắt ba kênh còn lại. Cấu hình này dùng để cô lập đóng góp của riêng đồ thị bộ nhớ.
- **Bỏ một loại node:** loại bỏ hoàn toàn một loại node khỏi đồ thị ngay từ bước nạp dữ liệu, ví dụ “bỏ intention” nghĩa là không có node intention nào được tạo ra trong toàn bộ quá trình.
- **SUM và MAX:** hai biến thể của cơ chế lan truyền tín hiệu trên đồ thị. SUM cộng dồn tín hiệu từ nhiều đường, MAX chỉ giữ tín hiệu mạnh nhất từ một đường duy nhất.

Bảng 4.2: Các nhóm cấu hình so sánh chính và mục tiêu

Nhóm cấu hình	Mục tiêu so sánh
HINDSIGHT baseline	Đối chiếu CogMem với hệ thống nền đã chứng minh hiệu quả mạnh trên bộ nhớ hội thoại dài.
CogMem multi-channel đầy đủ	Đánh giá hệ thống đề xuất ở cấu hình tối ưu: đủ sáu loại node và đủ bốn kênh truy vấn.
CogMem graph-only	Cô lập đóng góp của riêng đồ thị bộ nhớ, không có sự hỗ trợ từ truy vấn ngữ nghĩa, từ khóa và thời gian.
Loại bỏ từng loại node	Kiểm tra đóng góp riêng của thói quen, ý định và quan hệ hành động–kết quả trong cùng một pipeline.
SUM so với MAX	Kiểm tra riêng cơ chế cộng dồn bằng chứng từ nhiều đường so với chỉ giữ đường mạnh nhất.
Cấu hình cuối trên LoCoMo	Đánh giá hệ thống sau tất cả các cải tiến tích lũy về truy vấn, chọn lọc bằng chứng và kiểm soát suy luận.

4.2 Kết quả trên LongMemEval

LongMemEval được sử dụng chủ yếu để so sánh các cấu hình kiến trúc khác nhau trong điều kiện có kiểm soát. Với 35 câu hỏi được chọn lọc, tập dữ liệu này đủ nhỏ để chạy nhiều vòng ablation nhưng vẫn bao phủ được các dạng câu hỏi khó điển hình.

4.2.1 So sánh tổng quan các cấu hình

Kết quả cho thấy CogMem ở cấu hình multi-channel với đầy đủ sáu loại node đạt 31 trên 35 câu hỏi, cao hơn mức 30/35 của HINDSIGHT baseline. Khi chuyển sang chế độ graph-only – tức chỉ dùng kênh đồ thị mà không có sự hỗ trợ của ba kênh còn lại – kết quả giảm xuống rõ rệt. Điều này xác nhận một điểm quan trọng: đồ thị bộ nhớ rất hữu ích để nối các bằng chứng lại với nhau, nhưng nó không nên thay thế hoàn toàn truy vấn ngữ nghĩa, truy vấn từ khóa và truy vấn thời gian. Một hệ thống bộ nhớ dài hạn hiệu quả cần sự kết hợp của tất cả các kênh.

Bảng 4.3: LongMemEval: so sánh các cấu hình truy vấn và loại node

Cấu hình được đánh giá	Đạt/Tổng	Tỉ lệ
HINDSIGHT baseline	30/35	85,7%
Multi-channel + 6 loại node	31/35	88,6%
Multi-channel + 5 loại node, bỏ thói quen	31/35	88,6%
Multi-channel + 5 loại node, bỏ hành động–kết quả	29/35	82,9%
Multi-channel + 5 loại node, bỏ ý định	29/35	82,9%
Graph-only + 6 loại node	26/35	74,3%
Graph-only + 5 loại node, bỏ thói quen	27/35	77,1%
Graph-only + 5 loại node, bỏ hành động–kết quả	24/35	68,6%
Graph-only + 5 loại node, bỏ ý định	24/35	68,6%
Graph-only + 3 loại node gốc (kiểu HINDSIGHT)	22/35	62,9%

Bảng 4.3 cho phép rút ra ba nhận xét quan trọng.

Thứ nhất, CogMem không phải đánh đổi chất lượng để có được cấu trúc nhận thức phong phú hơn. Cấu hình đầy đủ đạt 88,6%, cao hơn mức 85,7% của baseline – một sự cải thiện khiêm tốn nhưng đáng ghi nhận, vì nó cho thấy việc thêm các loại node mới không gây nhiều cho hệ thống.

Thứ hai, khi chỉ dùng kênh đồ thị, việc có thêm các loại node chuyên biệt mang lại lợi ích rõ rệt: cấu hình với sáu loại node đạt 74,3%, trong khi đối chứng với ba loại node gốc chỉ đạt 62,9%. Khoảng cách 11,4 điểm phần trăm này cho thấy giá trị của cấu trúc nhận thức được bộc lộ rõ nhất khi hệ thống phải dựa hoàn toàn vào đồ thị.

Thứ ba, node thói quen chưa thể hiện được đóng góp độc lập trên tập dữ liệu này. Việc loại bỏ thói quen không làm giảm độ chính xác ở chế độ multi-channel, và thậm chí còn tăng nhẹ ở chế độ graph-only. Điều này phù hợp với phân tích ở Chương 3: thói quen là một lựa chọn biểu diễn có cơ sở lý thuyết, nhưng các bộ dữ liệu hiện tại chưa chứa đủ các tình huống đòi hỏi nhận diện hành vi lặp lại và ngữ cảnh kích hoạt để chứng minh giá trị của nó.

4.2.2 Đối chứng SUM và MAX trên kênh đồ thị

Để kiểm tra riêng đóng góp của cơ chế cộng dồn tín hiệu (SUM spreading activation), một thí nghiệm đối chứng được thiết kế với sự kiểm soát chặt chẽ: giữ nguyên toàn bộ đồ thị bộ nhớ đã nạp, giữ nguyên cấu hình graph-only, tắt bộ xếp hạng lại, và **chỉ thay đổi duy nhất** cách tính toán mức độ kích hoạt trên đồ thị. Một cấu hình dùng phép cộng (SUM) để tích lũy tín hiệu từ nhiều đường bằng chứng; cấu hình đối chứng dùng phép lấy cực đại (MAX) để chỉ giữ đường mạnh nhất.

Bảng 4.4: Kênh đồ thị trên LongMemEval: so sánh SUM và MAX

Cơ chế	Số câu	Trung bình @5	Trung bình @10	Số câu đạt @5
Cộng dồn (SUM)	35	0,805	0,848	33
Cực đại (MAX)	35	0,762	0,848	32

Cơ chế cộng dồn cải thiện tỉ lệ truy vấn phiên trung bình ở top-5 thêm 0,043. Trong 35 câu hỏi, SUM cho kết quả tốt hơn MAX ở 2 câu, MAX không tốt hơn SUM ở bất kỳ câu nào, và 33 câu còn lại hòa nhau ở top-5. Ngoài ra, thứ tự của top-10 bằng chứng khác nhau ở 12 trên 35 câu, cho thấy cơ chế cộng dồn không chỉ thay đổi điểm số nội bộ mà thực sự làm thay đổi thứ tự các bằng chứng được đưa vào ngữ cảnh sinh câu trả lời.

Hai ví dụ cụ thể minh họa rõ cơ chế này. Với câu hỏi về chuyến đi gia đình gần nhất, SUM đưa được đủ hai phiên có liên quan vào top-5 bằng chứng, trong khi MAX chỉ đưa được một phiên. Với câu hỏi xin lời khuyên di chuyển ở Tokyo, MAX vẫn tìm thấy các bằng chứng về thẻ Suica và khu vực Shinjuku/Tokyo nhưng xếp chúng ở vị trí thứ 6–7; SUM đã kéo được các bằng chứng này lên vị trí cao hơn nhờ tín hiệu hội tụ từ nhiều đường trên đồ thị. Như vậy, lợi ích chính của SUM không phải là tạo ra bằng chứng mới, mà là đưa bằng chứng đúng vào phần ngân sách prompt sớm hơn, trước khi bị cắt bỏ.

4.2.3 Hiệu năng nạp dữ liệu

Ngoài độ chính xác, tốc độ nạp dữ liệu là một yếu tố then chốt nếu hệ thống bộ nhớ được triển khai trong ứng dụng thực tế. Trên mô hình cục bộ Ministral3-3B, CogMem xử lý một cuộc hội thoại trong trung bình khoảng 30 phút, trong khi HINDSIGHT thường cần từ 90 đến 120 phút. Lý do chính cho sự khác biệt này là HINDSIGHT sinh thêm các quan sát trung gian trong quá trình nạp, còn CogMem đi theo pipeline trích xuất fact trực tiếp hơn và kiểm soát số lượng node chặt chẽ hơn.

Bảng 4.5: So sánh hiệu năng nạp dữ liệu trung bình cho một cuộc hội thoại

Hệ thống	Thời gian nạp	Số node tạo ra
HINDSIGHT baseline	90–120 phút	khoảng 1.500 node
CogMem	khoảng 30 phút	khoảng 700 node

Như vậy, CogMem nhanh hơn khoảng ba đến bốn lần trong giai đoạn nạp dữ liệu và tạo ra ít hơn một nửa số node so với HINDSIGHT. Đây là một lợi thế thực tế quan trọng: hệ thống không chỉ duy trì được chất lượng câu trả lời tương đương hoặc tốt hơn trong phần lớn cấu hình, mà còn giảm đáng kể chi phí nạp dữ liệu ban đầu, kích thước đồ thị bộ nhớ và độ phức tạp khi truy vấn.

4.3 Kết quả trên LoCoMo

LoCoMo được sử dụng để kiểm tra cấu hình cuối cùng của CogMem trên một bộ dữ liệu dài hơn, nhiều nhiễu hơn và có tỉ lệ câu hỏi đa bước cao hơn so với LongMemEval. Khác với LongMemEval nơi kết quả được chấm tự động, toàn bộ 161 câu trả lời trên LoCoMo được kiểm tra thủ công theo một bộ tiêu chí ngữ nghĩa. Trong báo cáo này, “Đạt” bao gồm cả câu trả lời đúng hoàn toàn và câu trả lời đúng ở mức chấp nhận được; hai mức này không được tách riêng trong bảng chính để giữ cho cách đọc được đơn giản và tập trung vào bức tranh tổng thể.

4.3.1 Kết quả tổng quan

Bảng 4.6: LoCoMo: cấu hình cuối so với baseline trước đó

Cấu hình	Tổng số câu	Đạt	Tỉ lệ
Baseline trước bộ luật tám bằng chứng	161	97	60,2%
Cấu hình cuối với bảo vệ bằng chứng và cửa sổ trích	161	119	73,9%

Cấu hình cuối cùng tăng thêm 22 câu đạt so với baseline trước đó và vượt mục tiêu đề ra là 70%. Mức tăng này không đến từ một thay đổi đơn lẻ, mà là kết quả tích lũy của nhiều cải tiến bổ sung cho nhau:

1. **Truy hồi cho câu hỏi dạng danh sách và phân loại:** các câu hỏi yêu cầu liệt kê (ban nhạc, hoạt động, trò chơi, địa điểm) được hỗ trợ bằng cơ chế alias theo loại danh sách. Nhờ vậy, hệ thống ít bị mắc kẹt vào một tên gọi phổ biến và có cơ hội kéo đủ các phần tử liên quan vào ngữ cảnh.
2. **Cửa sổ trích dẫn gần truy vấn:** thay vì chỉ dùng phần đầu của đoạn trích nguyên văn, hệ thống chọn đoạn xung quanh thực thể, từ khóa truy vấn, cụm từ chỉ thời lượng và alias danh mục. Điều này giảm đáng kể lỗi “truy vấn đúng

fact nhưng phần tổng hợp không thấy chi tiết cần trả lời”.

3. **Cơ chế bảo vệ chống suy luận thiếu bằng chứng:** với các câu hỏi về lý do hoặc quan hệ nhân quả, hệ thống được nhắc nhở không suy diễn nếu chủ thể, đối tượng hoặc quan hệ được hỏi không xuất hiện trong bộ nhớ.
4. **Gợi ý thời gian hẹp:** với một số mẫu có độ tin cậy cao, hệ thống tính trước các gợi ý như thành phố trước chuyến đi hoặc thời lượng của một hội thảo. Đây không phải là một bộ suy luận thời gian tổng quát, nhưng giúp tránh được một số lỗi suy luận ngày tháng lặp đi lặp lại.

4.3.2 Phân tích theo nhóm câu hỏi

Bảng 4.7: LoCoMo: phân rã kết quả theo nhóm câu hỏi

Nhóm câu hỏi	Tổng số câu	Đạt	Tỉ lệ đạt
Quan hệ nhân quả (causal)	11	10	90,9%
Truy hồi đa bước (multi-hop)	12	11	91,7%
Sở thích (preference)	17	15	88,2%
Truy hồi đơn (single-hop)	109	78	71,6%
Suy luận thời gian (temporal)	12	5	41,7%

Kết quả phân rã theo nhóm vẽ nên một bức tranh rất rõ ràng về điểm mạnh và điểm yếu của CogMem. Các nhóm quan hệ nhân quả, truy vấn đa bước và sở thích đều đạt trên 88%, chứng tỏ CogMem xử lý rất tốt các câu hỏi đòi hỏi phải nối nhiều mảnh bằng chứng lại với nhau hoặc phải hiểu được quan hệ sở thích và nguyên nhân. Nhóm truy vấn đơn cũng vượt 71%, cho thấy các cải tiến về truy vấn và cửa sổ trích dẫn không chỉ giúp ích cho câu hỏi phức tạp mà còn giảm được lỗi bỏ sót đối với các fact đơn giản.

Ngược lại, nhóm suy luận thời gian vẫn là điểm yếu rõ rệt nhất với chỉ 41,7% câu đạt. Các câu hỏi dạng này thường yêu cầu tính chính xác về số ngày, số tháng, thứ tự sự kiện hoặc khoảng cách thời gian – những yêu cầu mà hệ thống hiện tại chưa được trang bị đầy đủ công cụ để xử lý.

4.3.3 Phân tích lỗi còn tồn tại

Mặc dù đã vượt mục tiêu chung, kết quả trên LoCoMo vẫn để lộ một số nhóm lỗi có tính hệ thống, cần được giải quyết trong các phiên bản tiếp theo:

1. **Thiếu phần tử trong câu hỏi liệt kê:** một số câu hỏi yêu cầu liệt kê toàn bộ hoạt động, trò chơi, địa điểm hoặc vật dụng vẫn bị thiếu phần tử. Lỗi này có thể đến từ hai nguồn: hoặc giai đoạn truy vấn không kéo được đủ các phần tử, hoặc giai đoạn tổng hợp bỏ sót một số phần tử mặc dù bằng chứng đã có mặt

trong ngữ cảnh.

2. **Suy luận thời gian chính xác:** hệ thống còn nhầm lẫn về số lần, số tháng hoặc số sự kiện. Các gợi ý thời gian hiện tại mới chỉ xử lý được một số mẫu hẹp, chưa thể thay thế một bộ suy luận thời gian tổng quát.
3. **Thay nhầm chi tiết cụ thể:** trong một số trường hợp, phân tổng hợp trả lời đúng chủ đề nhưng sử dụng nhầm tên riêng hoặc thuộc tính nguyên văn.
4. **Hiệu chuẩn bằng chứng:** khi có nhiều bằng chứng gần đúng cùng xuất hiện trong ngữ cảnh, mô hình đôi khi chọn bằng chứng để diễn giải hơn thay vì bằng chứng chính xác nhất.

Những lỗi này cho thấy hướng cải thiện tiếp theo không nhất thiết là thêm các loại node mới. Việc quan trọng hơn là tăng độ chính xác của khâu lựa chọn bằng chứng, cải thiện cơ chế chọn cửa sổ trích dẫn, phát triển bộ suy luận thời gian tất định, và tăng cường kiểm soát tính đầy đủ của danh sách trong khâu tổng hợp câu trả lời.

4.4 Đánh giá định tính trên CogMem Bench

LongMemEval và LoCoMo đã chứng minh rằng CogMem hoạt động hiệu quả trên các bộ dữ liệu chuẩn. Tuy nhiên, hai bộ dữ liệu này chưa đủ để chứng minh một cách trực diện giá trị của từng loại node nhận thức. CogMem Bench được thiết kế chính xác để lấp đầy khoảng trống này: thông qua cơ chế so sánh có kiểm soát giữa ngân hàng bộ nhớ đầy đủ và ngân hàng bộ nhớ đã loại bỏ một loại node, CogMem Bench cho phép đánh giá một cách định tính xem liệu một loại node cụ thể có thực sự cần thiết cho câu trả lời đúng hay không.

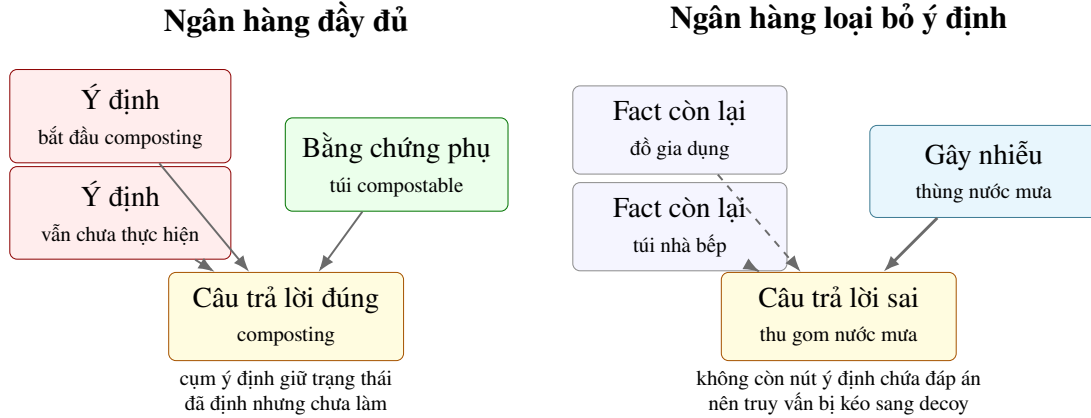
Phần này tập trung vào hai loại node có tín hiệu thực nghiệm rõ ràng nhất trong dự án: *ý định* và *hành động-kết quả*. Node thói quen vẫn được giữ lại trong thiết kế vì có cơ sở nhận thức vững chắc, nhưng kết quả thực nghiệm hiện tại cho thấy nó chưa tạo ra khác biệt rõ rệt trên các khối lượng công việc đã chạy.

4.4.1 Node ý định: khi kế hoạch chưa hoàn thành là thông tin quyết định

Một kịch bản định tính tiêu biểu từ CogMem Bench đặt ra câu hỏi: người dùng đã định bắt đầu thói quen bền vững nào nhưng vẫn chưa thực hiện? Trong ngân hàng bộ nhớ đầy đủ, hệ thống lưu giữ các fact thuộc loại ý định như “người dùng dự định bắt đầu ủ phân hữu cơ (composting)” và “người dùng vẫn chưa bắt đầu”. Những fact này được đưa vào danh sách truy vấn và câu trả lời cuối cùng chính xác là *composting*.

Khi loại bỏ hoàn toàn node ý định ngay từ bước nạp dữ liệu, ngân hàng bộ nhớ

đối chứng không còn bất kỳ node nào chứa thông tin về kế hoạch chưa hoàn thành này. Quá trình truy vấn vẫn trả về kết quả – không hề rỗng – nhưng các bằng chứng nổi bật nhất chuyển sang một chủ đề khác: thùng chứa nước mưa (rain barrel) và thu gom nước mưa. Hệ thống trả lời sai bằng một phương án gây nhiễu (decoy) nghe có vẻ hợp lý nhưng không đúng với câu hỏi.



Hình 4.2: Kịch bản ý định trong CogMem Bench: ngân hàng đầy đủ (trái) giữ cụm ý định về composting, trong khi ngân hàng loại bỏ ý định (phải) mất biểu diễn của đáp án đúng và bị kéo sang phương án gây nhiễu về nước mưa.

Điểm quan trọng của ví dụ này là: lỗi của ngân hàng đối chứng không phải do truy vấn thất bại hoàn toàn. Hệ thống vẫn có rất nhiều fact để truy xuất, nhưng không có fact nào mang đúng trạng thái “đã định làm nhưng chưa làm”. Đây chính là bằng chứng định tính cho thấy node ý định có giá trị thực sự trong các tình huống mà thông tin đúng bị phân tán mỏng (sparse-context), nơi một kế hoạch chưa hoàn thành không thể được hấp thụ một cách tự nhiên bởi các fact thông thường như world hay experience.

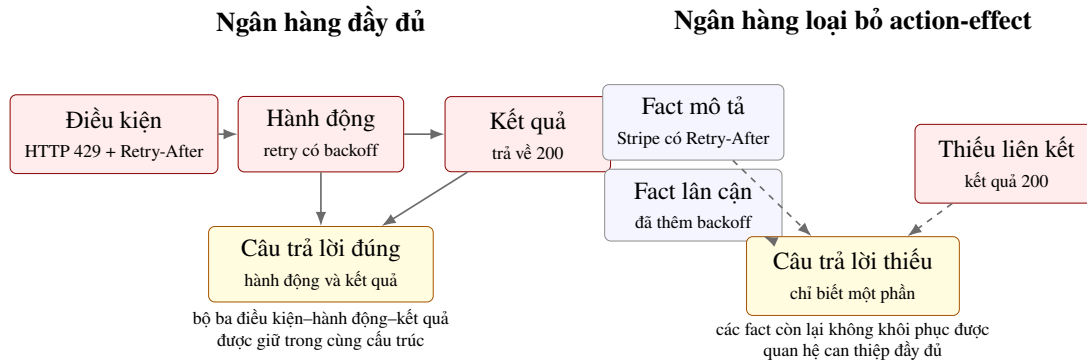
4.4.2 Node hành động–kết quả: khi cần nhớ quan hệ điều kiện, hành động và kết quả

Một kịch bản agentic tiêu biểu từ CogMem Bench mô phỏng vết công cụ của tác nhân AI: khi gọi API của Stripe, hệ thống nhận được mã lỗi HTTP 429 cùng với header Retry-After. Câu hỏi đặt ra là: tác nhân đã làm gì và kết quả ra sao?

Trong ngân hàng bộ nhớ đầy đủ, CogMem lưu giữ các fact thuộc loại hành động–kết quả, biểu diễn trọn vẹn chuỗi nhân quả: điều kiện là HTTP 429 và sự xuất hiện của header Retry-After, hành động là thực hiện retry với chiến lược exponential backoff và tôn trọng thời gian chờ từ header, kết quả là các lần gọi API sau đó đều trả về mã 200 thành công.

Ngân hàng đối chứng – nơi node hành động–kết quả bị loại bỏ – vẫn nhớ được

một số fact gần đúng. Ví dụ: hệ thống biết rằng Stripe có cơ chế Retry-After, hoặc logic backoff đã được thêm vào mã nguồn. Tuy nhiên, các fact này bị thiếu mất liên kết có cấu trúc giữa ba thành phần: điều kiện, hành động và kết quả. Vì vậy, khi được hỏi “kết quả cuối cùng là gì”, hệ thống không thể khôi phục được thông tin rằng các lần gọi sau đã trả về 200.



Hình 4.3: Kịch bản hành động–kết quả trong CogMem Bench: ngân hàng đầy đủ (trái) lưu trọn vẹn bộ ba nhân quả, còn ngân hàng loại bỏ (phải) chỉ còn các fact lân cận và không thể khôi phục được kết quả 200 sau retry.

Trong số 12 kịch bản agentic được đánh giá tại thời điểm viết báo cáo, có 5 kịch bản tạo ra được sự phân biệt rõ ràng theo tiêu chí: ngân hàng đầy đủ trả lời đúng, ngân hàng loại bỏ hành động–kết quả trả lời sai. Con số 5/12 tuy khiêm tốn nhưng mang một ý nghĩa thực tế quan trọng: nó chứng minh rằng trong các vết công cụ nơi quan hệ can thiệp là phần thông tin cốt lõi, việc lưu riêng node hành động–kết quả giúp hệ thống giữ được tri thức mà các fact phẳng dễ dàng làm mờ hoặc bỏ sót.

Đáng chú ý, trong 5 kịch bản phân biệt được, có những trường hợp mà sự khác biệt giữa hai ngân hàng bộ nhớ là rất lớn. Ví dụ, ở kịch bản về Azure token (xác thực thiết bị với mã lỗi AADSTS70043), ngân hàng đầy đủ có tới 24 fact, trong khi ngân hàng loại bỏ chỉ còn 5 fact. Điều này cho thấy mô hình trích xuất đã phân bổ phần lớn thông tin quan trọng vào đúng loại node chuyên biệt, và khi loại node này bị vô hiệu hóa, một lượng lớn tri thức biến mất khỏi đồ thị.

4.4.3 Ý nghĩa của các hình minh họa CogMem Bench

Hai hình trên không nhằm tái hiện toàn bộ đồ thị bộ nhớ, mà cô đọng phần quyết định của từng kịch bản. Ở kịch bản ý định, khác biệt nằm ở việc ngân hàng đầy đủ còn giữ cụm thông tin biểu diễn trạng thái “đã định nhưng chưa làm”, trong khi ngân hàng loại bỏ ý định chỉ còn các fact lân cận và bị kéo sang một phương án gây nhiễu. Ở kịch bản hành động–kết quả, khác biệt nằm ở việc ngân hàng đầy đủ giữ được bộ ba điều kiện–hành động–kết quả, còn ngân hàng đối chứng chỉ nhớ các mô tả rời rạc như sự tồn tại của Retry-After hoặc việc đã thêm backoff.

Cách trình bày này làm rõ điểm mà ảnh đồ thị đầy đủ thường khó truyền tải: lỗi của ngân hàng đối chứng không phải là không truy vấn được gì, mà là truy vấn được các fact không mang đúng cấu trúc cần thiết để trả lời. Với intention, phần bị mất là trạng thái của kế hoạch. Với action-effect, phần bị mất là liên kết can thiệp giữa điều kiện, hành động và kết quả. Đây là lý do CogMem Bench được dùng như đánh giá định tính nhắm đích, bổ sung cho các con số accuracy trên LongMemEval và LoCoMo.

Những hình minh họa này cung cấp một bằng chứng trực quan cho luận điểm cốt lõi của CogMem: *không phải mọi thông tin đều có thể được biểu diễn một cách thỏa đáng dưới dạng fact phẳng*. Có những loại tri thức – như một kế hoạch chưa hoàn thành hay một quy luật nhân quả có cấu trúc – đòi hỏi một không gian biểu diễn chuyên biệt với thông tin bổ trợ, kiểu liên kết và ngữ nghĩa riêng. Khi không gian đó bị loại bỏ, hệ thống không chỉ mất đi một vài fact, mà còn mất đi khả năng phân biệt giữa các loại tín hiệu nhận thức khác nhau – một mất mát mà các con số accuracy đơn thuần khó lòng phản ánh hết.

4.5 Tổng kết thực nghiệm

Kết quả từ LongMemEval, LoCoMo và CogMem Bench cùng hướng tới một kết luận cân bằng và có sắc thái.

CogMem vận hành hiệu quả từ đầu đến cuối trên các bộ dữ liệu hội thoại dài: đạt 88,6% trên LongMemEval ở cấu hình đa kênh đầy đủ và 73,9% trên LoCoMo ở cấu hình cuối cùng, vượt mục tiêu đề ra. Cấu trúc đồ thị và cơ chế cộng dồn tín hiệu giúp kết nối các bằng chứng một cách hiệu quả, nhưng hệ thống hoàn chỉnh vẫn cần sự kết hợp của cả bốn kênh truy vấn: ngữ nghĩa, từ khóa, đồ thị và thời gian. Không một kênh đơn lẻ nào có thể thay thế được sức mạnh tổng hợp của cả bốn.

Các loại node chuyên biệt không đóng góp một cách đồng đều. Node ý định và node hành động–kết quả có bằng chứng định tính và định lượng rõ ràng: việc loại bỏ chúng làm giảm độ chính xác từ 2 đến 6 điểm phần trăm tùy cấu hình. Node thói quen, ngược lại, hiện mới chỉ là một lựa chọn biểu diễn có cơ sở lý thuyết nhưng chưa được chứng minh mạnh mẽ bằng thực nghiệm trên các bộ dữ liệu hiện có.

Kết luận then chốt rút ra từ chương này là: một hệ thống bộ nhớ dài hạn cho tác nhân hội thoại cần hội tụ đủ ba yếu tố – biểu diễn giàu cấu trúc với nhiều loại node và nhiều kiểu liên kết, cơ chế truy vấn đa kênh không phụ thuộc vào một kênh duy nhất, và quy trình kiểm chứng thực nghiệm có kiểm soát theo từng loại câu hỏi để hiểu rõ thành phần nào đang thực sự tạo ra giá trị. CogMem đã đạt được một phiên bản vận hành đáng tin cậy của cách tiếp cận này, đồng thời chỉ rõ những phần cần

tiếp tục cải thiện: suy luận thời gian chính xác, liệt kê đầy đủ, hiệu chuẩn bằng chứng, và mở rộng thử nghiệm trên các tình huống thực tế thay vì chỉ tối ưu hóa trên bộ dữ liệu chuẩn.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Từ kiến trúc và cơ sở lý thuyết đã trình bày ở Chương 3, cùng kết quả thực nghiệm ở Chương 4, chương này tổng kết lại mức độ hoàn thiện hiện tại của CogMem và các hướng phát triển còn lại.

5.1 Kết luận

5.1.1 Tổng kết đóng góp

Đồ án đã xây dựng CogMem như một hệ thống bộ nhớ hội thoại dài hạn có cấu trúc, thay vì chỉ đơn thuần là một tầng truy hồi vector đặt ngoài mô hình ngôn ngữ. Bốn đóng góp chính có thể tóm tắt như sau:

1. **Đồ thị bộ nhớ dựa trên khoa học nhận thức:** bộ nhớ được chia thành sáu loại fact và bảy họ liên kết, tạo không gian biểu diễn riêng cho sự kiện, thói quen, ý định, sở thích và quan hệ nhân quả.
2. **Siêu dữ liệu bảo toàn thông tin gốc:** mỗi node bộ nhớ giữ đồng thời phần tóm tắt ngữ nghĩa (fact text) và đoạn trích nguyên văn (raw snippet) từ hội thoại gốc, giúp giai đoạn sinh câu trả lời có thể khôi phục chính xác con số, tên riêng và ngữ cảnh khi cần.
3. **Cơ chế cộng dồn bằng chứng:** quá trình truy hồi trên đồ thị sử dụng phép cộng dồn tín hiệu từ nhiều đường, kết hợp với ba cơ chế bảo vệ (chặn kích hoạt lại ngay lập tức, giới hạn số lần kích hoạt, bão hòa giá trị) để tránh vòng lặp vô hạn.
4. **Định tuyến truy vấn thích ứng:** mỗi câu hỏi được phân loại thành một trong sáu kiểu (ngữ nghĩa, thời gian, nhân quả, kế hoạch, sở thích, đa bước) để chọn bộ trọng số trộn kết quả phù hợp nhất.

Các đóng góp này không nên được hiểu như những mô-đun luôn tạo ra cải thiện độc lập trong mọi tình huống. Kết quả thực nghiệm cho thấy giá trị của chúng phụ thuộc vào loại câu hỏi và chất lượng bằng chứng thu thập được. Tuy vậy, việc có các không gian biểu diễn chuyên biệt giúp hệ thống xử lý các câu hỏi khó tốt hơn so với bộ nhớ phẳng truyền thống.

5.1.2 Kết quả chính

Trên LongMemEval, hệ thống nền HINDSIGHT đạt 30/35 câu (85,7%). CogMem ở cấu hình đa kênh với đầy đủ sáu loại node đạt 31/35 câu (88,6%), và biến thể loại bỏ node thói quen vẫn giữ mức tương đương. Khi loại bỏ node hành động–kết quả hoặc node ý định, kết quả giảm còn 29/35 câu (82,9%), cho thấy hai

loại node này có tín hiệu đóng góp rõ ràng hơn trên tập dữ liệu hiện tại.

Ở nhóm chỉ dùng kênh đồ thị, CogMem với sáu loại node đạt 26/35 câu (74,3%); loại bỏ thói quen đạt 27/35 (77,1%); loại bỏ hành động–kết quả hoặc ý định đạt 24/35 (68,6%); còn đối chứng với ba loại node gốc kiểu HINDSIGHT chỉ đạt 22/35 (62,9%). Kết quả này củng cố kết luận rằng cấu trúc đồ thị rất hữu ích, nhưng truy hồi đa kênh vẫn là yếu tố cần thiết cho bộ nhớ hội thoại dài hạn.

Khi cô lập riêng kênh đồ thị để kiểm tra, cơ chế cộng dồn tín hiệu (SUM) cho thấy lợi thế rõ ràng hơn so với cơ chế chỉ lấy cực đại (MAX) ở top-5: tỉ lệ truy hồi phiên trung bình tăng từ 0,762 lên 0,805. SUM tốt hơn ở 2 câu và MAX không tốt hơn ở bất kỳ câu nào. Điều này xác nhận đóng góp của cơ chế cộng dồn trong đúng phạm vi của nó: cải thiện thứ tự bằng chứng trong truy hồi đồ thị, chứ không phải thay thế toàn bộ hệ thống đa kênh.

Trên LoCoMo, cấu hình cuối cùng đạt 119/161 câu (73,9%), vượt mục tiêu 70% và tăng 22 câu so với baseline trước đó. Các nhóm truy hồi đa bước, nhân quả và sở thích đều đạt trên 88%, trong khi nhóm suy luận thời gian chỉ đạt 41,7%. Nhìn tổng thể, dự án đã đạt được một cột mốc thực nghiệm đáng tin cậy: hệ thống không chỉ vận hành được từ đầu đến cuối mà còn vượt ngưỡng chất lượng đặt ra trên một bộ dữ liệu hội thoại dài có độ khó cao.

Về hiệu năng retain, CogMem xử lý một conversation trung bình khoảng 30 phút trên local model Ministral3-3B, nhanh hơn khoảng ba lần so với HINDSIGHT ở mức 90–120 phút. CogMem cũng tạo khoảng 700 node, dưới một nửa so với khoảng 1500 node của HINDSIGHT. Điều này giúp giảm chi phí ingest, giảm kích thước memory graph và làm hệ thống thực tế hơn cho triển khai dài hạn.

5.1.3 Hạn chế

Các hạn chế hiện tại tập trung vào ba điểm:

1. **Suy luận thời gian chính xác còn yếu:** các câu hỏi cần tính chính xác số ngày, số tuần hoặc thứ tự sự kiện vẫn tạo ra nhiều lỗi hơn các nhóm nhân quả, sở thích hay truy hồi đa bước.
2. **Tính đầy đủ của danh sách chưa ổn định:** khi câu hỏi yêu cầu liệt kê toàn bộ các mục, hệ thống vẫn có thể bỏ sót một số phần tử hoặc thêm vào những mục không có trong đáp án chuẩn.
3. **Node thói quen chưa được chứng minh mạnh mẽ:** thói quen có cơ sở biểu diễn hợp lý về mặt lý thuyết, nhưng các bộ dữ liệu hiện tại chưa cho thấy sự cải thiện rõ rệt như node hành động–kết quả hay node ý định trong các thí nghiệm nhắm đích.

Những hạn chế này chủ yếu nằm ở khâu lựa chọn bằng chứng, suy luận tất định và kiểm soát quá trình sinh câu trả lời, hơn là ở việc thiếu một loại fact mới.

5.1.4 Hướng phát triển

Các hướng phát triển có tác động cao nhất cho phiên bản tiếp theo nên tập trung vào tính hữu dụng thực tế, không chỉ tối ưu thêm một benchmark:

1. **Tình huống bộ nhớ cho tác nhân AI:** thử nghiệm CogMem trên các vết hội thoại dài của tác nhân AI thực tế, nơi node hành động–kết quả có thể lưu quan hệ “điều kiện – hành động công cụ – kết quả” thay vì chỉ dựa vào hội thoại mô phỏng ngắn.
2. **Kịch bản trợ lý hướng tương lai:** thiết kế các tình huống thực tế về kế hoạch chưa hoàn thành, kế hoạch bị hủy và kế hoạch được thực hiện để kiểm tra mạng Ý định trong vòng đời dài hơn.
3. **Nhật ký thói quen và hành vi lặp lại:** xây dựng tập dữ liệu dạng nhật ký nhiều tuần để kiểm tra mạng Thói quen trên các yếu tố như ngữ cảnh kích hoạt, sự gián đoạn thói quen và hành vi lặp lại, thay vì chỉ hỏi các fact đơn lẻ.
4. **Độ tin cậy trong triển khai thực tế:** bổ sung bộ suy luận thời gian, cơ chế kiểm soát tính đầy đủ của danh sách và hiệu chuẩn phủ định để giảm lỗi trong các ứng dụng trợ lý thực tế.

Nhìn chung, CogMem đã chứng minh rằng đồ thị nhận thức, bảo toàn bằng chứng gốc, truy hồi đa kênh và định tuyến thích ứng là đủ để tạo ra một API bộ nhớ dài hạn khả dụng; các điểm yếu còn lại nằm ở suy luận thời gian, liệt kê chính xác và bằng chứng đặc thù cho thói quen.

Báo cáo khép lại tại đây. Các phụ lục tiếp theo cung cấp thêm chi tiết về dữ liệu và cấu hình thí nghiệm.

PHỤ LỤC A. Knowledge Extraction Prompts (Retain Pipeline)

Phụ lục này trình bày các prompt cốt lõi của retain pipeline. Từ thời điểm hệ thống chuyển sang cơ chế trích xuất hai pass, Pass 1 và Pass 2 không còn đóng cùng một vai trò: Pass 1 xử lý ngữ cảnh mixed-speaker để nhận diện đủ các fact type, còn Pass 2 tập trung tinh chỉnh các fact mang tính persona của người dùng.

A.0.1 A.1. Prompt Trích xuất Pass 1 (Fact Extraction)

Pass 1 chịu trách nhiệm đọc bối cảnh hội thoại rộng và trích xuất đủ sáu fact type của CogMem, bao gồm cả *world* và *action_effect*. Đây là nơi hệ thống thu nhận các quy luật nhân quả, các sự kiện khách quan và các tín hiệu không thuần túy thuộc persona của người dùng.

System Prompt: Pass 1 Fact Extraction

```
You are a memory extraction assistant.
Extract durable facts from conversations
for long-term storage.
OUTPUT RULE: Respond ONLY with valid JSON
- no prose, no markdown fences.
Format: {"facts": [<fact>, ...]}
or {"facts": []} if nothing to store.
EVERY fact MUST have these REQUIRED fields:
- "fact_type": one of: world | experience |
  opinion | habit | intention | action_effect
- "what": core statement, under 80 words -
  must include: WHO did WHAT to/with WHAT.
- "entities": ALL named entities - people,
  places, orgs, tech tools, product names.
FACT TYPE GUIDE:
1. "world" - objective fact, not time-bound:
  job, role, skill.
2. "experience" - USER's personal past event
  at a specific time.
3. "opinion" - belief or preference; add
  "confidence": 0.0-1.0.
4. "habit" - repeating behavior; triggers:
  always, usually, every day.
5. "intention" - future plan or goal; add
  "intention_status":
  "planning"|"fulfilled"|"abandoned".
6. "action_effect" - causal relationship
```

```
(A causes B). Add "precondition",
"action", "outcome", "devalue_sensitive".
```

A.0.2 A.2. Prompt Tinh chỉnh Pass 2 (Persona-Focused Revision)

Pass 2 được dùng để rà lại các memory mang tính người dùng và loại bỏ nhiều hội thoại. Khác với Pass 1, pass này không nhằm tới việc trích xuất toàn bộ thể giới tri thức, mà ưu tiên giữ các fact cá nhân bền vững như experience, habit, intention và opinion.

System Prompt: Pass 2 Persona-Focused

```
You are a memory refinement assistant.
Your task is to filter and rewrite raw facts
to ensure they STRICTLY capture the USER's
long-term persona, history, and preferences.
OUTPUT RULE: Respond ONLY with valid JSON
- no prose, no markdown fences.
Format: {"facts": [<fact>, ...]}
or {"facts": []}.
RULES:
1. DELETE generic conversational filler, AI
   system prompts, and temporary state updates.
2. DELETE assistant actions (e.g. "Assistant
   summarized the meeting").
3. REWRITE facts to be User-centric. If it's
   about the USER doing, liking, or knowing
   something, keep it. If it's about the
   Assistant, drop it.
4. ONLY emit experience | habit | intention |
   opinion in this pass. World and
   action_effect stay in Pass 1.
```

PHỤ LỤC B. Evaluation and Reflection Prompts

Phụ lục này ghi lại các prompt ở tầng synthesis và evaluation. Đây là phần nối giữa evidence do recall pipeline trả về và câu trả lời cuối cùng của hệ thống, đồng thời cũng là nơi phục vụ chấm điểm tự động trong các benchmark chuẩn.

B.0.1 B.1. Prompt Tổng hợp Câu trả lời (Reflection & Generation)

Prompt generation nhận đầu vào là tập bằng chứng đã được recall. Ở kiến trúc hiện tại, *raw_snippet* có thể được đưa inline cùng mỗi memory khi policy generate yêu cầu thêm grounding nguyên văn; vì vậy, prompt nhấn mạnh việc trả lời chỉ từ evidence được cung cấp.

System Prompt: Answer Generation

```
You are an AI assistant designed to answer
the user's latest query using ONLY the
provided long-term memory evidence.
```

EVIDENCE RULES:

1. Base your answer COMPLETELY on the text listed under [EVIDENCE].
2. If the [EVIDENCE] contradicts your internal knowledge, you MUST trust the [EVIDENCE].
3. DO NOT state "Based on the evidence..." or "The text provided says...". Respond directly.
4. If the evidence does not contain the answer, say exactly: "I do not have enough specific memory to answer this question."

```
<--- BEGIN EVIDENCE --->
```

```
{evidence}
```

```
<--- END EVIDENCE --->
```

B.0.2 B.2. Prompt Chấm điểm Tự động (LLM-as-a-Judge)

Như đã trình bày trong Chương 4, đề án sử dụng phương pháp **LLM làm giám khảo** để đánh giá chất lượng câu trả lời của hệ thống. Mô hình giám khảo nhận vào ba thông tin: câu hỏi gốc, đáp án chuẩn và câu trả lời do CogMem sinh ra. Nhiệm vụ của nó là gán một điểm số từ 0,0 đến 1,0 và giải thích ngắn gọn lý do.

Các nguyên tắc chấm điểm cốt lõi bao gồm: ưu tiên độ chính xác về mặt ngữ nghĩa hơn là trùng khớp từng ký tự, chấp nhận khác biệt nhỏ về cách diễn đạt, và khoan dung với sai số một đơn vị trong các câu hỏi về thời gian.

Dưới đây là prompt đầy đủ được sử dụng cho mô hình giám khảo:

Prompt: LLM Judge – Chấm điểm câu trả lời

You are an evaluation judge for a memory system. I will give you a question, a correct answer, and a model response.

Score rubric (0.0-1.0):

- 1.0: correct and complete.
- 0.7-0.9: correct but slightly rephrased or minor detail missing.
- 0.3-0.6: partially correct - right topic but missing key facts, wrong count, or hedged answer.
- 0.0-0.2: wrong, fabricated, or completely missing the answer.

SPECIAL RULE FOR TEMPORAL QUESTIONS:

Do NOT penalize off-by-one errors for counts of days, weeks, or months (e.g., answering "18 days" when the label is "19 days" is still scored 1.0).

OUTPUT FORMAT:

Respond ONLY with a JSON object.

```
{"score": <float between 0.0 and 1.0>,  
  "reason": "<string>"}
```